Orleans University

Polytech Engineering school

AL & Big Data Lab Report

Prepared by:

Vincent Mwenda Mworia

Instructor:

Prof. Frédéric Ros

Date:

Tuesday 30th December 2025

Academic Year 2025/2026

Table of Contents

Lab 1 - K-Means Clustering

Objective

This experiment aims to explore how the K-Means clustering algorithm groups data into clusters based on similarity.

The lab focuses on:

- Understanding the K-Means process.
- Applying it to a simple 2D dataset.
- Finding the best number of clusters using the Elbow method.
- Testing it on a real-world dataset (Iris).
- Observing its strengths and weaknesses.

Theory

K-Means is an unsupervised learning algorithm used to discover natural groupings (clusters) in data. It partitions the data into K clusters such that points within a cluster are more similar to each other than to those in other clusters.

The algorithm follows these main steps:

1. **Start:** Choose *K* random points as initial centroids.
2. **Assign:** Allocate each data point to the nearest centroid.
3. **Update:** Recalculate centroids as the average of all points in each cluster.
4. **Repeat:** Continue until cluster centers no longer move significantly.

The algorithm minimizes the **sum of squared distances** between each point and its assigned centroid (called *inertia*).

Code

The K-Means clustering code used in this experiment is available at the following GitHub repository: K-Means_Code

Implementation on Synthetic Data

To visualize how K-Means behaves, a simple 2D dataset with three clusters was created using make_blobs().

The model successfully identified three distinct clusters in the dataset. The centroids, represented by black “X” symbols, indicate the center of each cluster. Each color in the plot corresponds to a separate cluster, clearly showing separation between the data groups and confirming that the K-Means algorithm effectively grouped similar data points together.

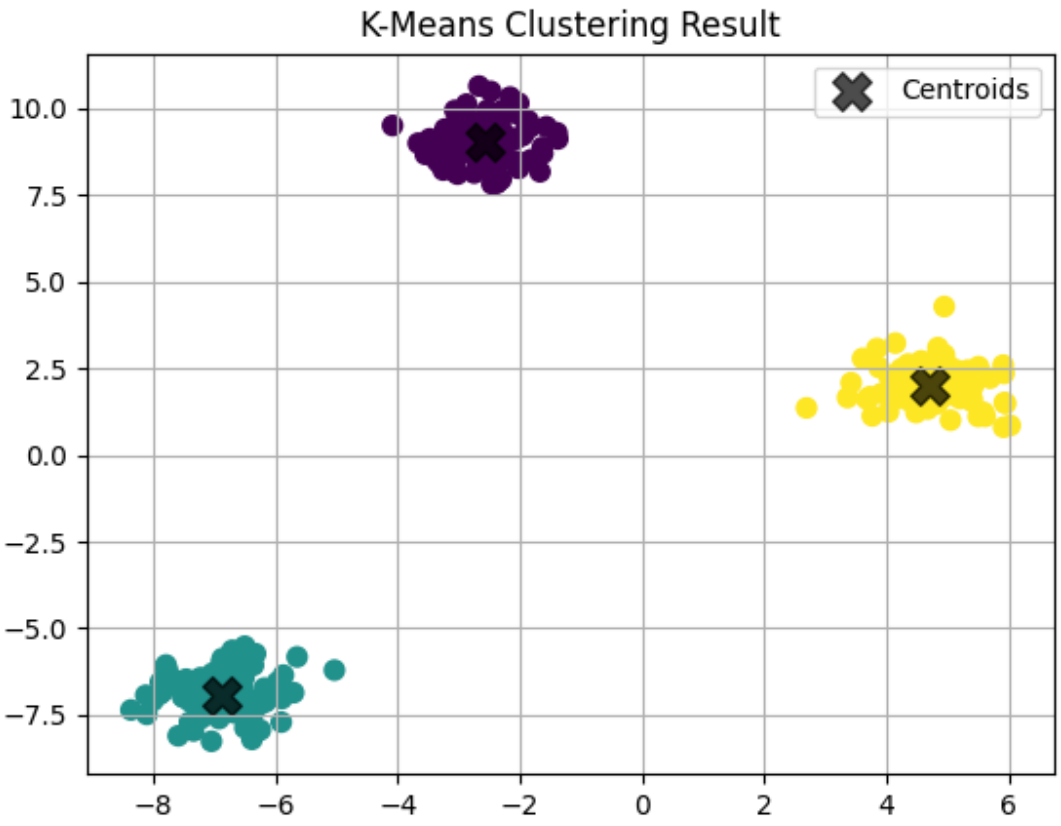


Figure 1. *K-Means clustering result on a 2D synthetic dataset.*

Determining the Optimal Number of Clusters

The Elbow Method was used to determine the most appropriate value for K. It measures how the total inertia (within-cluster sum of squares) changes as K increases.

As K grows, inertia naturally decreases because clusters become smaller and data points move closer to their respective centroids. However, beyond a certain point, the improvement becomes minimal hence additional clusters start capturing random noise rather than meaningful structure. Moreover, using too many clusters can lead to overfitting, where the model fits the data too precisely and loses generalization ability.

Therefore, the optimal value of K is chosen at the “elbow point” where adding more clusters no longer results in a significant decrease in inertia. This choice balances accuracy with simplicity, preventing the model from overfitting.

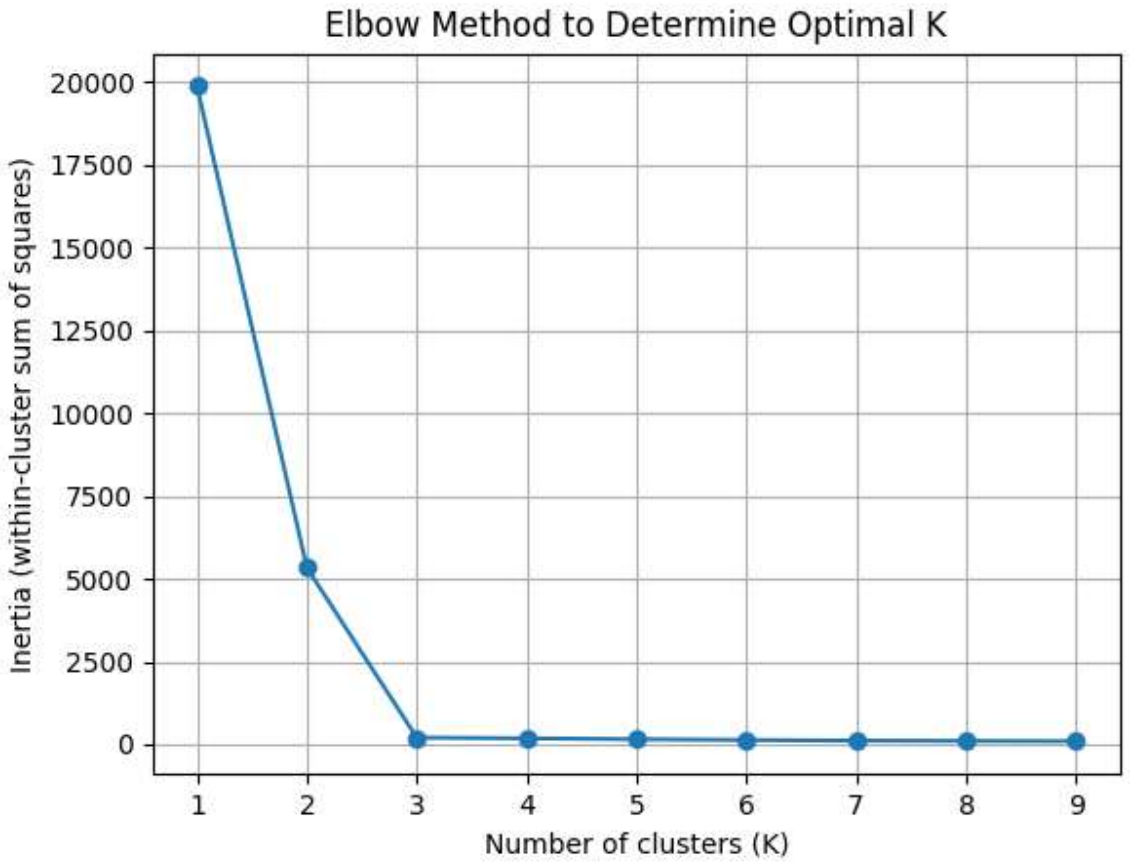


Figure 2. *Elbow Method used to determine the optimal number of clusters.*

Applying K-Means to the Iris Dataset

The algorithm was tested on the Iris flower dataset, which contains 150 samples and four numerical features.

Before clustering, the data was standardized using StandardScaler() to ensure that all features contributed equally to the distance calculations. This transformation scales each feature to have a mean of 0 and a standard deviation of 1, making the variables comparable across dimensions.

Principal Component Analysis (PCA) was also applied to reduce the data from four dimensions to two. This dimensionality reduction technique preserves most of the variance in the dataset while enabling effective visualization of the clustering results in a 2D plot.

K-Means successfully identified three major groups, roughly corresponding to the three Iris species. However, some overlap occurred between the purple cluster (versicolor) and the yellow cluster (virginica), indicating that the K-Means algorithm can struggle when clusters overlap or are not linearly separable in feature space.

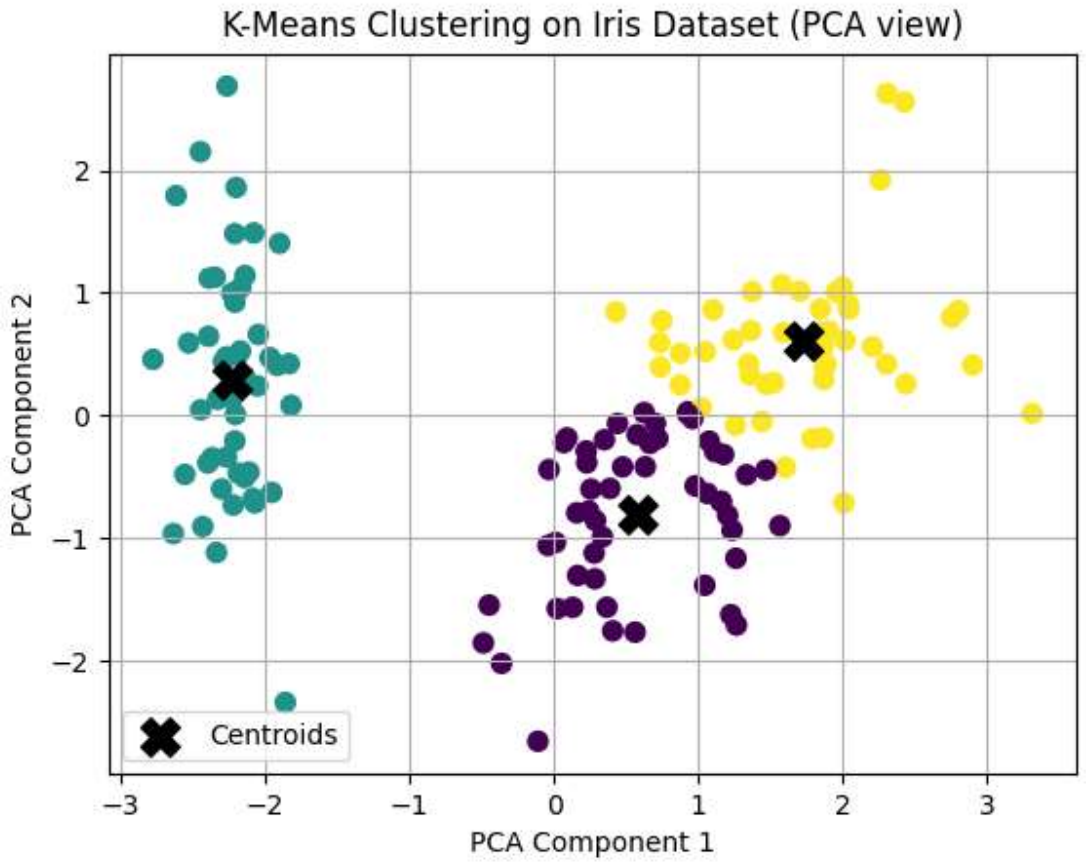


Figure 3. *K-Means clustering on the Iris dataset (2D PCA projection).*

Exploring Limitations with Non-Spherical Data

To examine how K-Means performs on irregularly shaped datasets, two synthetic datasets were generated. One was shaped like interlocking moons and another like concentric circles.

In both cases, the algorithm failed to correctly separate the curved clusters because K-Means relies on Euclidean distance to assign points to the nearest centroid. As a result, it naturally forms circular cluster boundaries. This assumption works well for spherical data but performs poorly when clusters are non-linear or non-spherical. Consequently, K-Means misclassifies several data points in regions where the clusters bend or overlap.

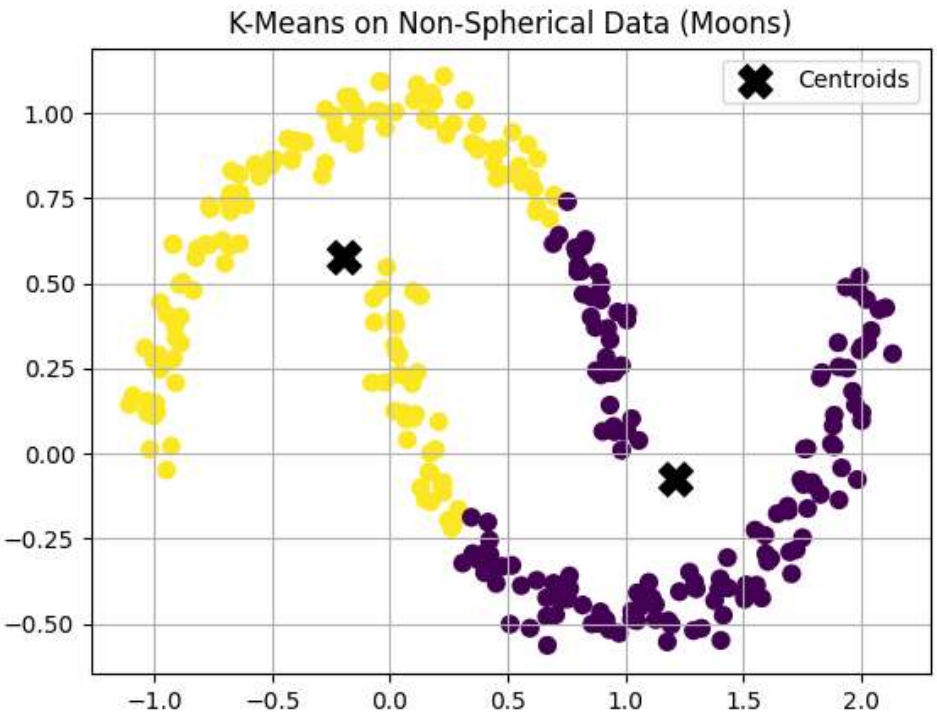


Figure 4. *K-Means clustering on non-spherical data (two interlocking moons)*

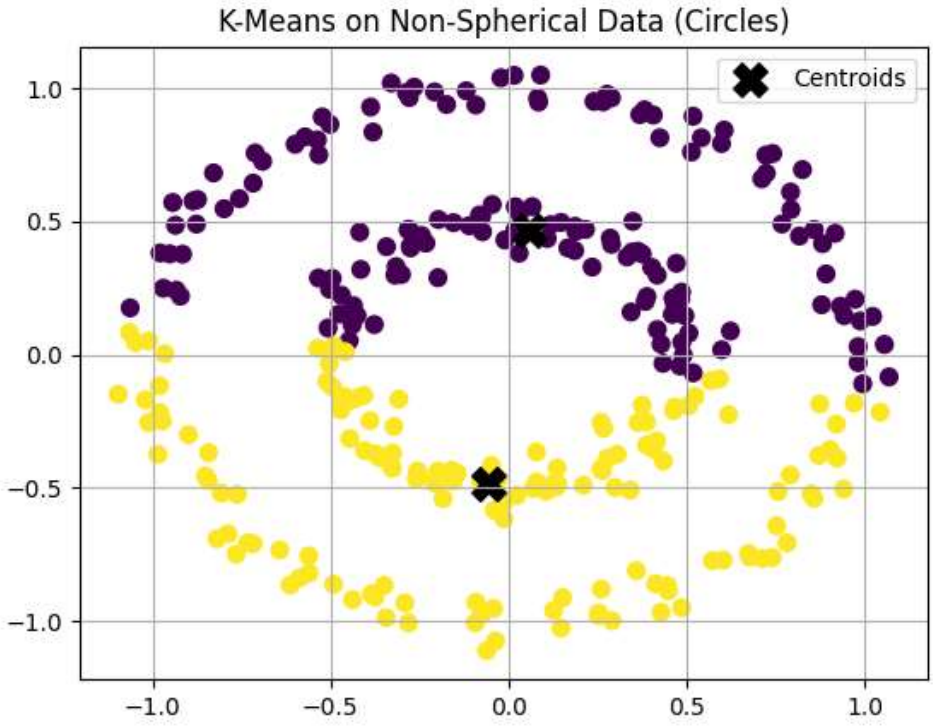


Figure 5. *K-Means clustering on non-spherical data (concentric circles)*

Cluster Evaluation (Silhouette Score)

The Silhouette Score measures how well each data point fits within its assigned cluster compared to other clusters. The score ranges from -1 to 1, where a value closer to 1 indicates that the data points are well clustered, while lower or negative values suggest poor separation or incorrect clustering. The highest silhouette value was observed at K = 2, indicating that the data is most cleanly divided into two distinct clusters. However, K = 3 still provides a reasonable structure that aligns with the known Iris species groupings.

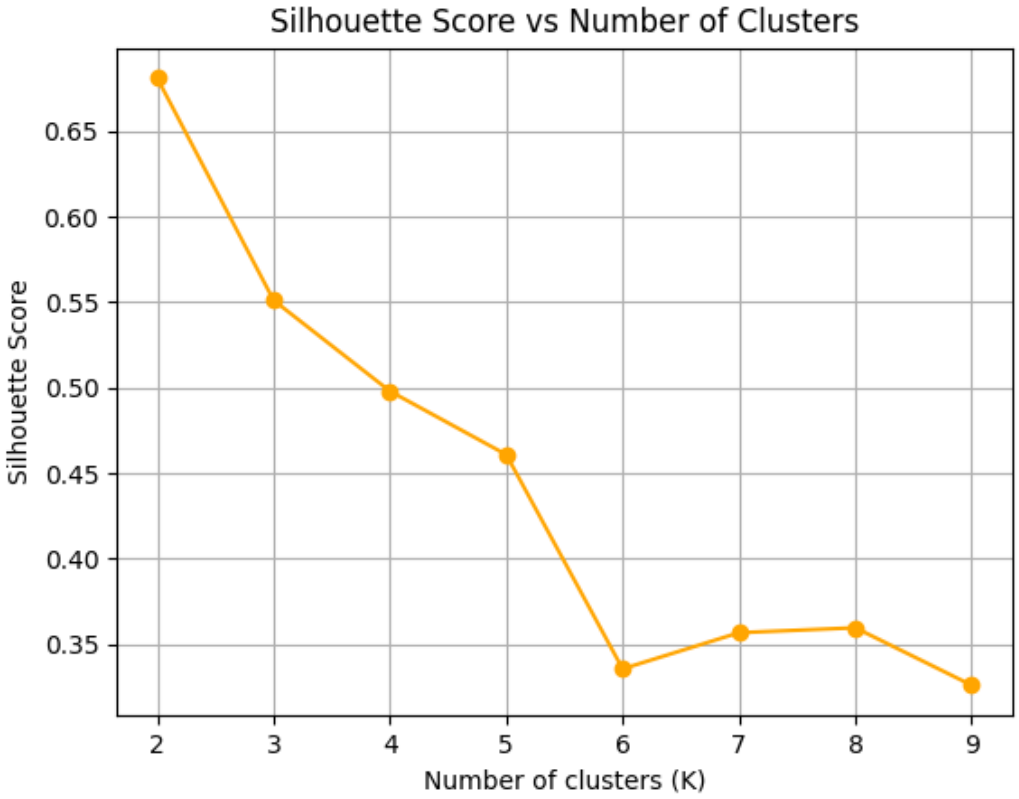


Figure 6. *Silhouette Score versus number of clusters for the Iris dataset.*

Questions for Discussion

1. What are the limitations of K-Means?

   K-Means requires specifying K in advance, assumes spherical and equally sized clusters, and performs poorly on irregular, overlapping, or outlier-heavy data.

2. What happens if clusters are not spherical?

   K-Means forces circular boundaries, so it struggles with curved or uneven clusters, often splitting points from the same true group or grouping distant points together incorrectly.

3. Does K-Means always find the best result?

   No. K-Means can get stuck in a local minimum, meaning it may find a suboptimal solution depending on where the centroids were initially placed.

4. What if centroids are initialized poorly?

   Poor initialization can lead to unbalanced or incorrect clusters and may also cause slower convergence. The K-Means++ method reduces this issue by choosing well-spaced initial centroids for more stable and accurate results.

Conclusion

K-Means is one of the most widely used and intuitive algorithms for identifying structure in data. It performs well on compact, well-separated clusters but struggles with irregular, overlapping, or non-spherical shapes. Overall, it serves as an excellent starting point for clustering analysis, though more advanced algorithms such as DBSCAN or Gaussian Mixture Models (GMMs) are better suited for complex or non-linear datasets.

K-Means is most commonly applied in customer segmentation, image compression, document grouping, and market analysis, where cluster boundaries are approximately convex and the number of groups can be estimated beforehand.

Lab 2 - DBSCAN Clustering

Objective

This experiment aims to explore how the DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm discovers clusters by local density and identifies outliers. The lab focuses on:

- Understanding DBSCAN's core ideas and parameters (**eps**, **min_samples**) and the roles of core, border, and noise points.
- Applying DBSCAN to a non-spherical 2D dataset (two moons) and examining parameter sensitivity.
- Comparing DBSCAN with K-Means on the same data to highlight shape and outlier handling differences.
- Running DBSCAN on the Iris dataset (with PCA for visualization) and counting clusters vs. noise points.
- Noting practical limitations (single global **eps**, varying densities) and when alternatives like HDBSCAN may be preferable.

Theory

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that groups together points closely packed in space, while marking isolated points as noise. It defines clusters based on two parameters:

- ε (eps): the neighborhood radius around a point.
- min_samples: the minimum number of points required to form a dense region.

Conceptually, the method considers a neighborhood of radius ε around each point in the dataset. If this neighborhood contains at least min_samples points, the point is classified as a **core point**. Points that lie within the ε-neighborhood of a core point but do not themselves meet the density requirement are referred to as **border points**. Points that are neither core nor border points are designated as **noise** or **outliers**.

Clusters are then constructed by connecting core points whose neighborhoods overlap, thereby forming continuous regions of high density. Unlike K-Means, DBSCAN does not require specifying the number of clusters and can identify clusters of arbitrary shapes, making it robust for non-spherical data and outlier detection. However, its performance depends strongly on suitable ε and min_samples values.

Code

The DBSCAN clustering code used in this experiment is available at the following GitHub repository: DBSCAN_Code

Implementation on Synthetic Non-Spherical Data

A two-dimensional non-spherical dataset was generated using the make_moons() function with 300 samples and a noise level of 0.05. This dataset was selected because it cannot be effectively separated by algorithms that assume spherical clusters, such as K-Means. The DBSCAN algorithm was applied with parameters ε = 0.2 and min_samples = 5 to identify the two crescent-shaped clusters.

These parameter values were selected empirically through visual inspection of the distance distribution, as they provided a balance between accurately detecting the two main clusters and minimizing the number of points classified as noise.

A smaller ε value tends to fragment the data into numerous small clusters, often increasing the number of noise points and resulting in underfitting, where the model is too rigid to capture the full cluster structure. Conversely, an excessively large ε may merge distinct regions into a single cluster, leading to overfitting, where the model becomes too general and fails to preserve meaningful boundaries. Similarly, a low min_samples value allows small or spurious clusters to form, while a high min_samples value enforces stricter density requirements that may exclude legitimate but less dense clusters.

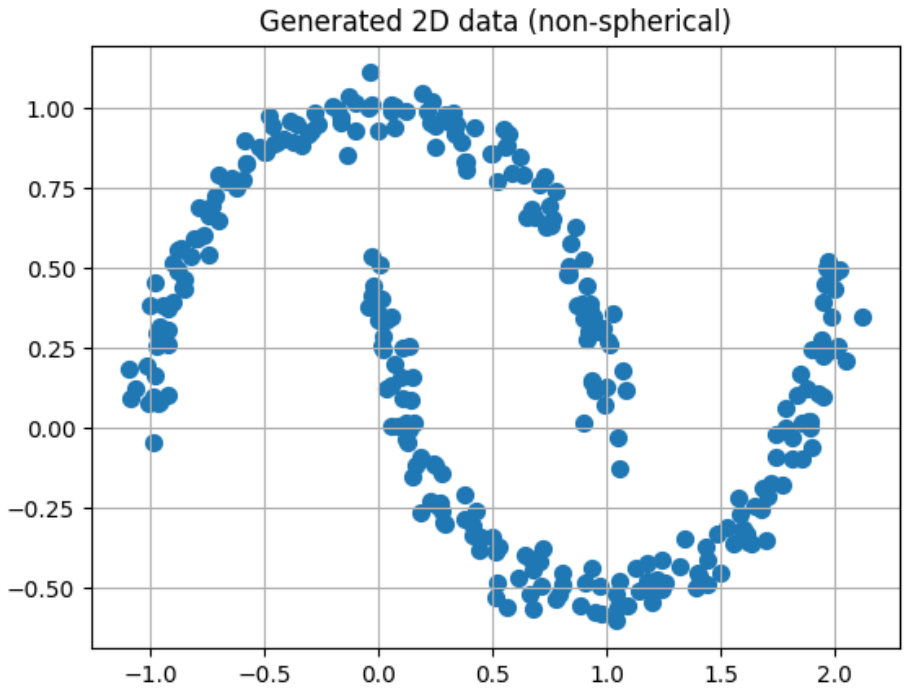


Figure 7. *Generated two-dimensional non-spherical dataset (two-moons) created using the make_moons() function.*

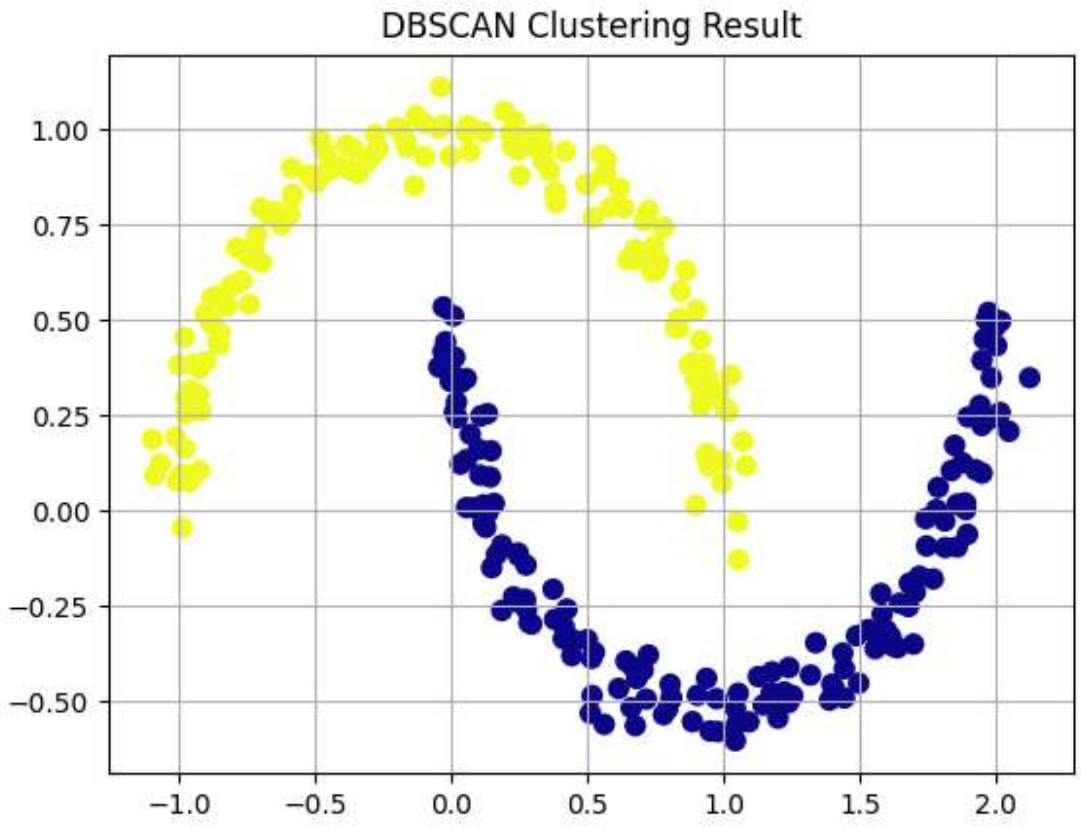


Figure 8. *DBSCAN clustering result on the two-moons dataset using parameters ε = 0.2 and min_samples = 5.*

Parameter Sensitivity

To examine the influence of DBSCAN's parameters, experiments were conducted by varying ε and min_samples independently.

As ε increased, the neighborhood radius expanded, causing the algorithm to transition from detecting multiple small, fragmented clusters (when ε was small) to merging the two crescent-shaped structures into a single cluster (when ε was large). Smaller ε values led to excessive fragmentation and noise (underfitting), while larger ε values caused overgeneralization and cluster merging (overfitting).

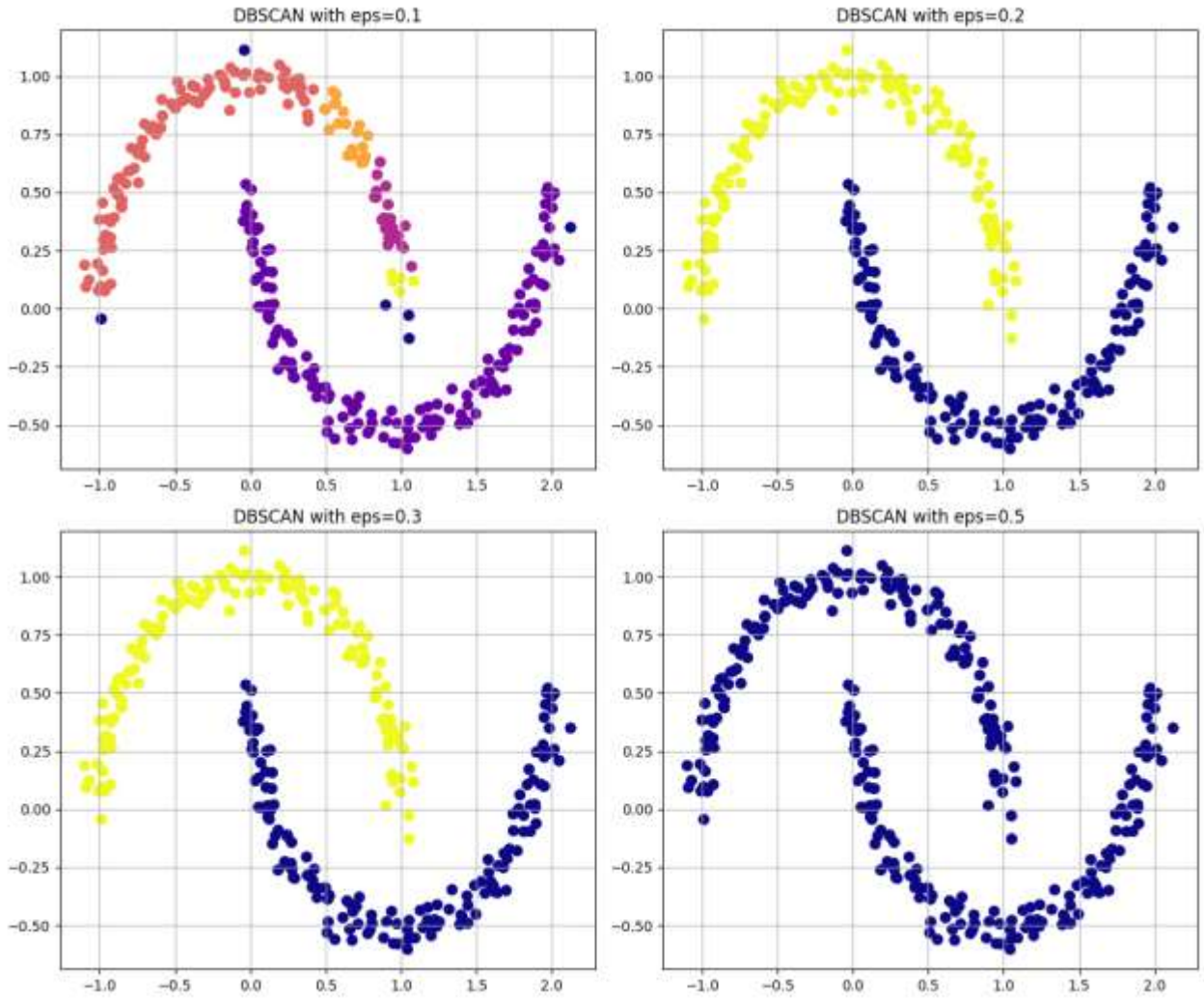


Figure 9. *DBSCAN results with varying ε values (0.1, 0.2, 0.3, 0.5) and fixed min_samples = 5.*

Similarly, increasing min_samples made the density criterion progressively stricter, reducing the number of core points and increasing noise, especially at higher values. Smaller min_samples values (e.g., 3–5) produced stable and coherent clusters, while very large values (e.g., 20) led to overly conservative clustering that excluded many valid points at the edges of each crescent.

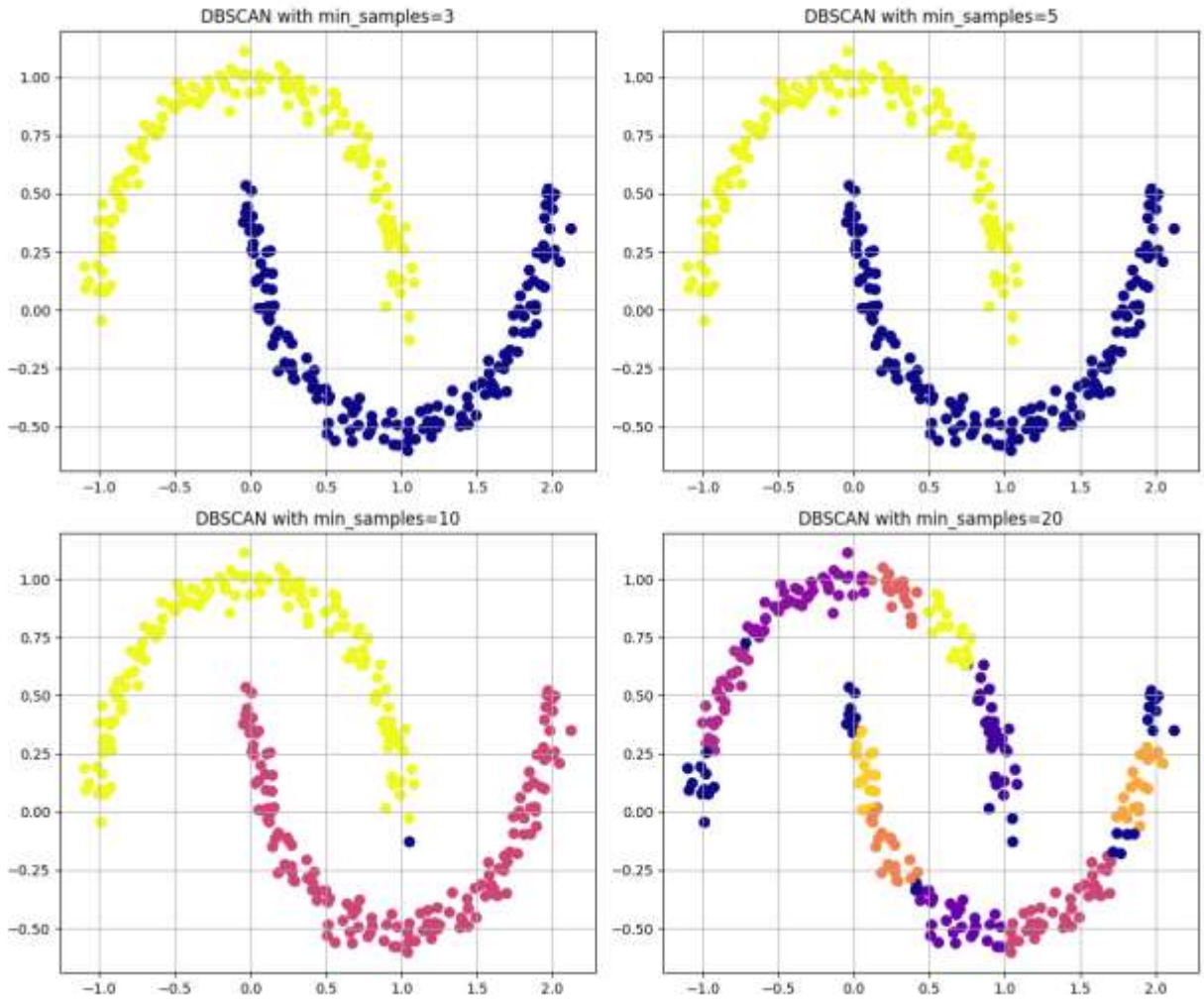


Figure 10. *DBSCAN results with varying min_samples values (3, 5, 10, 20) and fixed ε = 0.2.*

An intermediate configuration (ε = 0.2, min_samples = 5) yielded the most coherent and balanced clustering.

Comparing to K-Means

The same non-spherical two-moons dataset was also clustered using K-Means with k = 2. K-Means divided the data into two regions using straight, centroid-based boundaries that do not follow the curved structure of the dataset. As a result, several points near the junction of the two crescents were misclassified due to the algorithm's assumption of spherical cluster geometry and its reliance on Euclidean distance to fixed centroids.

DBSCAN, in contrast, accurately identified the two curved clusters without requiring the number of clusters in advance. Its density-based approach captured the true manifold of the data and correctly separated the clusters while labeling a few boundary points as noise. This result demonstrates DBSCAN's ability to model irregular and non-convex shapes that K-Means cannot effectively represent.

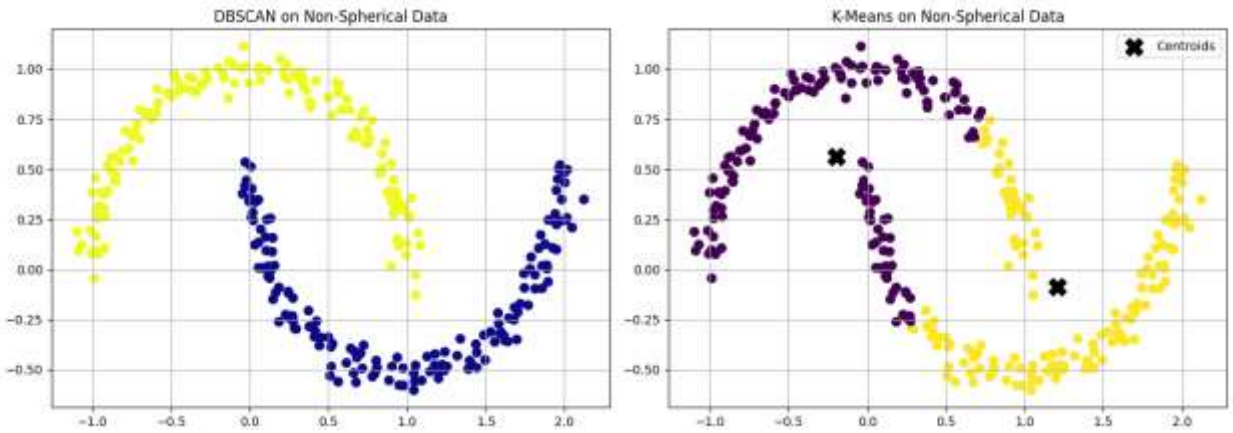


Figure 11. *Comparison of clustering performance on the non-spherical two-moons dataset between DBSCAN (ε = 0.2, min_samples = 5) on the left and K-Means (k = 2) on the right.*

DBSCAN on the Iris Dataset (Real-World Multidimensional Data)

The Iris dataset was used to evaluate DBSCAN on real-world, multi-dimensional data. The feature variables were standardized using StandardScaler to ensure equal contribution of all attributes to the distance metric. Dimensionality reduction was then performed using Principal Component Analysis (PCA) to project the four-dimensional data into two principal components for visualization purposes only.

DBSCAN was applied to the standardized data using parameters ε = 0.6 and min_samples = 5. The algorithm identified several compact regions of high density corresponding to potential clusters while labeling certain ambiguous samples as noise. The resulting two-dimensional PCA projection revealed visually separable groups that broadly corresponded to the Iris species, although some overlap occurred between Iris versicolor and Iris virginica, which are known to have similar feature distributions.

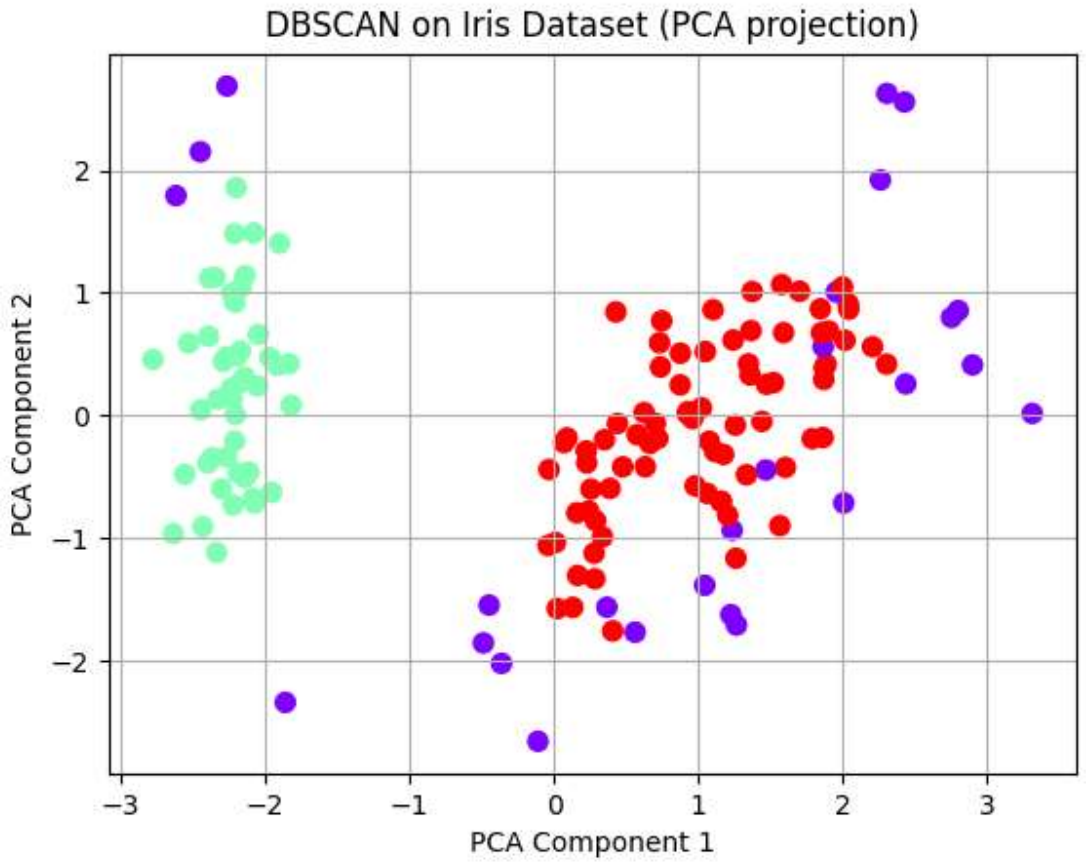


Figure 12: *DBSCAN clustering of the Iris dataset visualized after PCA projection (ε = 0.6, min_samples = 5).*

Data with Varying Density

To examine DBSCAN's behaviour when cluster densities differ, a synthetic dataset was generated using the make_blobs() function with cluster standard deviations of [0.5, 1.5, 0.3], creating three groups of unequal density. The algorithm was executed with parameters ε = 0.8 and min_samples = 5.

The resulting plot shows that DBSCAN correctly identifies the two denser clusters but partially merges or misclassifies points in the sparser, more diffuse cluster. This occurs because DBSCAN applies a single global density threshold (ε), which cannot simultaneously accommodate both dense and sparse regions. Consequently, points belonging to low-density clusters are either labeled as noise or absorbed into neighboring clusters.

This experiment highlights DBSCAN's sensitivity to varying density and motivates the use of adaptive approaches such as HDBSCAN, which can handle non-uniform density distributions by adjusting neighborhood thresholds locally.

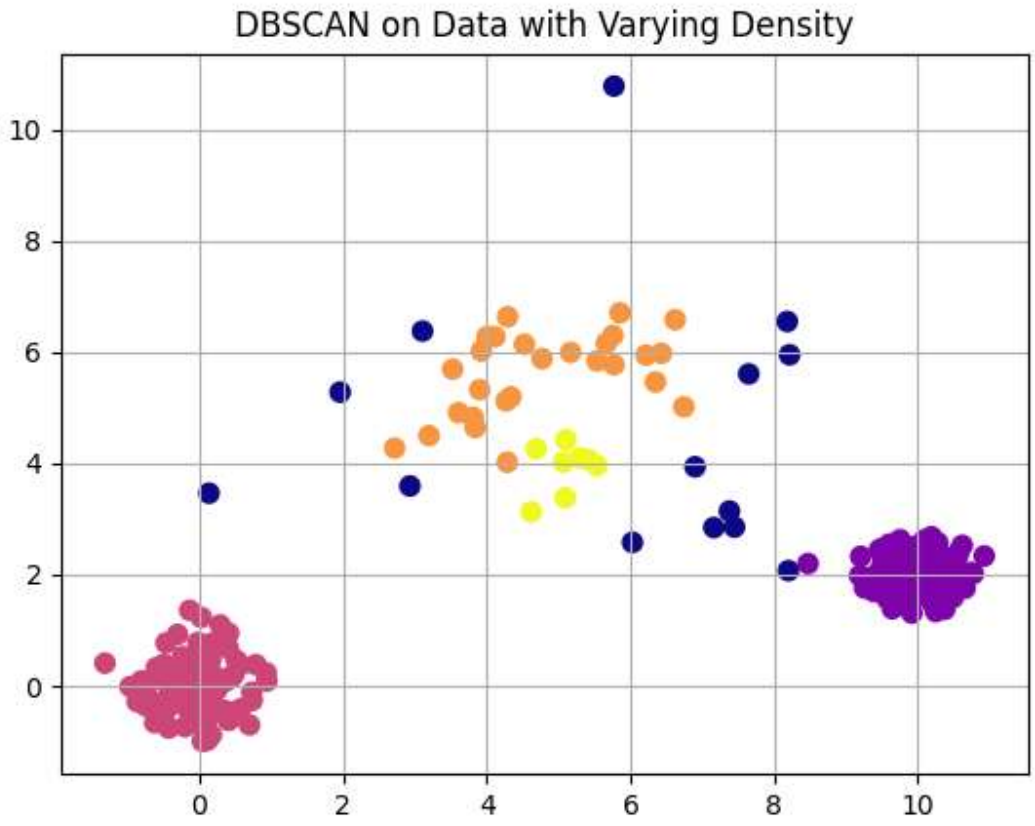


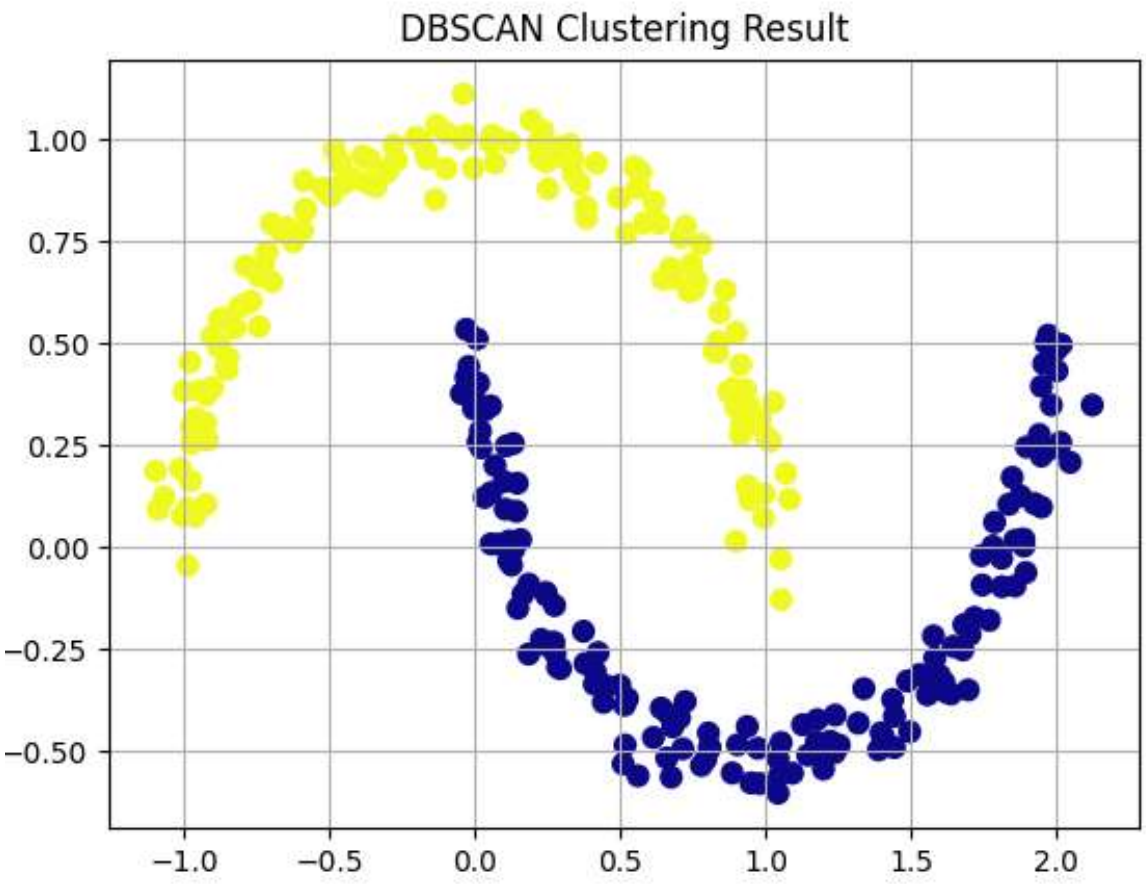
Figure 13: *DBSCAN clustering of the Iris dataset visualized after PCA projection (ε = 0.6, min_samples = 5).*

Using Silhouette Score and Adjusted Rand Index to Evaluate Clustering Quality

To quantitatively assess DBSCAN's performance, two standard clustering evaluation metrics were computed on the two-moons dataset: the Silhouette Score and the Adjusted Rand Index (ARI).

The Silhouette Score of 0.328 indicates moderate cluster separation. Points within each cluster are generally closer to one another than to points in other clusters. However, some points near the edges of the crescent-shaped regions lie close in Euclidean distance to points in the neighboring cluster. Because the Silhouette metric evaluates separation based purely on straight-line distances, it penalizes these boundary points even though they are correctly assigned. This results in a moderate score, which is expected for non-spherical data such as the two-moons structure.

The Adjusted Rand Index (ARI) achieved a perfect score of 1.000, confirming that DBSCAN's cluster assignments exactly matched the true underlying structure of the dataset. This result demonstrates DBSCAN's strength in identifying complex, non-spherical cluster shapes that are difficult for centroid-based algorithms such as K-Means to represent accurately.



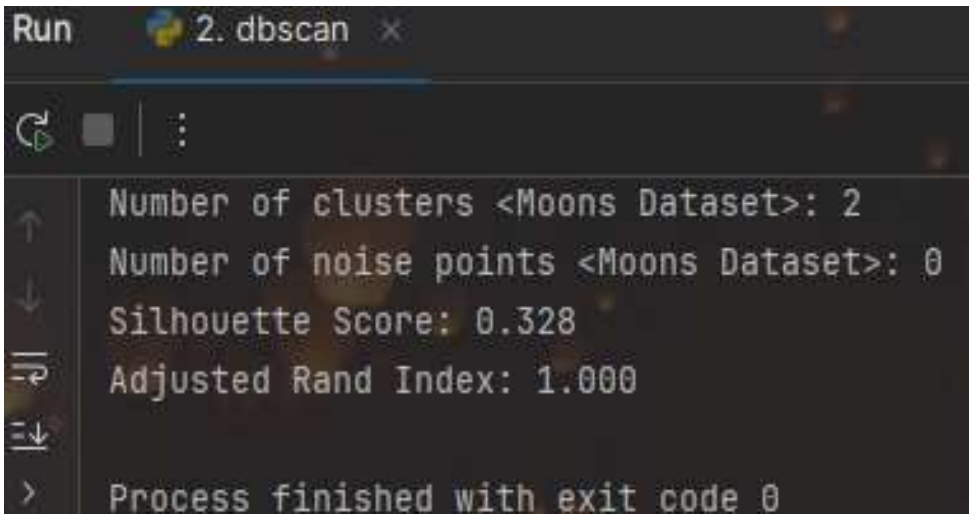


Figure 14: *Quantitative evaluation of DBSCAN performance on the two-moons dataset (ε = 0.2, min_samples = 5).*

HDBSCAN on Varying-Density Data

To address DBSCAN's limitation in handling clusters of unequal density, the same synthetic dataset with varying cluster standard deviations [0.5,1.5,0.3] was analyzed using HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise). Unlike DBSCAN, HDBSCAN does not rely on a single global ε value; instead, it builds a hierarchy of density levels and extracts stable clusters based on their persistence across scales.

With parameters min_cluster_size = 15 and min_samples = 5, HDBSCAN successfully detected three main clusters and a small number of noise points. Compared to DBSCAN, which struggled to separate the sparser middle group, HDBSCAN adaptively adjusted to local density variations, resulting in more accurate cluster boundaries and reduced misclassification of low-density points as noise.

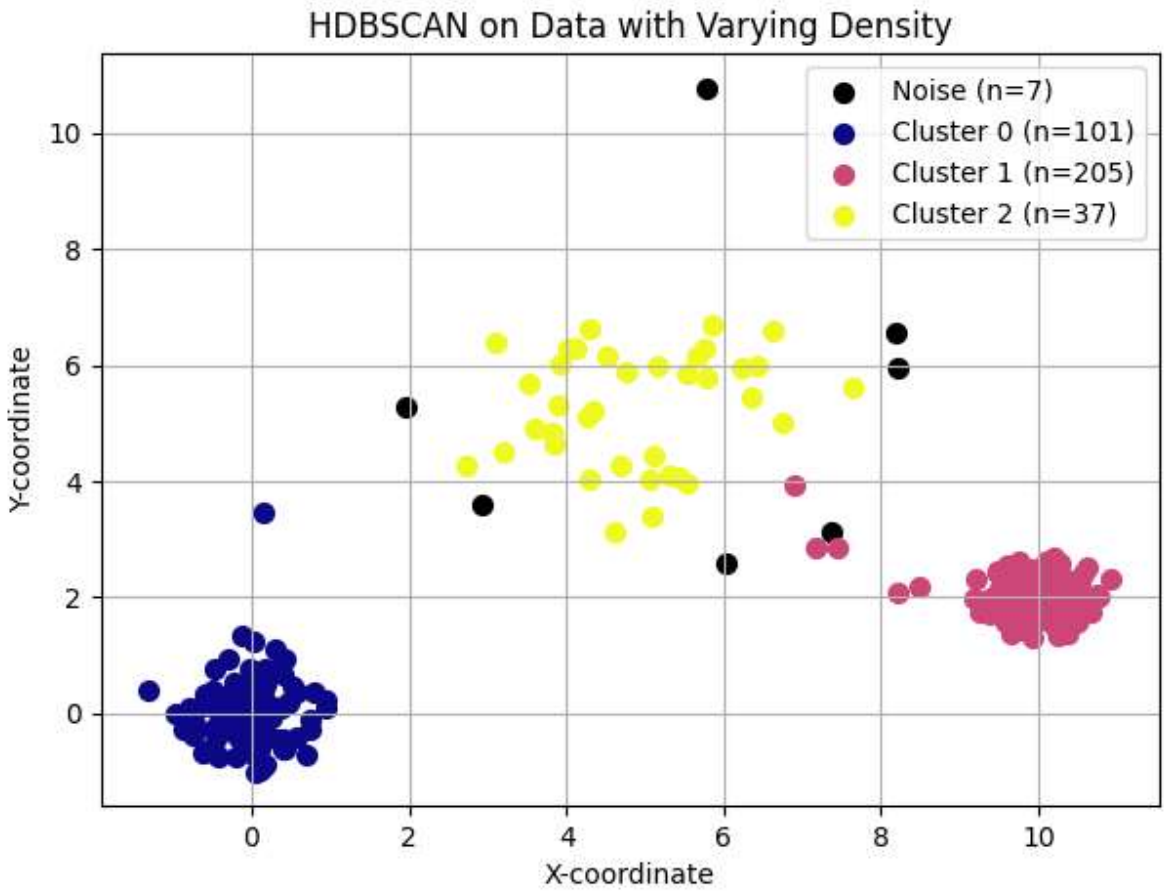


Figure 15: *HDBSCAN clustering on data with varying density (min_cluster_size = 15, min_samples = 5).*

Questions for Discussion

1. What are the advantages of DBSCAN over K-Means?

   DBSCAN does not require the number of clusters to be specified in advance and can discover clusters of arbitrary shape, such as non-spherical or irregular structures that K-Means cannot represent. It is also able to detect outliers and label them as noise.

2. What are the limitations of DBSCAN?

   DBSCAN is sensitive to its parameters ε (epsilon) and min_samples. Small changes in these values can significantly affect the resulting clusters. It also struggles with datasets containing clusters of varying density, since a single global ε cannot accommodate both dense and sparse regions simultaneously. In addition, DBSCAN's performance can degrade on very large or high-dimensional datasets because distance measures become less meaningful in such spaces.

3. Can DBSCAN identify noise in the Iris dataset?

   Yes. DBSCAN labels points that do not belong to any sufficiently dense region as noise (label = −1).

4. Would DBSCAN work well in high-dimensional datasets?

   Not very well. In high-dimensional spaces, the notion of "distance" becomes less informative due to the curse of dimensionality, making it difficult to define meaningful density thresholds. As a result, DBSCAN may either classify most points as noise or merge clusters incorrectly. Dimensionality reduction techniques such as PCA or t-SNE are often applied before using DBSCAN on high-dimensional data.

Conclusion

Through experiments on both synthetic and real-world datasets, DBSCAN was shown to effectively identify clusters of arbitrary shape without requiring the number of clusters to be specified in advance. Its ability to label low-density points as noise further distinguishes it from centroid-based algorithms such as K-Means, which assume spherical clusters and assign all data points to a group.

However, DBSCAN's performance was found to be highly dependent on the choice of its parameters, ε and min_samples, and it struggled when clusters exhibited varying densities. In contrast, HDBSCAN addressed this limitation by adaptively determining density thresholds, yielding more stable results and better separation of clusters with differing densities.

DBSCAN is most commonly applied in scenarios where data exhibit irregular shapes, variable densities, or contain noise that should not be forced into clusters. It is widely used in spatial data analysis, such as identifying geographic regions of interest or detecting hotspots in GPS data, as well as in anomaly and outlier detection in network security, manufacturing, and sensor data. DBSCAN is also applied in image processing, genomics, and astronomical data analysis, where natural groupings are complex and not easily captured by algorithms assuming spherical clusters or uniform densities.

Lab 3 - Hierarchial Clustering

Objective

This experiment aims to explore how hierarchical clustering groups data by progressively merging similar observations to form a tree-structured representation of clusters. The lab focuses on:

- Understanding the principles of agglomerative hierarchical clustering (the bottom-up approach), which is the most commonly used method.
- Visualizing how clusters are formed through a dendrogram, which reveals the order and distance at which points or groups are merged.
- Examining the effect of different linkage criteria (single, complete, average, and Ward) and how they influence cluster shapes and merging behavior.
- Applying hierarchical clustering to both synthetic 2D data and a real-world dataset (Iris), to perform dimensionality reduction using PCA for visualization.
- Investigating methods for extracting clusters from a dendrogram and comparing hierarchical clustering with K-Means and DBSCAN on the same dataset.

Theory

Hierarchical clustering is an unsupervised learning technique that groups data based on similarity while preserving the structure of how clusters are formed. Unlike partitioning algorithms such as K-Means, it does not require specifying the number of clusters in advance. Instead, hierarchical clustering organizes data into a multi-level hierarchy, often visualized as a tree-like structure called a dendrogram.

There are two main approaches:

1. **Agglomerative clustering (bottom-up):**
   This method starts by treating each data point as its own cluster. At each step, the two closest clusters are merged based on a chosen distance metric and linkage criterion. This process continues until all points form a single cluster. Agglomerative clustering is the most widely used approach and the one explored in this lab.
2. **Divisive clustering (top-down):**
   This approach begins with all data points in a single cluster and iteratively splits clusters into smaller ones. Although conceptually opposite to the agglomerative method, divisive clustering is less common due to its higher computational cost.

To decide which clusters to merge (or split), hierarchical clustering relies on a linkage criterion, which determines how distances between clusters are calculated. The most common linkage methods include:

1. **Single linkage:** Single linkage calculates the distance between two clusters by looking at the closest pair of points across them. This approach often creates long chain-like clusters because a single close connection can link two otherwise distant groups.
2. **Complete linkage:** Complete linkage measures the distance between the farthest points in each cluster. This method produces compact and well-separated clusters, but it can sometimes split clusters that lie close to each other.
3. **Average linkage:** Average linkage uses the mean distance between all pairs of points across the two clusters. This approach provides a balanced behaviour, avoiding the chaining effect of single linkage and the strict compactness of complete linkage.
4. **Ward linkage:** Ward linkage merges clusters in a way that causes the smallest increase in total within-cluster variance. It often creates the most compact and well-structured clusters, which makes it a common default choice in many applications.

Hierarchical clustering is most useful when the goal is not only to cluster data but also to understand the relationships between clusters, visualize how they form, and explore structure at multiple granularity levels. However, it can be computationally expensive for large datasets and may be sensitive to noise depending on the linkage method used.

Code

The Hierarchical Clustering Code used in this experiment is available at the following GitHub repository: Hierarchial_Clustering_Code

Synthetic Data and Clustering

To begin the experiment, a simple two-dimensional dataset was generated using the make_blobs function. The dataset contains 200 samples grouped around three well-separated cluster centers, each with a moderate standard deviation. This setup provides a clear structure that allows hierarchical clustering to reveal how points gradually merge into larger groups.

The scatter plot of the generated data shows three compact and visually distinct clusters. Each point represents an individual sample in the dataset. Because the clusters are well defined and do not overlap, this synthetic example offers an ideal starting point for observing how the hierarchical algorithm progressively builds the clustering hierarchy during the merging process.

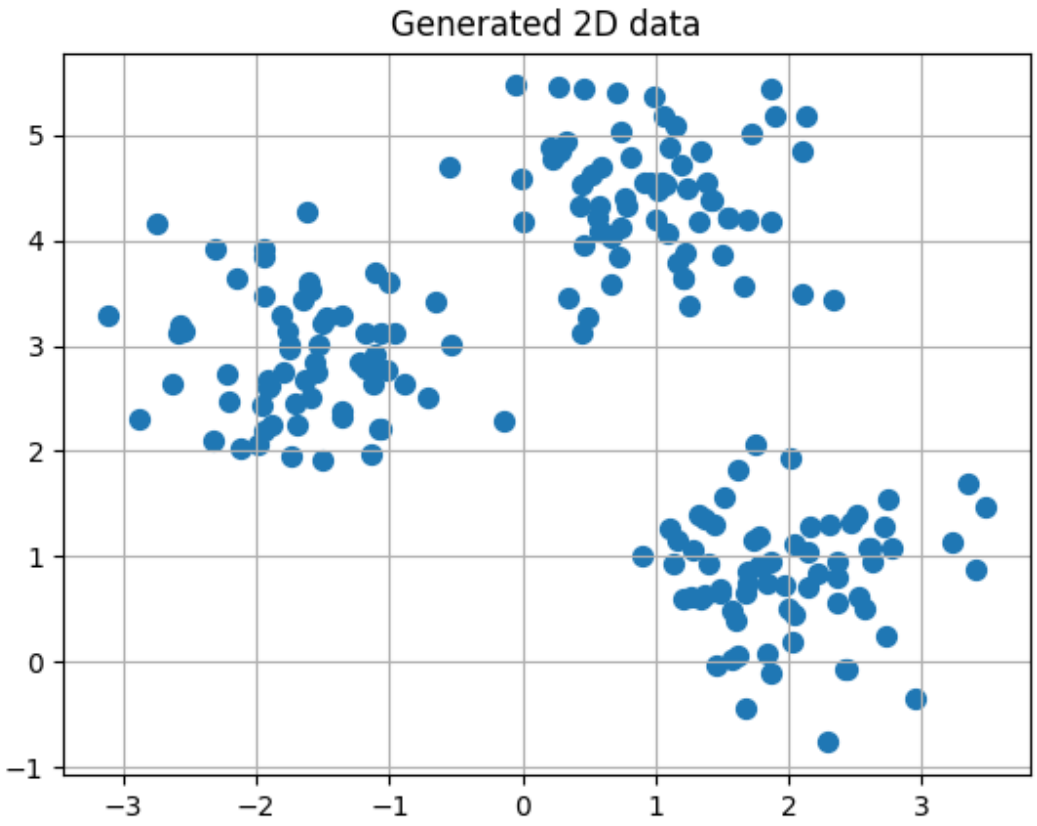


Figure 16. *Generated 2D synthetic dataset.*

Agglomerative Clustering with Dendrogram

After generating the synthetic dataset, the next step involves applying agglomerative hierarchical clustering and visualizing how the clusters merge over time. The algorithm begins with each data point treated as an individual cluster. At each iteration, it identifies the two clusters that lie closest to each other according to the chosen linkage method (Ward's method) and merges them. This repeated merging gradually builds the full hierarchical structure.

The dendrogram provides a visual summary of this merging process. Each horizontal line in the diagram represents a merge between two clusters, and the height of the line reflects the distance at which the merge occurs. Lower merges indicate pairs of points or clusters that lie close together, while higher merges correspond to groups that are more dissimilar. By examining the overall structure of the dendrogram, it becomes possible to see how the three main clusters form and at what distances they join.

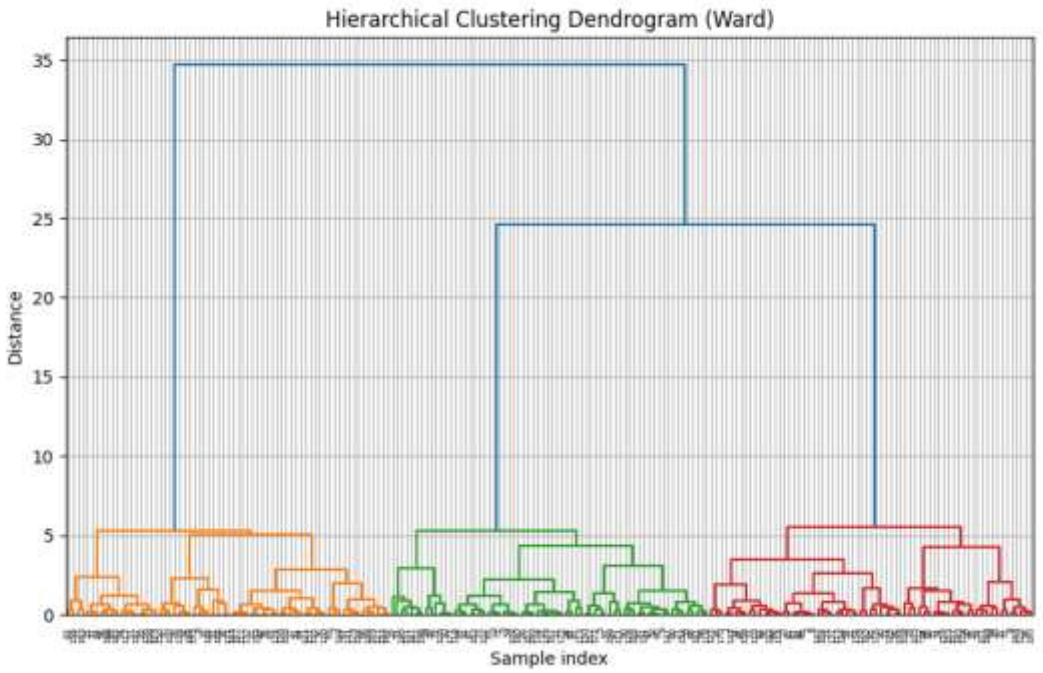


Figure 17. *Hierarchical clustering dendrogram (Ward linkage).*

Cluster Extraction

Once the hierarchical structure is built, the next step is to extract a specific number of clusters from the dendrogram. In this experiment, the agglomerative algorithm uses Ward linkage and forms three clusters, which matches the natural grouping in the synthetic dataset. By specifying n_clusters = 3, the algorithm stops the merging process at the appropriate level and assigns each data point to one of the resulting clusters.

The scatter shows the final clustering outcome. Each color represents one of the three clusters identified by the algorithm. The model clearly separates the data into the same groups visible in the original dataset, and the cluster boundaries align well with the natural structure. This result confirms that hierarchical clustering can accurately recover well-defined groups and provides a transparent merging path through the dendrogram.

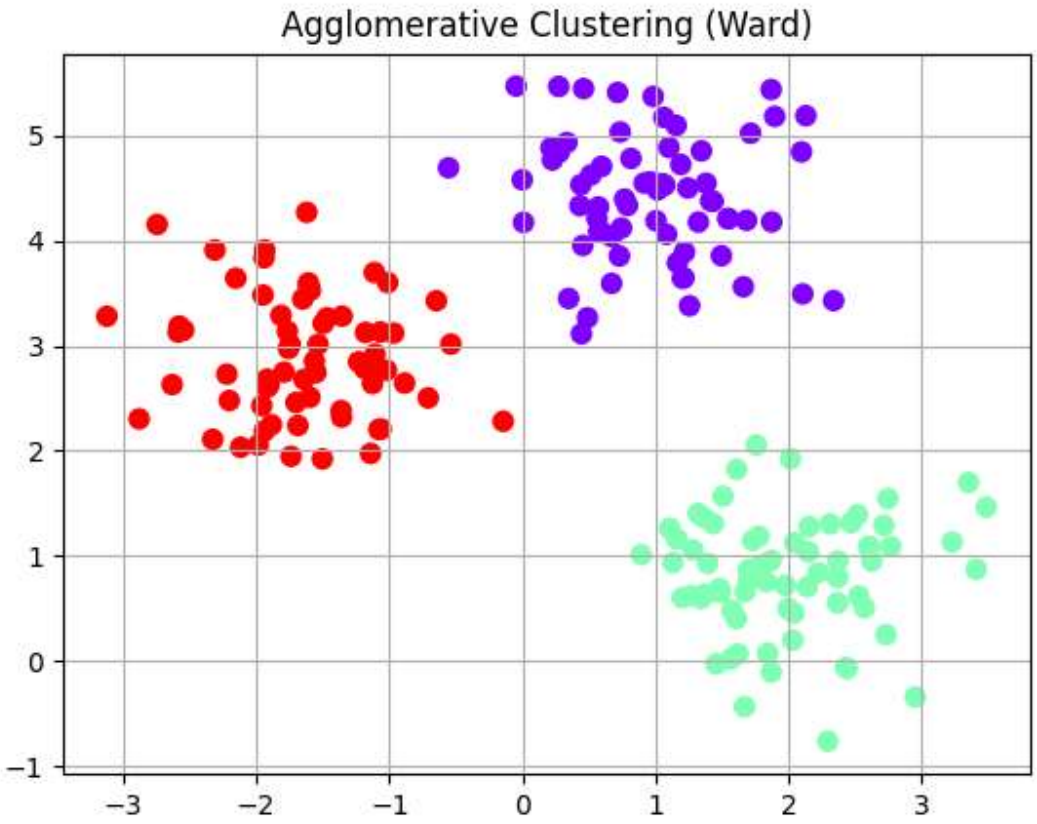


Figure 18. *Agglomerative clustering result using Ward linkage.*

Comparing Linkage Methods

Different linkage methods create noticeably different clustering results because each method uniquely calculates the distance between clusters. In this part of the lab, four linkage strategies (single, complete, average, and Ward) were applied to the same synthetic dataset from the earlier sections. Each method attempted to merge clusters based on similarity, but each one used a different definition of distance, which led to variations in cluster shapes and boundaries. These differences showed how strongly the choice of linkage influenced the structure of the final clustering result.

Figure 19 shows how the algorithm partitions the data under each linkage method. Single linkage often produces elongated, chain-like clusters because it merges groups as soon as any pair of points lies close together. Complete linkage behaves more conservatively by considering the farthest points between clusters, which results in compact and well-separated groups but can sometimes split clusters that sit close to one another. Average linkage balances these two extremes by merging clusters based on the mean distance between all point pairs across them. Ward linkage typically generates the most compact and stable clusters because it minimizes the growth of within-cluster variance at every step.

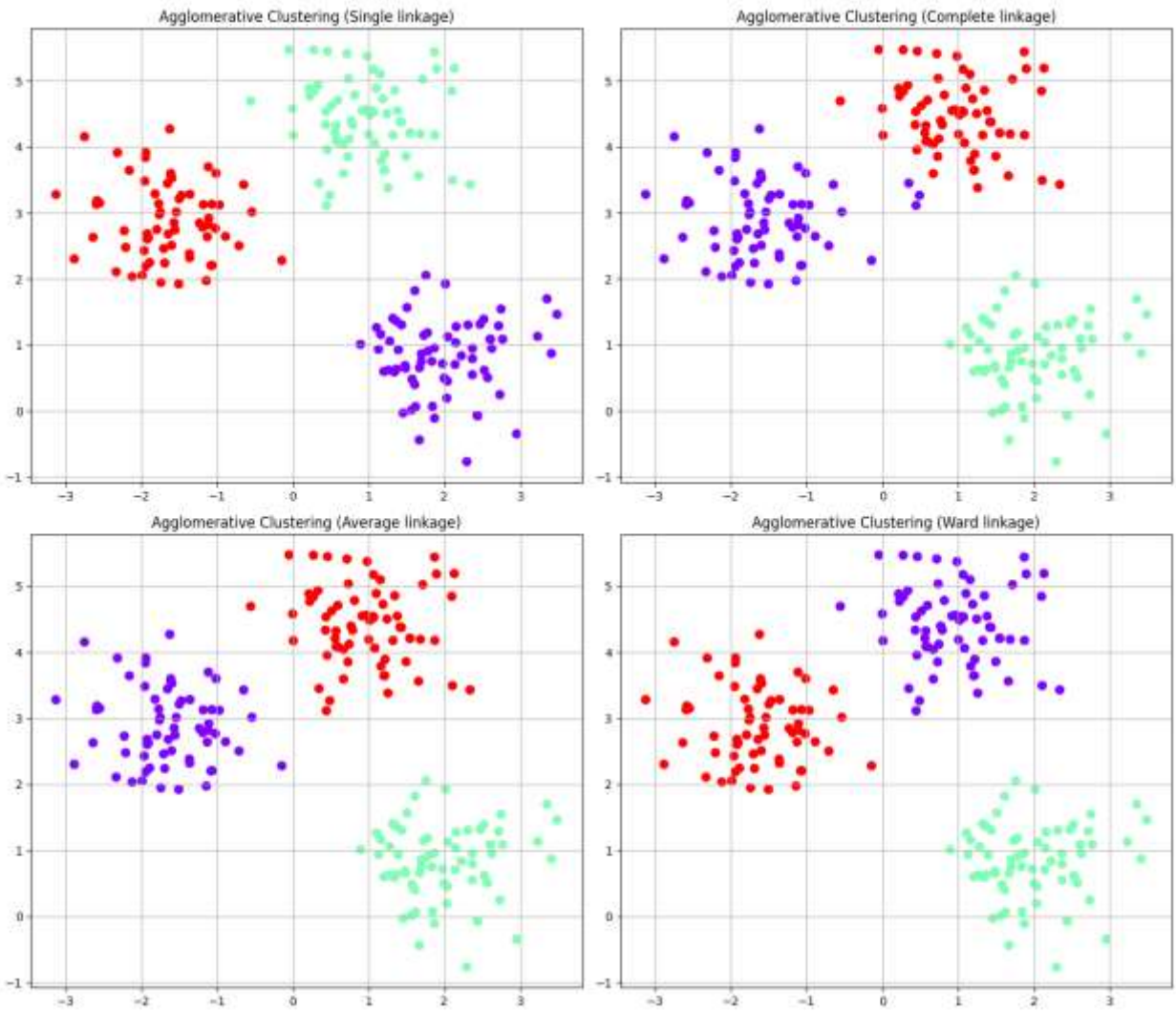


Figure 19. *Clustering results using single, complete, average, and Ward linkage methods.*

The dendrograms in Figure 20 further illustrate how each linkage method builds the cluster hierarchy. The height of the merges and the structure of the branches vary from method to method, showing how different distance interpretations influence the merging order. These differences demonstrate that the choice of linkage plays a significant role in shaping the final clustering structure, especially when the data contains subtle patterns or when cluster boundaries are not sharply defined.

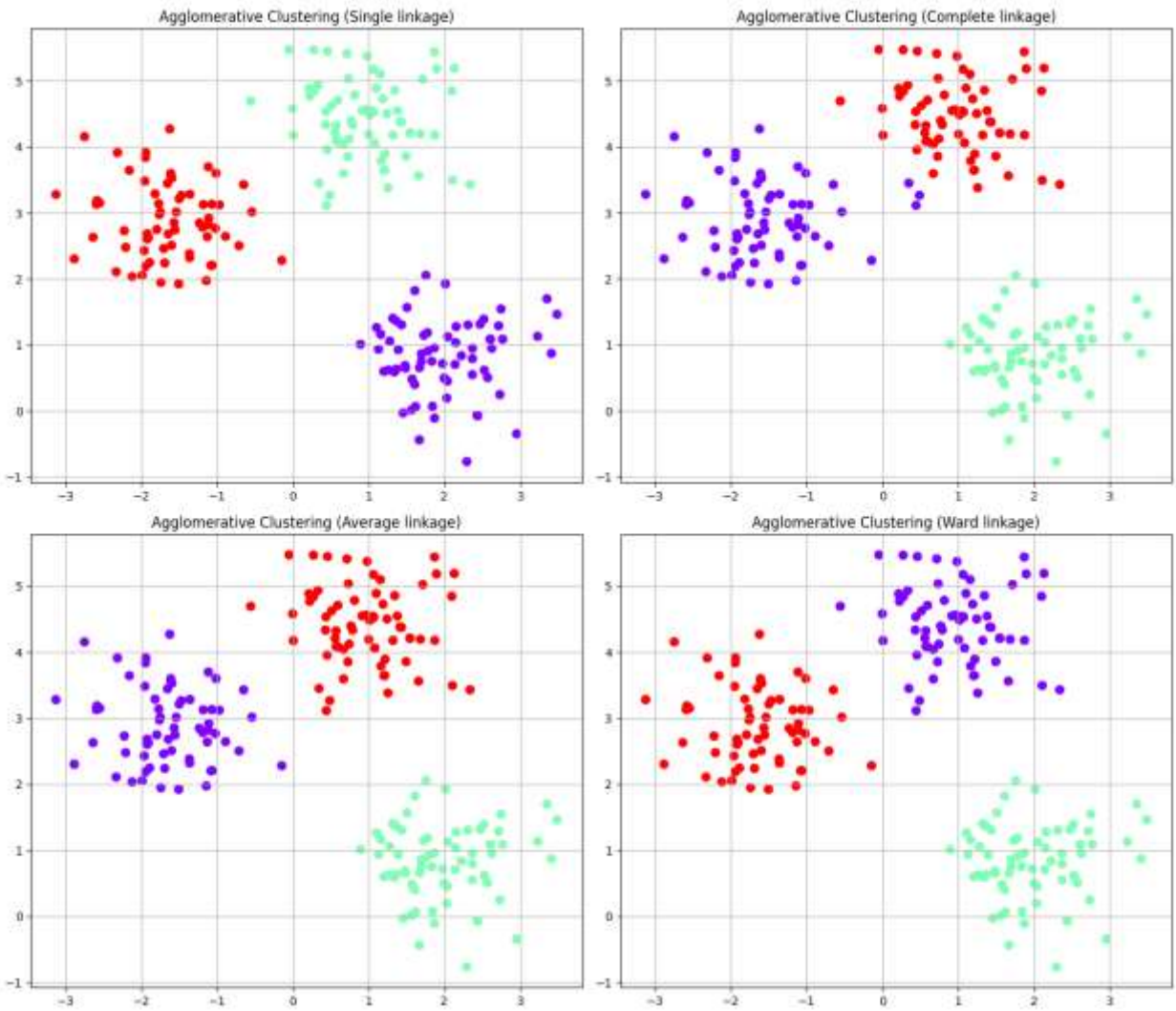


Figure 20. *Dendrograms for each linkage method.*

Iris Dataset (2D Projection with PCA)

The experiment used the Iris dataset to evaluate hierarchical clustering on real-world, multi-dimensional data. This dataset contains 150 flower samples with four numerical features that describe sepal and petal dimensions. Before running the clustering algorithm, the experiment scaled all features with Standard Scaler so that each variable contributed equally to the distance computations.

After scaling, the experiment applied Principal Component Analysis (PCA) and reduced the data to two principal components. This transformation preserved most of the variance in the dataset while providing a convenient two-dimensional representation for visualization. The clustering step then used agglomerative hierarchical clustering with Ward linkage and requested three clusters, which corresponds to the three known Iris species.

The figure below shows the clustering result projected onto the first two principal components. The three clusters appear as distinct groups in the PCA space, and their structure broadly matches the expected species separation. One cluster aligns clearly with Iris setosa, while the other two clusters overlap more, reflecting the known similarity between Iris versicolor and Iris virginica. This behavior agrees with observations from previous labs, where algorithms also found it harder to separate these two species.

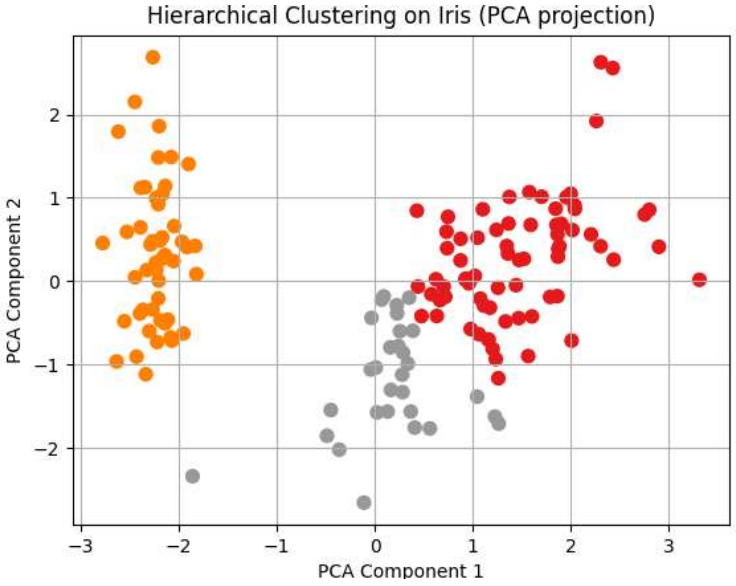


Figure 21. *Hierarchical clustering on the Iris dataset visualized in the first two principal components.*

The figure below presents the dendrogram obtained from the scaled Iris data using Ward linkage. The diagram reveals several meaningful merge levels and confirms the presence of three main groups, while also showing how individual samples join these groups at different distances. The dendrogram therefore adds an extra layer of interpretability beyond the 2D scatter plot.

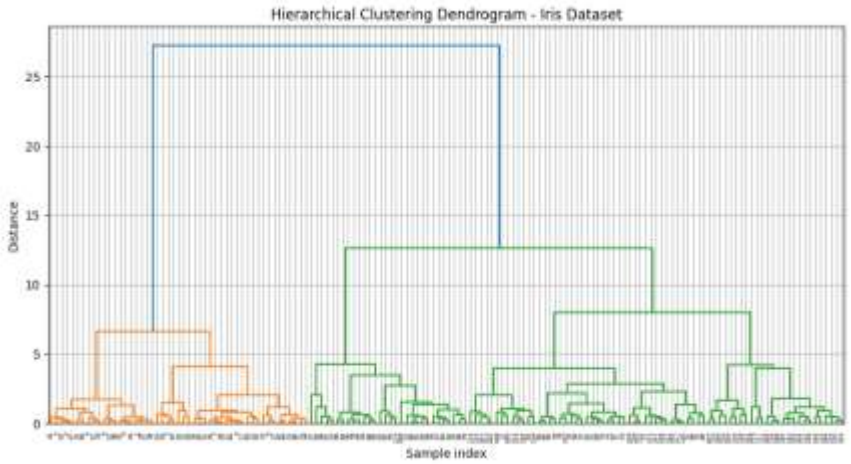


Figure 22. *Dendrogram for the Iris dataset using Ward linkage.*

Extracting Clusters Using Distance Thresholds

To explore the hierarchical structure in more detail, the experiment used the fcluster function to extract clusters by cutting the dendrogram at specific distances. This approach makes it possible to control the level of granularity in the clustering result. A smaller cut height produces more clusters by stopping the merging process early, while a larger cut height merges points into broader groups. The figure below shows the clusters obtained when the dendrogram was cut at a distance of 10. The algorithm returned three clusters, which aligns with the natural structure of the synthetic dataset.

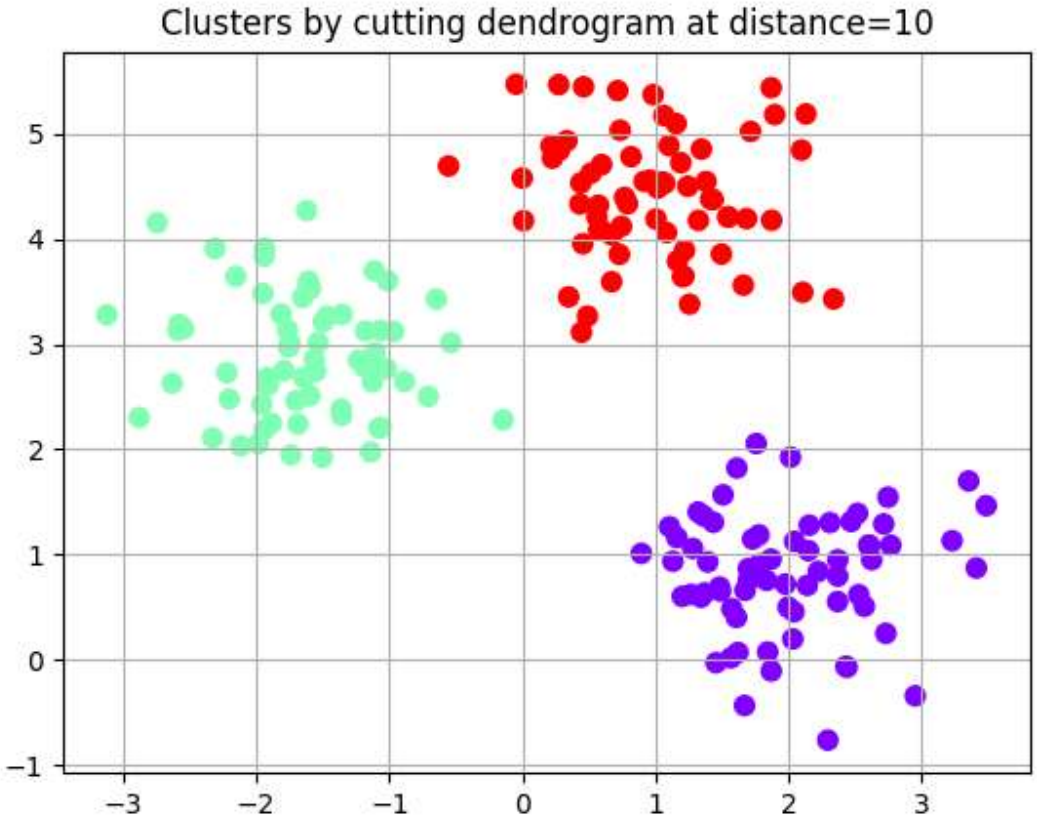


Figure 23. *Clusters extracted by cutting the dendrogram at a distance threshold of 10.*

Effect of Different Distance Thresholds on Clustering

To understand how the threshold influences the number of clusters, the experiment applied four distance values: 5, 10, 15, and 20. Each value produced a different number of clusters, ranging from several small groups at low thresholds to a nearly unified structure at high thresholds. These results show how hierarchical clustering reveals different layers of structure depending on where the dendrogram is cut. The figure below illustrates how the clustering result evolves across the four thresholds and highlights the sensitivity of the method to the choice of cut height.

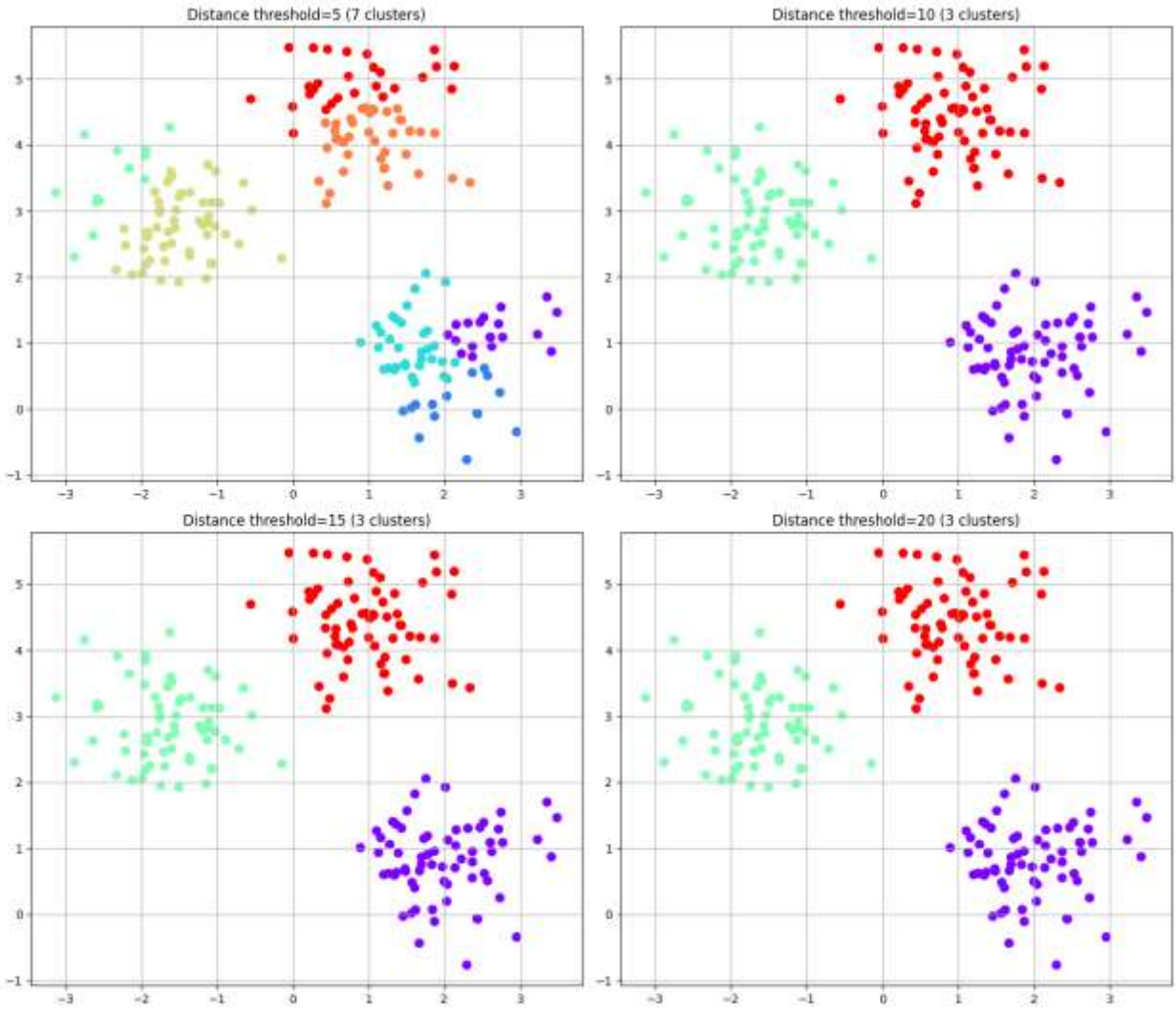


Figure 24. *Clustering results for distance thresholds of 5, 10, 15, and 20.*

Comparing Hierarchical Clustering with K-Means and DBSCAN

The comparison shows that each algorithm defines clusters differently and therefore produces distinct interpretations of the same dataset. K-Means works well when clusters are roughly spherical and similar in size, as it partitions the data using distances to fixed centroids. DBSCAN relies on local density, allowing it to capture arbitrary shapes and identify noise, but its performance depends strongly on the choice of parameters and can degrade when cluster densities vary. Hierarchical clustering takes a distance-based merging approach that preserves the relationships between points across multiple scales, enabling meaningful clusters to be selected directly from the dendrogram.

The side-by-side results show how each algorithm interprets the same dataset under their respective assumptions. K-Means separates the data into three compact, centroid-based clusters, which aligns well with the underlying blob structure. Hierarchical clustering with Ward linkage

produces a very similar partition, recovering the same three natural groups using a distance-based merging strategy. In contrast, DBSCAN groups all points into a single cluster and identifies no noise, indicating that the dataset does not contain strong density variations for DBSCAN to separate. This comparison illustrates how cluster geometry and density structure influence the outcome of each algorithm and why no single method behaves consistently across all datasets.

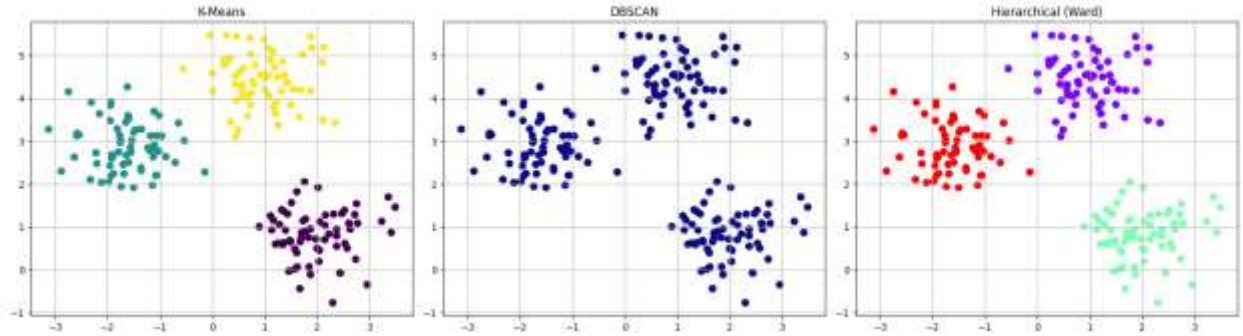


Figure 25. *Comparison between K-Means, DBSCAN, and hierarchical clustering.*

Questions for Discussion

1. What are the advantages of hierarchical clustering?
   Hierarchical clustering works well on small to medium-sized datasets because it does not require specifying the number of clusters in advance and provides a dendrogram that reveals the full merging history, allowing you to explore the data's structure at multiple levels of granularity with a level of interpretability that many other clustering methods cannot offer.
2. When is it better to use single or Ward linkage?
   Single linkage is most effective when the data contain elongated or irregular patterns that require a flexible chaining structure, whereas Ward linkage offers a stronger choice when the objective is to obtain compact, well-defined, and clearly separated clusters by controlling the growth of within-cluster variance.
3. What are the limitations compared to K-Means or DBSCAN?
   Hierarchical clustering becomes increasingly expensive on large datasets because it must compute and store a full distance matrix, and unlike DBSCAN it cannot naturally detect noise or adapt to variations in density, which makes it more sensitive to outliers and less practical than methods such as K-Means or DBSCAN when the data scale or density structure becomes complex.
4. How can we automatically determine the number of clusters?
   A dendrogram allows you to estimate the number of clusters by choosing a horizontal cut where large vertical gaps reveal natural separations in the data, and this decision can be supported by techniques such as inconsistency coefficients or distance thresholds, which

help identify a meaningful cut height without requiring prior knowledge of how many clusters should be extracted.

Conclusion

In conclusion, hierarchical clustering provides a powerful and interpretable framework for understanding the structure of data at multiple resolutions. Unlike K-Means, it does not require specifying the number of clusters beforehand, and unlike DBSCAN, it does not depend on density thresholds or struggle with datasets of mixed densities. Instead, it offers a complete hierarchical representation that can be explored visually through a dendrogram, making it especially valuable for exploratory data analysis and for domains where understanding relationships between clusters is as important as the clusters themselves.

However, hierarchical clustering also has limitations: it can be computationally expensive for large datasets, sensitive to noise, and influenced by the choice of linkage method. The experiments in this lab showed that Ward linkage produces compact, well-defined clusters, while other linkages such as single or complete may behave very differently depending on the data.

When compared to K-Means and DBSCAN, hierarchical clustering strikes a balance between flexibility and interpretability. K-Means remains efficient and effective for spherical, well-separated clusters; DBSCAN excels at discovering arbitrary shapes and identifying noise; and hierarchical clustering provides a multi-scale view that reveals how clusters emerge and merge. Understanding these differences is essential for selecting the most appropriate algorithm for a given dataset.

Lab 4 – Regression (Simple, Multiple, Polynomial)

Objective

The objective of this laboratory session is to study and apply regression techniques for modeling and predicting continuous numerical variables using supervised learning methods. The experiment focuses on understanding how relationships between dependent and independent variables can be quantified, interpreted, and evaluated using statistical and machine learning tools.

Specifically, this lab aims to:

- Implement Simple Linear Regression to model the relationship between a single explanatory variable and a target variable, and evaluate the strength of this relationship using error metrics and goodness-of-fit measures.
- Apply Multiple Linear Regression to analyze the combined influence of several numerical and categorical predictors on a continuous outcome and explore the use of one-hot encoding for categorical variables.
- Explore Polynomial Regression as a means of capturing nonlinear relationships, and examine how increasing model complexity affects performance and the risk of overfitting.
- Assess model quality using quantitative metrics such as Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), R^2, and Adjusted R^2.
- Perform residual analysis to verify key regression assumptions, such as error normality and model adequacy.
- Compare linear, multiple, and polynomial models in terms of interpretability, predictive accuracy, and suitability for different types of data distributions.

Through experiments on both real-world and simulated datasets, this lab provides practical insight into the strengths, limitations, and appropriate use cases of regression models in data analysis and predictive modeling.

Theory

Regression is a supervised learning technique used to model and predict a continuous numerical variable based on one or more explanatory variables. The goal of regression analysis is to quantify the relationship between independent variables (predictors) and a dependent variable (target), and to use this relationship to make accurate predictions on unseen data.

At its core, regression seeks to determine a mathematical function that best approximates the underlying data distribution by minimizing the prediction error. In most practical settings, this is achieved using the least squares method, which minimizes the sum of squared differences between observed values and model predictions.

Simple Linear Regression

Simple linear regression models the relationship between a single independent variable $x$and a dependent variable $y$using a linear function:

$$y = b_0 + b_1 x$$

where $b_0$represents the intercept and $b_1$represents the slope of the regression line. The parameters are estimated by minimizing the sum of squared residuals, defined as the difference between observed values and predicted values.

Simple linear regression assumes:

- A linear relationship between variables
- Independence of errors
- Homoscedasticity (constant error variance)
- Approximate normality of residuals

The quality of the model is often assessed using metrics such as Mean Squared Error (MSE) and the coefficient of determination (R^2), which measures the proportion of variance in the dependent variable explained by the model.

Multiple Linear Regression

Multiple linear regression extends the simple linear model to include multiple independent variables, allowing the combined influence of several predictors to be analyzed simultaneously:

$$y = b_0 + b_1 x_1 + b_2 x_2 + \cdots + b_p x_p$$

This formulation is particularly useful when the target variable depends on several factors. However, care must be taken when adding new predictors, since the standard R^2 value never decreases as more variables are included, even if they are not relevant.

To address this limitation, the Adjusted R^2 metric is used. It introduces a penalty for unnecessary predictors and provides a more reliable measure of model quality when comparing models with different numbers of regressors.

In practical applications, datasets may include categorical variables, which cannot be directly used in linear regression. These variables must be transformed using dummy variables (one-hot encoding), allowing categorical information to be incorporated into the linear model without introducing artificial numerical orderings.

Polynomial Regression

Polynomial regression is used when the relationship between variables is nonlinear but can still be expressed as a linear combination of transformed features. Instead of fitting a straight line, the model includes polynomial terms of the independent variable:

$$y = b_0 + b_1 x + b_2 x^2 + \cdots + b_d x^d$$

Although the model becomes nonlinear in x, it remains linear in its parameters, which allows standard linear regression techniques to be applied.

Increasing the polynomial degree generally improves the fit to the training data; however, overly complex models may capture noise rather than meaningful structure, leading to overfitting. Therefore, polynomial regression requires careful evaluation using performance metrics and residual analysis to balance model flexibility and generalization ability.

Code

The Regression Code used in this experiment is available at the following GitHub repository: Regression_Code

Simple Linear Regression (Salary Dataset)

This experiment uses the Salary dataset to illustrate the principles of simple linear regression, where a single explanatory variable is used to predict a continuous target variable. The objective is to understand how a linear regression model is built, evaluated, and validated, rather than to perform an in-depth economic analysis of salaries.

Dataset inspection and suitability

The *Salary_Data.csv* dataset is loaded using pandas.read_csv(). It contains 30 observations and two variables, making it well suited for demonstrating simple linear regression in a controlled and interpretable setting. The dataset consists of one explanatory variable, YearsExperience, and one target variable, Salary.

Inspection of the first rows confirms that both variables are numerical and continuous, with no missing or anomalous values. This satisfies the basic requirements of simple linear regression and ensures that parameter estimation and error analysis are not affected by data quality issues.

Descriptive statistics and linear dependency

Descriptive statistics computed using df.describe() summarize the central tendency and dispersion of both variables. Years of experience span from 1.1 to 10.5 years, while salaries range from €37,731 to €122,391, indicating sufficient variability to meaningfully evaluate a predictive relationship.

The correlation analysis reveals a correlation coefficient of 0.978 between the explanatory and target variables. This extremely strong positive correlation indicates that the assumption of a

linear relationship is well satisfied. In the context of simple linear regression, such a high correlation suggests that a large proportion of the variability in the target variable can potentially be explained by the predictor.

The correlation heatmap shown in **Figure 27** visually confirms this result and provides empirical justification for applying a linear regression model rather than a nonlinear alternative.

Basic statistics:

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

Figure 26. *Descriptive Statistics*

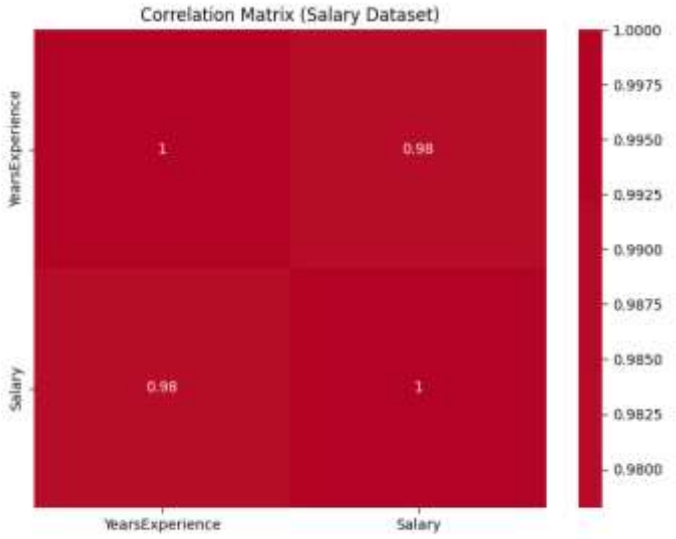


Figure 27. *Correlation Matrix (Salary Dataset)*

Model construction and evaluation

A simple linear regression model is constructed by assigning YearsExperience to X and Salary to y. Before training, a scatter plot is generated to visually assess the form of the relationship between the variables.

As shown in **Figure 28**, the data points follow a clear upward linear pattern with limited dispersion. This visual inspection supports the assumption of linearity and indicates that a straight-line model is appropriate for capturing the underlying relationship.

Figure 28. *Experience vs Salary*

The dataset is then split into 70% training data and 30% test data to evaluate both model fitting and generalization. The regression model is trained on the training set and evaluated on both subsets using multiple complementary metrics.

Prediction accuracy is assessed using MSE, RMSE, and MAE, which quantify the magnitude of prediction errors from different perspectives. Model explanatory power is evaluated using R^2, which measures the proportion of variance in the target variable explained by the predictor, and Adjusted R^2, which serves as a consistency check for model validity.

The R^2 value of 0.94 on the training set indicates that the model explains approximately 94% of the variance in the target variable. The R^2 value of 0.97 on the test set demonstrates excellent generalization, indicating that the learned linear relationship is stable and not the result of overfitting.

Adjusted R^2 values remain very close to R^2, as expected for a model with a single explanatory variable. Error metrics further confirm strong performance: the test RMSE of approximately €4,834 and MAE of €3,737 represent relatively small errors compared to the overall scale of the target variable.

The regression results are visualized in **Figure 29** and **Figure 30**. In both cases, predicted values closely follow observed values, illustrating the ability of simple linear regression to capture the dominant linear trend in the data.

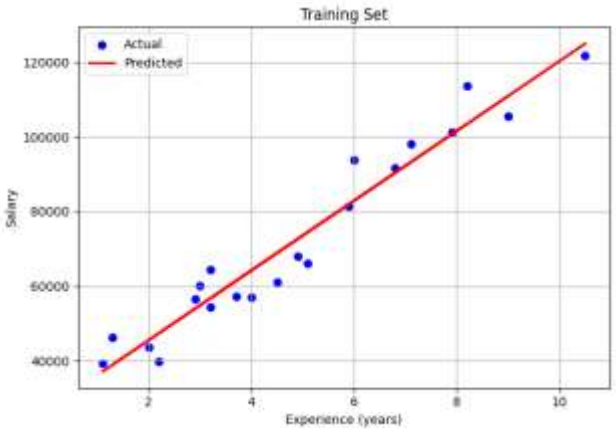


Figure 29. Training Set

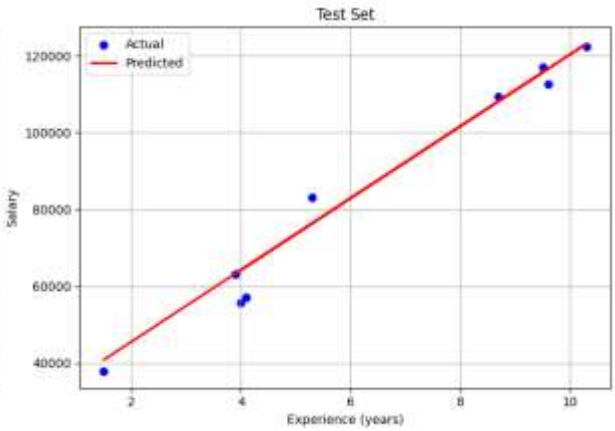


Figure 30. Test Set

Residual analysis and validation of assumptions

Residual analysis is conducted to verify the assumptions underlying simple linear regression. Residuals are computed as the difference between actual and predicted values. The mean residual for the training set is effectively zero, indicating an unbiased model. The slightly negative mean residual for the test set (≈ **−1,695**) suggests a minor underestimation tendency, which remains negligible relative to the target variable's scale.

The residual histogram in **Figure 31** shows an approximately symmetric distribution centered near zero, supporting the assumption of normally distributed errors. The Q–Q plot in **Figure 32** further confirms this observation, with most points closely aligned with the reference diagonal. Minor deviations at the extremes are typical in practical applications and do not significantly violate regression assumptions.

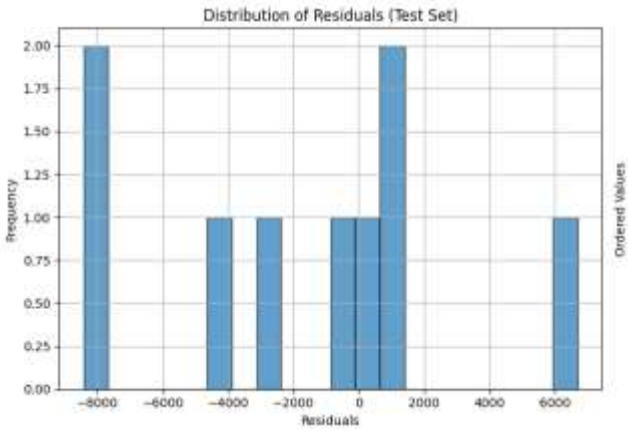


Figure 31. Residual Distribution (Test Set)

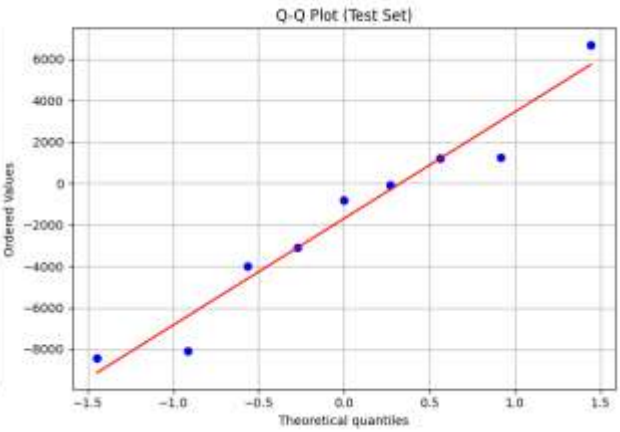


Figure 32. Q–Q Plot (Test Set Residuals)

Multiple Linear Regression (50_Startups Dataset)

This part of the experiment extends simple linear regression to multiple linear regression, where several explanatory variables are used simultaneously to predict a continuous target variable. The

objective is to understand how multiple predictors jointly contribute to a model, how categorical variables are handled, and how model evaluation changes as dimensionality increases.

Dataset inspection and structure

The dataset is loaded using pandas.read_csv() and contains 50 observations and five variables. The target variable is Profit, while the explanatory variables consist of three numerical predictors (R&D Spend, Administration, Marketing Spend) and one categorical predictor (State).

Inspection of the dataset confirms that all variables are complete and numerically well-defined, with no missing values. The presence of both numerical and categorical features makes this dataset particularly suitable for illustrating real-world preprocessing requirements in multiple linear regression.

Descriptive statistics and correlation analysis

Descriptive statistics indicate that the numerical predictors operate on markedly different scales. R&D Spend and Marketing Spend show large ranges and high variability, whereas Administration varies within a narrower interval. This difference in scale and dispersion is common in real-world datasets and highlights that predictors may contribute unequally to a regression model.

The correlation matrix shown in **Figure 33** clarifies the linear relationships between each predictor and the target variable. R&D Spend exhibits a very strong positive correlation with Profit (0.97), indicating that it is the primary driver of the model's explanatory power. Marketing Spend shows a moderate positive correlation (0.75), suggesting a secondary contribution, while Administration displays a weak correlation (0.20), implying a limited direct linear effect on profit.

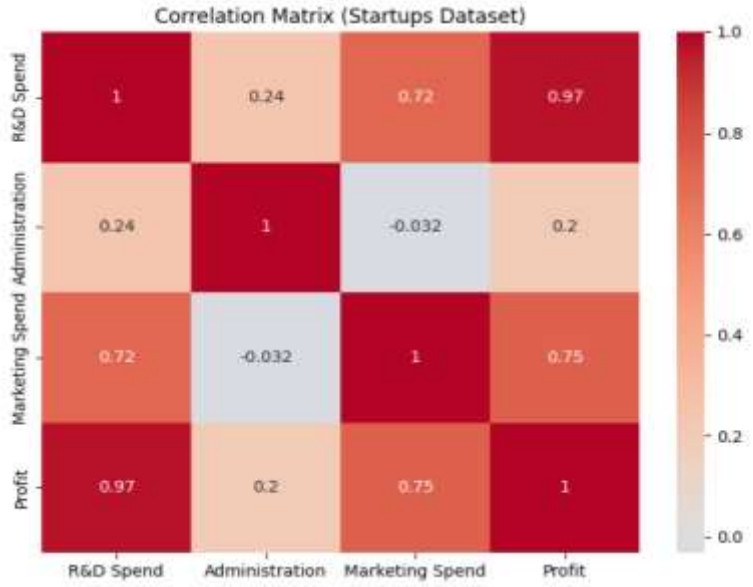


Figure 33. *Correlation Matrix (50_Startups Dataset)*

The matrix also reveals a substantial correlation between R&D Spend and Marketing Spend (0.72). This interdependence illustrates an important characteristic of multiple linear regression: predictors are not necessarily independent. Such correlations between explanatory variables can influence coefficient estimates and complicate interpretation, making global performance metrics and residual analysis essential for reliable model evaluation.

Exploratory visualization of predictors

To further examine individual predictor–target relationships, scatter plots are generated for each numerical feature against Profit. As shown in **Figures 34–36**, R&D Spend displays a strong linear trend, Marketing Spend shows a weaker but still visible relationship, and Administration exhibits substantial scatter with no clear linear structure.

These visualizations reinforce the correlation analysis and demonstrate why multiple regression is necessary: while one variable may dominate, additional predictors can still contribute explanatory power when combined appropriately.

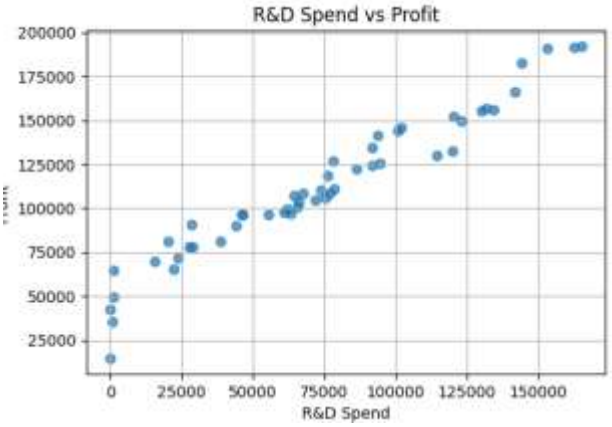


Figure 34. *R&D Spend vs Profit*

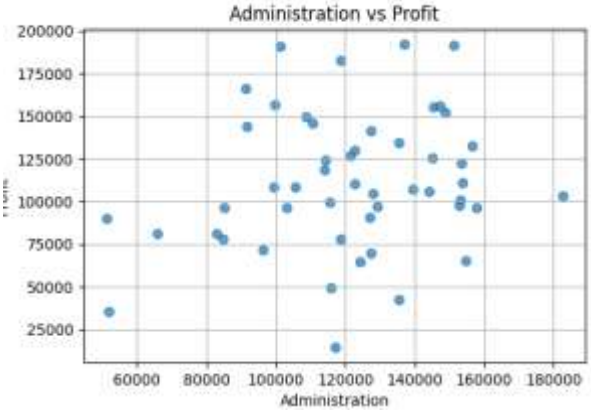


Figure 35. *Administration vs Profit*

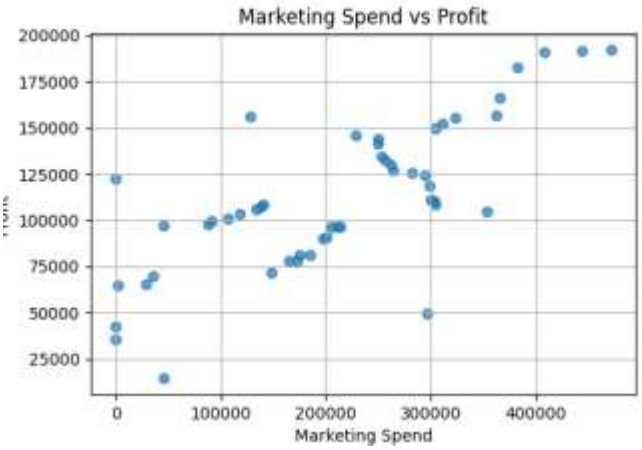


Figure 36. *Marketing Spend vs Profit*

Handling categorical variables

Multiple linear regression requires numerical input features. Since State is a categorical variable, it cannot be directly used in the regression model. To address this, one-hot encoding is applied using a ColumnTransformer combined with OneHotEncoder.

This transformation converts the categorical variable into a set of binary dummy variables, allowing the model to account for state-specific effects without imposing any artificial numerical ordering. All numerical predictors are passed through unchanged, resulting in a fully numerical feature matrix suitable for regression.

Model training and performance evaluation

After preprocessing, the dataset is split into 70% training data and 30% test data. A multiple linear regression model is trained on the training set and evaluated on both subsets using error-based and goodness-of-fit metrics.

The training R^2 value of 0.95 indicates that approximately 95% of the variance in profit is explained by the combined predictors. The test R^2 value of 0.94 confirms strong generalization to unseen data, indicating that the model captures stable linear relationships rather than noise.

In contrast to simple linear regression, Adjusted R^2 plays a critical role here. The Adjusted R^2 decreases from 0.94 (training) to 0.89 (test), reflecting the penalty applied for increased model complexity. This reduction indicates that while the additional predictors improve explanatory power, not all contribute equally, and some may add limited information beyond the strongest variables.

Error metrics further support these conclusions. The test MSE of approximately 61.9 million reflects larger absolute errors than in the simple regression case, which is expected due to the broader scale and variability of the target variable. Nevertheless, the consistency between training and test errors indicates that the model is not overfitting. The comparison between actual and predicted profits on the test set is shown in **Figure 37**, where predicted values closely track observed values across sample

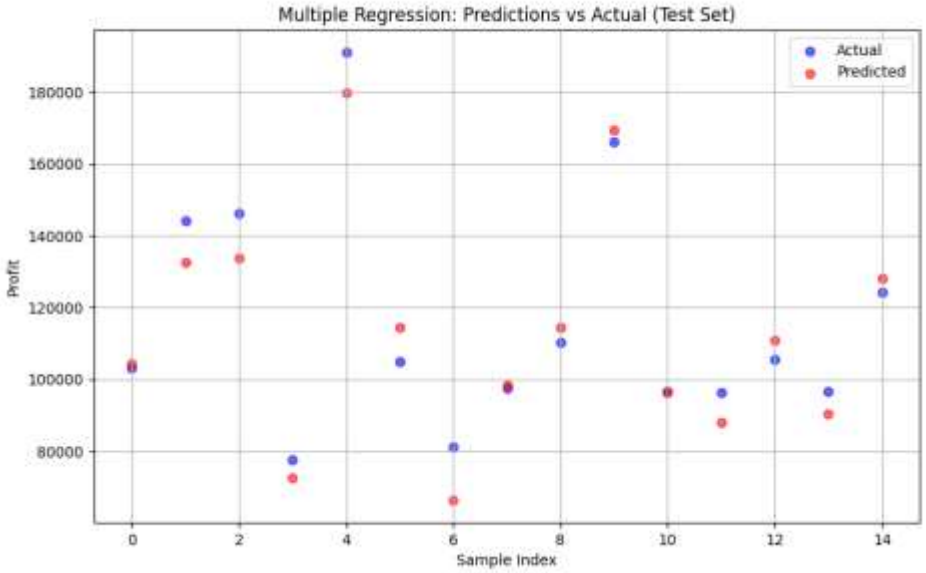


Figure 37. *Multiple Linear Regression — Actual vs Predicted (Test Set)*

Residual analysis and model validation

Residuals are computed on the test set to verify regression assumptions. The residual histogram shown in **Figure 38** is approximately symmetric and centered around zero, suggesting that the model does not exhibit systematic bias.

The Q–Q plot in **Figure 39** shows that residuals largely follow a normal distribution, with minor deviations at the extremes. Such deviations are common in real-world datasets and do not significantly undermine the validity of the model.

Together, these diagnostics indicate that the assumptions of multiple linear regression are reasonably satisfied.

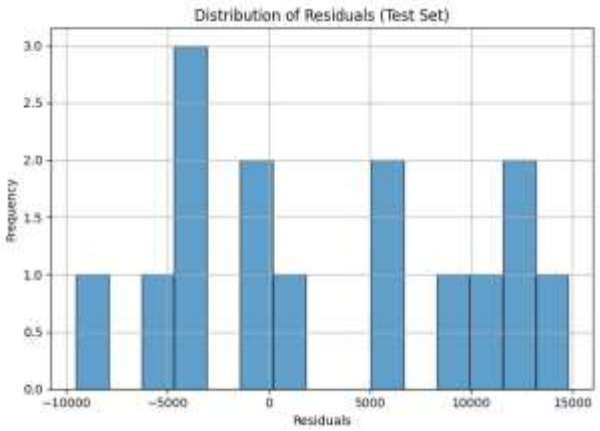


Figure 38. *Residual Distribution (Test Set)*

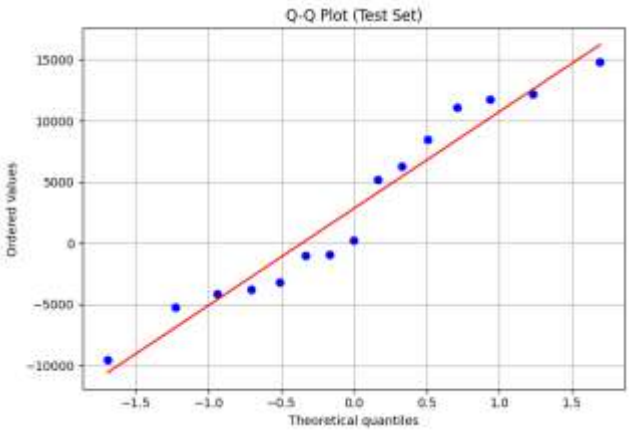


Figure 39. *Q–Q Plot (Test Set Residuals)*

Example prediction and practical interpretation

To illustrate the practical use of the multiple linear regression model, an example prediction is performed using a hypothetical startup configuration. After applying one-hot encoding to the categorical variable *State*, a new input representing a startup located in New York with an R&D Spend of €130,000, Administration costs of €140,000, and Marketing Spend of €300,000 is provided to the trained model.

The model predicts a profit of approximately €158,745 for this configuration. This result demonstrates how multiple linear regression combines the weighted contributions of several predictors to generate a single numerical estimate. Unlike simple regression, the predicted value reflects the simultaneous influence of multiple operational factors rather than reliance on a single explanatory variable.

```
# Example prediction (after one-hot encoding)
sample_input = [[1, 0, 0, 130000, 140000, 300000]]  # [NewYork, California, Florida, R&D, Admin, Marketing]
prediction = regressor_multi.predict(sample_input)
print(f"\nPrediction for input {sample_input}: ${prediction[0]:,.2f}")
```

```
Prediction for input [[1, 0, 0, 130000, 140000, 300000]]: $158,745.12
```

Figure 40. *Snapshot of python code and response*

Polynomial Regression

Polynomial regression is applied in this experiment to model relationships where changes in the target variable accelerate or decelerate as the input increases. Unlike linear models, polynomial regression allows the rate of change itself to vary, making it suitable for datasets where growth is clearly non-uniform across the input range.

This section examines polynomial regression in two contexts:

- A structured dataset illustrating hierarchical salary progression.
- A simulated nonlinear dataset used to analyze model behavior, complexity, and alternative modeling strategies.

Polynomial Regression on the Position Salaries Dataset

Data structure and nonlinear behavior

The **Position_Salaries.csv** dataset describes salary as a function of hierarchical position level. When visualized (Figure 41), the data exhibits a clear nonlinear pattern: salary increases slowly at lower levels and rises sharply at senior levels. This structure reflects a common real-world phenomenon where promotions at higher ranks produce disproportionately larger compensation increases. This curvature indicates that a constant-rate model is inadequate and motivates the use of polynomial regression to capture changes in growth intensity across position levels.

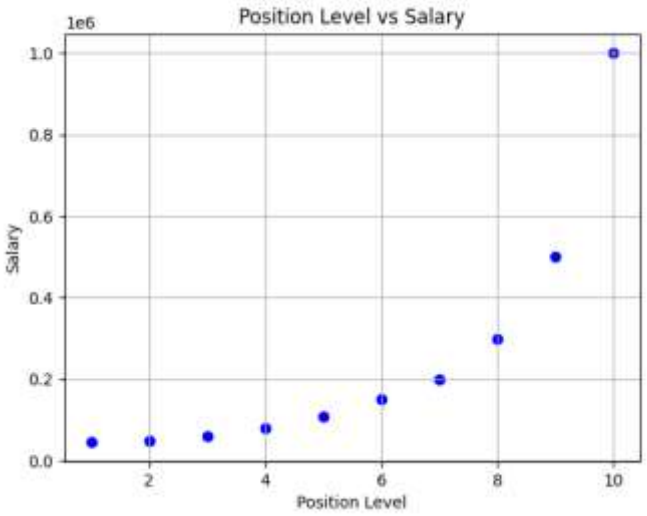


Figure 41. *Position Level vs Salary*

A linear regression model is first fitted to serve as a baseline. As shown in **Figure 42**, the fitted line systematically underestimates salaries at higher position levels and overestimates them at lower levels. This mismatch is not caused by noise, but by the model's inability to represent accelerating growth.

The relatively low explanatory performance confirms that the relationship is not well approximated by a constant slope, reinforcing the need for a more flexible functional form.

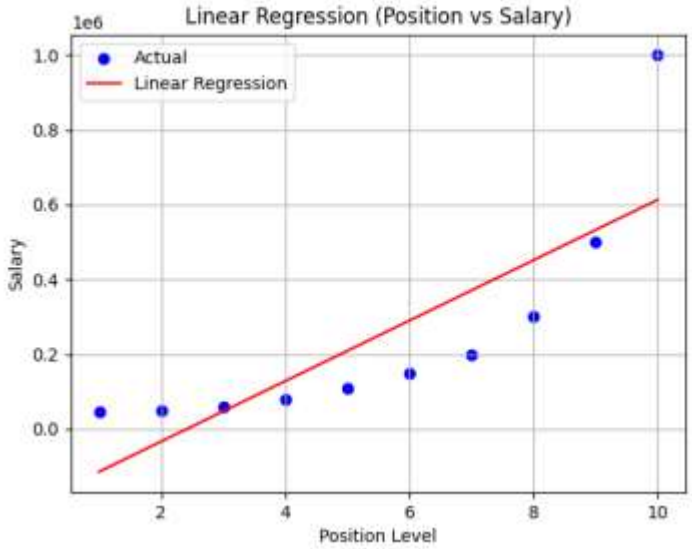


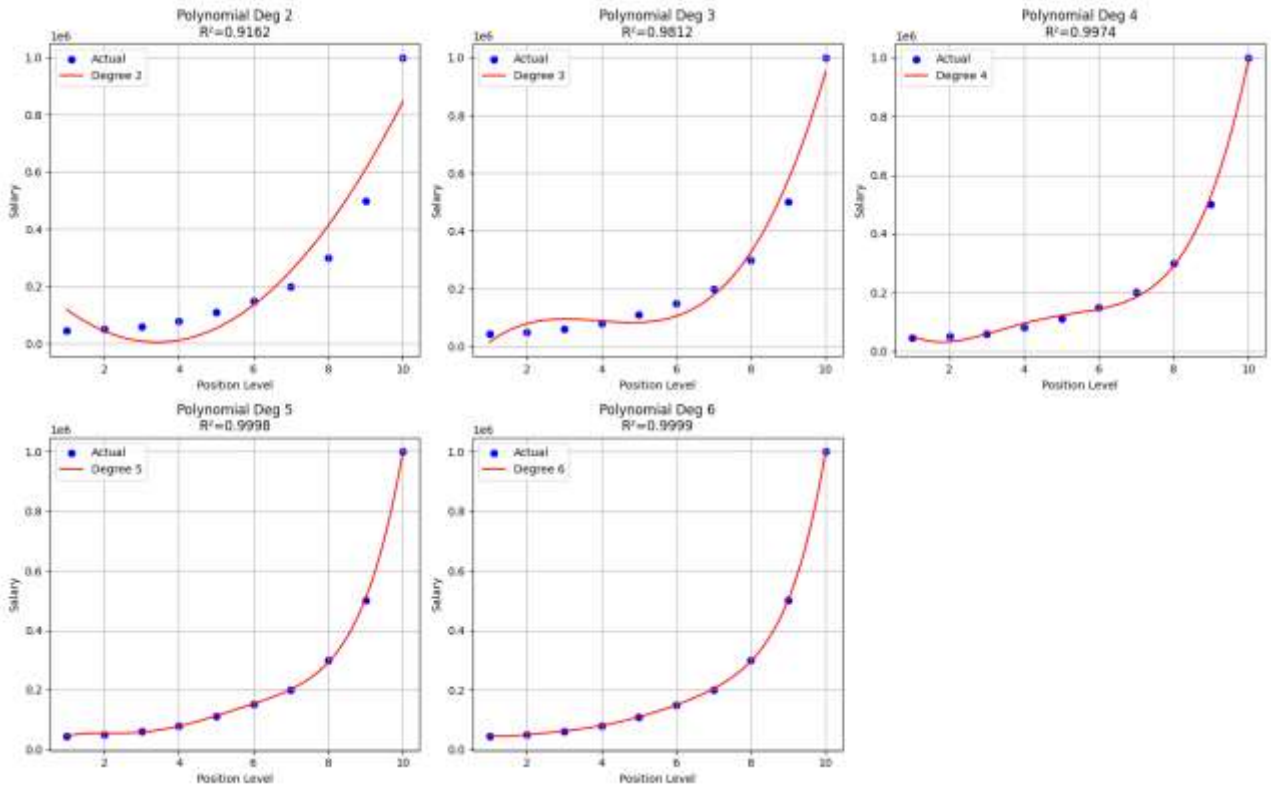
Figure 42. *Linear Regression (Position vs Salary)*

Polynomial regression models of degrees 2 through 6 are then fitted to the data. As illustrated in **Figures 43–47**, increasing the polynomial degree progressively improves the model's ability to follow the observed salary trajectory.

The following observations were made:

- Lower-degree models capture basic curvature but still underestimate rapid salary growth.
- Intermediate degrees provide a strong balance between smoothness and accuracy.
- Higher-degree models closely interpolate the data points, achieving near-perfect fit.

This progression demonstrates how polynomial regression adapts model flexibility by increasing degree, allowing the fitted curve to reflect nonlinear trends that are structurally present in the data.



Figures 43–47. *Polynomial Regression Models (Degrees 2–6)*

The improvement in R^2 values with increasing polynomial degree confirms that higher-order models explain a larger proportion of variance. However, this improvement must be interpreted cautiously. The dataset contains a small number of observations, and higher-degree models risk learning the specific data configuration rather than a generalizable pattern.

Adjusted R^2 values provide insight into this trade-off. While they remain high, gains diminish as degree increases, indicating that additional complexity yields progressively smaller explanatory benefits.

Polynomial Regression on Simulated Nonlinear Data

To further analyze polynomial behavior, a synthetic dataset is generated where purchase amount decreases proportionally to the square of page loading time. The resulting scatter plot (Figure 48) exhibits a steep nonlinear decay, with large values at short loading times and rapid convergence toward zero.

This dataset is intentionally constructed to challenge polynomial regression, as the underlying relationship is not naturally polynomial.

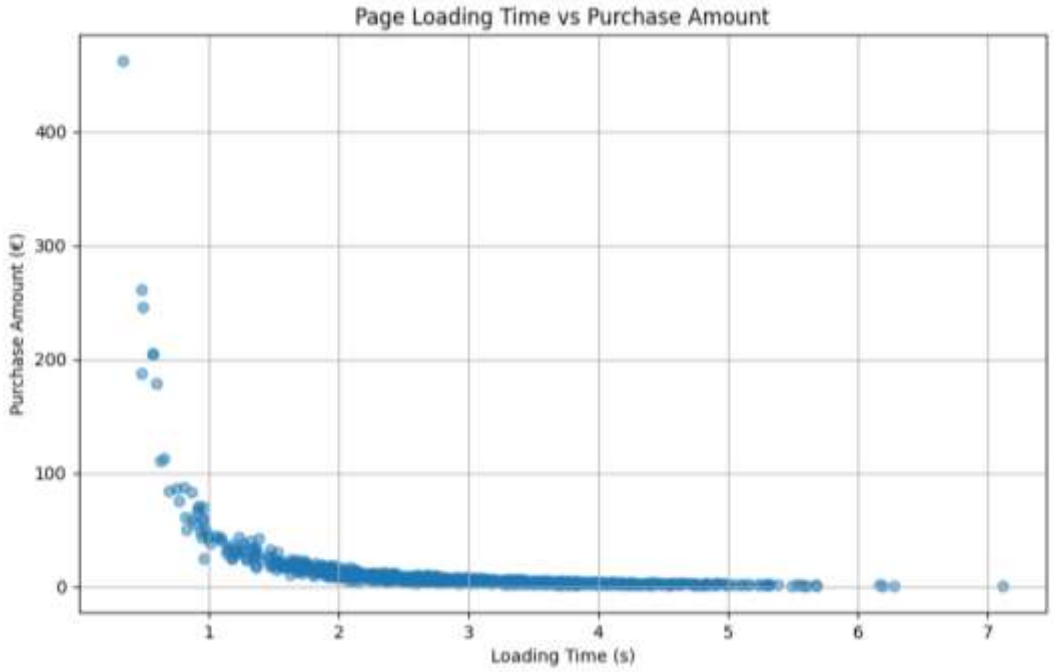
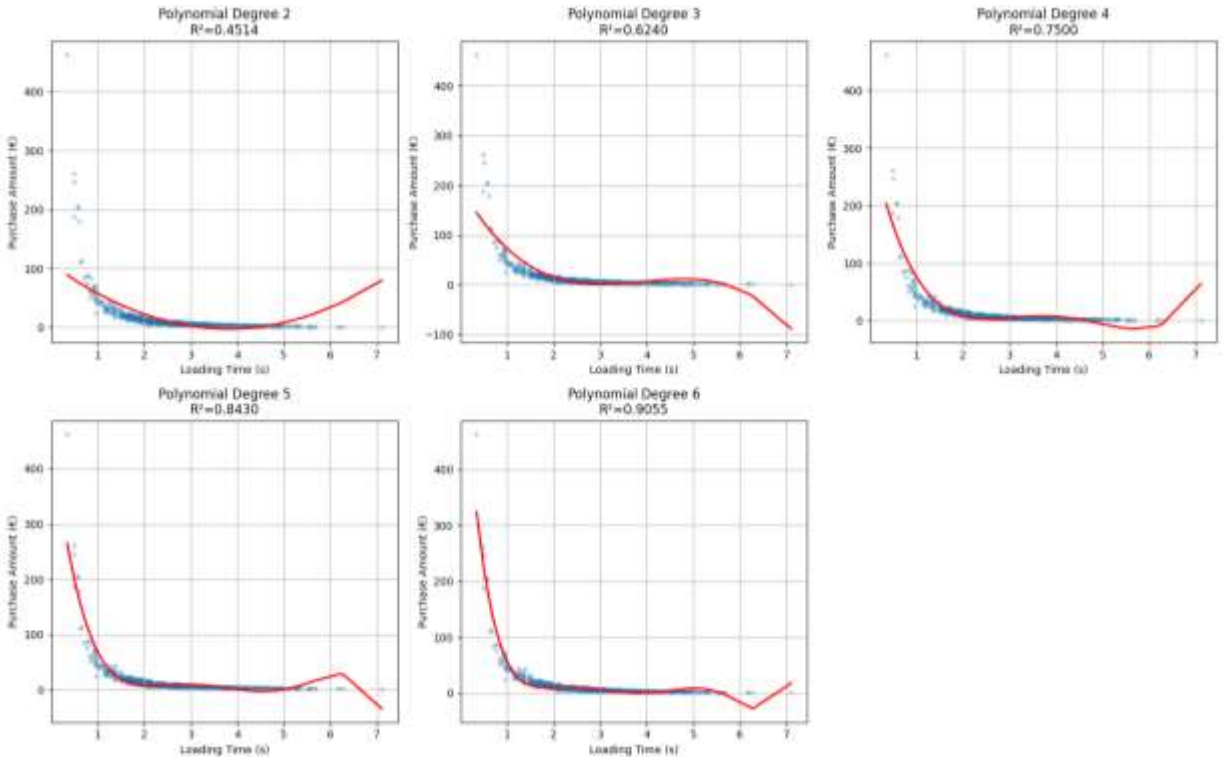


Figure 48. *Page Loading Time vs Purchase Amount*

Polynomial approximation and instability

Polynomial models of increasing degree are fitted to the simulated data, as shown in **Figures 49–53**. Although higher-degree models achieve improved R^2 values, the fitted curves display oscillatory behavior and instability, particularly near the boundaries of the input range.

These artifacts illustrate a key limitation of polynomial regression: when the functional form does not match the true relationship, increasing degree can improve numerical fit while degrading interpretability and stability.



Figures 49–53. *Polynomial Regression on Simulated Data (Degrees 2–6)*

Feature transformation and model reinterpretation

To address this issue, the explanatory variable is transformed using a **1/x²** mapping, reflecting the known structure of the data-generating process. After transformation, a simple linear regression model is applied. As shown in **Figure 54**, the transformed model achieves a high R^2 value (≈ 0.97) while producing a smooth and stable fit without oscillations. This result demonstrates that appropriate feature transformation can outperform high-degree polynomial models, both in accuracy and robustness.

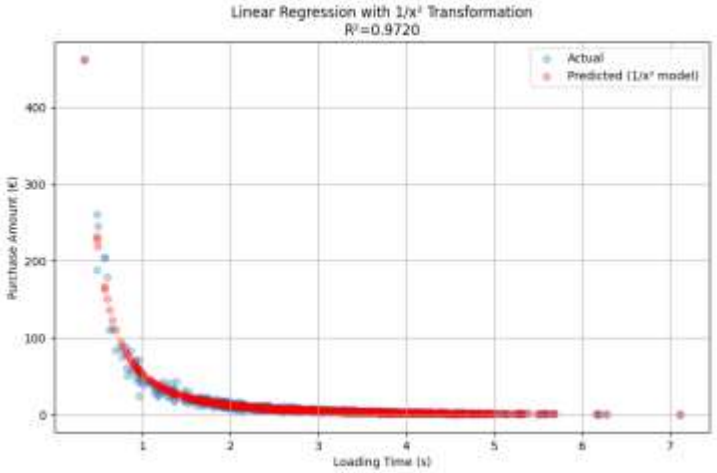


Figure 54. *Linear Regression with 1/x² Transformation*

Conclusion

This laboratory successfully demonstrated how regression models can be constructed, evaluated, and validated for predicting continuous numerical outputs using supervised learning. Across all parts of the experiment, the main focus was not the datasets themselves, but how regression methods behave under different data structures and modeling requirements.

In the simple linear regression experiment, the lab showed how a single predictor can be used to learn an interpretable linear relationship and produce accurate predictions when the linearity assumption is satisfied. The strong goodness-of-fit values and small error metrics confirmed that the model captured the dominant trend effectively, while residual diagnostics supported the validity of key assumptions such as unbiased errors and approximate normality.

The multiple linear regression experiment expanded this approach to a realistic setting with several predictors and a categorical feature. This part further emphasized practical preprocessing using one-hot encoding and demonstrated how multiple variables jointly contribute to prediction. The strong test performance confirmed that the model generalized well, while the drop in Adjusted R^2 highlighted an important methodological point: adding predictors increases complexity, and not all variables necessarily provide meaningful additional explanatory power. Residual analysis further supported the suitability of the linear model for the startups dataset.

Finally, polynomial regression illustrated how nonlinear patterns can be captured by introducing higher-degree feature transformations. In the Position Salaries dataset, increasing polynomial degree improved model fit by matching the accelerating salary trend. However, the experiment also made clear that better fit does not automatically mean better modeling, especially with small datasets where high-degree polynomials can become overly sensitive and risk overfitting. This limitation became even clearer in the simulated nonlinear dataset: polynomial models improved with degree but produced unstable curves, particularly at the boundaries. The transformation-based approach ($1/x^2$) achieved superior accuracy with a simpler and more stable model, reinforcing a key lesson: selecting a representation that matches the underlying relationship often outperforms increasing model complexity.

Lab 5 – Logistic Regression

Objective

The objective of this laboratory session is to study and apply logistic regression as a supervised learning technique for binary classification problems. The experiment focuses on understanding how probabilistic models can be used to separate data into two classes, evaluate classification performance, and interpret decision boundaries in feature space.

Specifically, this lab aims to:

- Implement logistic regression to model the probability of a binary outcome based on numerical input features.
- Understand the role of feature scaling in gradient-based classification models.
- Analyze how logistic regression estimates class probabilities and converts them into discrete class predictions using decision thresholds.
- Evaluate classification performance using accuracy, precision, recall, F1-score, and confusion matrices.
- Visualize the learned decision boundary in a two-dimensional feature space for both training and test datasets.
- Assess model quality using Receiver Operating Characteristic (ROC) curves and the Area Under the Curve (AUC) metric.
- Interpret the trade-off between false positives and false negatives through probabilistic predictions.

Through experiments on a real-world social network advertising dataset, this lab provides practical insight into how logistic regression operates as a baseline classification model, its strengths in interpretability and efficiency, and its limitations when class boundaries are not linearly separable.

Theory

Logistic regression is a supervised learning method for binary classification, where the target variable takes one of two possible values, commonly encoded as 0 and 1. Instead of predicting a continuous numerical outcome, logistic regression models the probability that an observation belongs to a particular class, making it especially suitable for decision-oriented tasks.

At its core, logistic regression constructs a linear combination of input features. Given a feature vector

$$X = (x_1, x_2, \dots, x_p),$$

the model computes a linear score:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

This linear score is then transformed using the sigmoid (logistic) function, which maps any real-valued input to the interval $[0, 1]$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The output of the sigmoid function is interpreted as a probability estimate:

$$P(y = 1 \mid X) = \sigma(z)$$

This probabilistic formulation allows logistic regression to express classification confidence, rather than producing only hard class assignments.

Decision rule

To convert predicted probabilities into class labels, a decision threshold is applied. By convention, a threshold of 0.5 is used:

- If $P(y = 1 \mid X) \geq 0.5$, the model predicts class 1
- Otherwise, it predicts class 0

The threshold can be adjusted depending on the application, enabling control over the trade-off between **false positives** and **false negatives**. This flexibility is particularly important in domains where the costs of misclassification are asymmetric.

Model training and optimization

The parameters of logistic regression are estimated by **maximizing the likelihood** of the observed data. In practice, this is equivalent to **minimizing the log-loss (cross-entropy loss)**, which strongly penalizes confident but incorrect predictions.

Unlike linear regression, logistic regression does not admit a closed-form solution for its parameters. Instead, optimization is performed using **iterative numerical methods**, such as gradient descent or more advanced solvers implemented in machine learning libraries.

Feature scaling and interpretability

Because logistic regression relies on gradient-based optimization, feature scaling is important for stable and efficient convergence. Standardization ensures that all input features contribute proportionally to parameter updates and prevents dominance by variables with larger numerical ranges.

The learned coefficients remain highly interpretable. Each coefficient represents the effect of the corresponding feature on the log-odds of the positive class, assuming all other features remain constant. This interpretability makes logistic regression a preferred model when transparency and explainability are required.

Model evaluation for classification

Evaluation of logistic regression differs from that of regression models, as performance must be assessed in terms of **classification quality** rather than numerical error. Common evaluation tools include:

- **Accuracy**, measuring overall correctness
- **Precision and recall**, capturing class-specific performance
- **Confusion matrices**, summarizing prediction outcomes
- **ROC curves**, evaluating performance across all decision thresholds
- **Area Under the Curve (AUC)**, measuring the model's ability to discriminate between classes independently of a fixed threshold

Together, these metrics provide complementary perspectives on model behavior and decision reliability.

Code

The Logistic Regression Code used in this experiment is available at the following GitHub repository: Logistic_Regression_Code

Implementation using Social Network Ads Dataset

This experiment applies logistic regression to a binary classification problem: predicting whether a user purchases a product (Purchased = 1) or not (Purchased = 0) based on two numerical features, Age and Estimated Salary. The goal is to demonstrate the full logistic-regression pipeline: selecting appropriate inputs, scaling features, training a probabilistic classifier, and interpreting results using classification-specific evaluation tools.

Dataset inspection and modeling setup

The *Social_Network_Ads.csv* dataset is first examined to verify its suitability for binary classification. It contains 400 observations and five variables, including demographic information and a binary outcome variable indicating whether a purchase was made.

For this experiment, Age and Estimated Salary are selected as explanatory variables, while Purchased is used as the target variable. The target is already encoded as a binary label (0 = not purchased, 1 = purchased), which aligns directly with the probabilistic formulation of logistic regression. Restricting the model to two numerical predictors enables direct visualization of the learned decision boundary and facilitates geometric interpretation of the classifier in feature space.

Feature scaling

Before training, both predictors are standardized using StandardScaler. This step is essential because EstimatedSalary has a much larger numeric scale than Age. Without scaling, the optimization procedure would be dominated by the salary feature's magnitude, slowing

convergence and potentially producing a poorly balanced boundary. Scaling ensures that the learned weights reflect *real predictive contribution* rather than the units of measurement.

Model training and probabilistic predictions

The logistic regression model is trained on the standardized training set. Instead of predicting a continuous value, the model learns a linear decision function in feature space and converts it into a probability of purchase. Final class predictions are obtained by applying a default 0.5 threshold (probability $\geq 0.5 \rightarrow$ Purchased = 1). This probability-based mechanism is important because it allows the decision rule to be adjusted later depending on whether the application prefers fewer false alarms or fewer missed buyers.

Evaluation of results

The logistic regression model achieves an overall accuracy of 81% on the test set, indicating that a substantial majority of users are classified correctly. However, accuracy alone does not fully describe classifier behavior, particularly when the costs of different error types are asymmetric. For this reason, a more detailed evaluation is conducted using class-specific metrics and the confusion matrix.

Performance differs noticeably between the two classes. For non-buyers (class 0), the model exhibits high precision (0.82) and very high recall (0.91), demonstrating strong capability in identifying users who do not make a purchase. In practice, this means that most non-buyers are correctly rejected, and predictions of "not purchased" are reliable. In contrast, for buyers (class 1), the model achieves good precision (0.79) but moderate recall (0.64). This indicates that while predictions of a purchase are usually correct, a significant proportion of actual buyers are not detected by the model.

This imbalance reveals a conservative classification strategy: the model tends to favor predicting the negative class when uncertainty exists, thereby reducing false positives at the expense of increasing false negatives. Such behavior is common in purchase prediction tasks, where buyers and non-buyers often overlap in feature space and decision boundaries cannot perfectly separate the two groups.

```
=== Accuracy ===
0.8100

=== Classification report ===
              precision    recall  f1-score   support

           0       0.82      0.91      0.86        64
           1       0.79      0.64      0.71        36

    accuracy                           0.81       100
   macro avg       0.81      0.77      0.78       100
weighted avg       0.81      0.81      0.80       100
```

Figure 55. *Evaluation of Logistic Regression*

The confusion matrix in **Figure 56** provides a concrete breakdown of these outcomes. The model correctly classifies 58 non-buyers and 23 buyers, while producing 6 false positives and 13 false negatives. The higher number of false negatives confirms the lower recall for the positive class and highlights the model's tendency to miss some true purchasers rather than incorrectly label non-buyers as buyers.

From an application perspective, this error structure has meaningful implications. Missing potential buyers corresponds to lost targeting opportunities, while false positives correspond to unnecessary marketing effort. The current model prioritizes minimizing wasted outreach over maximizing buyer detection. Whether this trade-off is desirable depends on the relative costs of these errors in the specific business context. Importantly, logistic regression's probabilistic output allows this balance to be adjusted by modifying the decision threshold, a point further explored through ROC analysis.

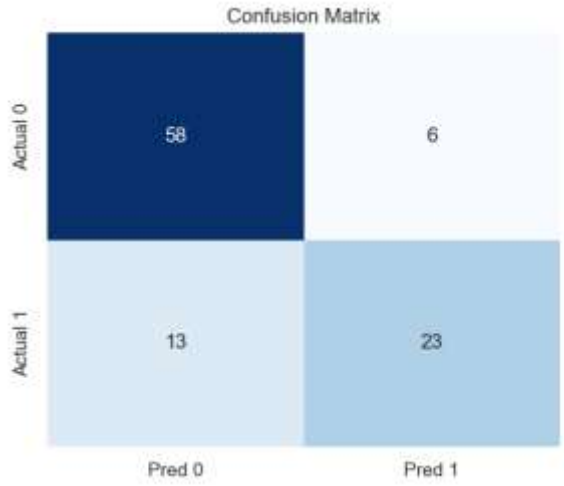


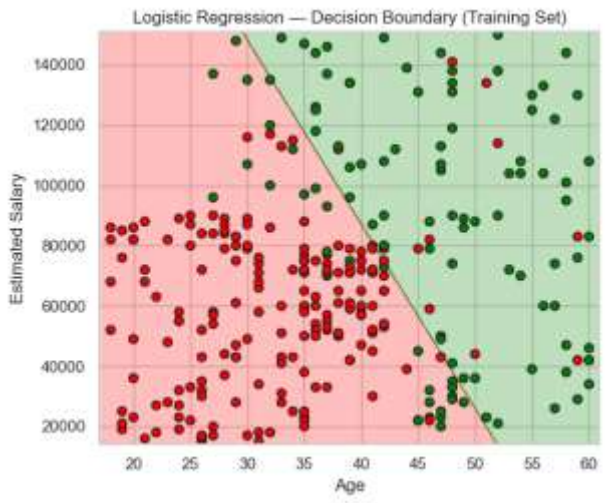
Figure 56. *Confusion Matrix*

Decision boundary visualization

The decision boundary visualizations in **Figures 57** and **58** illustrate how the logistic regression model separates the feature space defined by Age and Estimated Salary into two decision regions corresponding to the predicted classes.

In both the training and test sets, the boundary divides the space into a low-probability region classified as *Not Purchased* and a high-probability region classified as *Purchased*. Observations close to the boundary represent cases where small changes in age or salary lead to different predicted outcomes. The presence of both classes in this narrow transition zone reflects genuine overlap in user behavior.

Importantly, the decision boundary maintains a similar orientation and position across the training and test plots. This consistency indicates that the model has learned a stable linear separation rather than fitting noise specific to the training data.



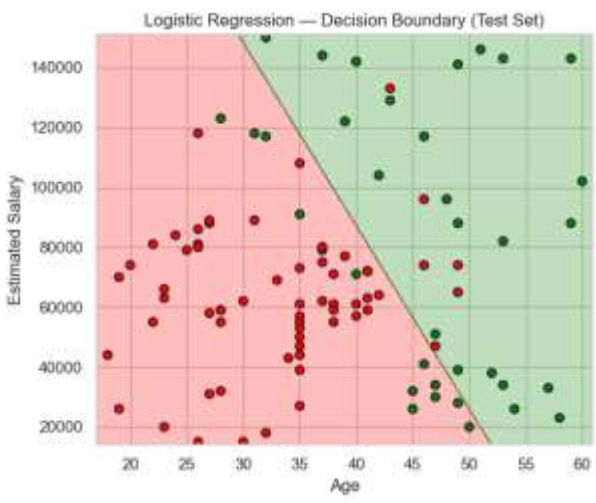


Figure 57. *Decision Boundary (Training Set)*

Figure 58. *Decision Boundary (Test Set)*

ROC curve and AUC (threshold-independent evaluation)

The Receiver Operating Characteristic (ROC) curve provides a threshold-independent evaluation of classifier performance by examining the trade-off between the true positive rate and the false positive rate across all possible probability thresholds. Unlike accuracy or confusion matrices, which depend on a fixed decision threshold, the ROC curve assesses how well the model ranks positive instances above negative ones.

The Area Under the Curve (AUC) value of **0.913** indicates strong discriminative power. This means that, in the majority of cases, the model assigns higher predicted probabilities to users who actually purchase than to those who do not. Such a value reflects a well-separated probability distribution between the two classes, even when their feature values partially overlap.

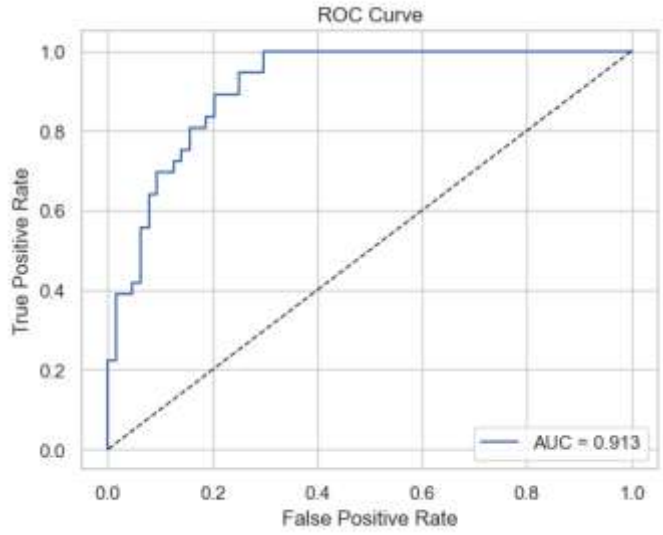


Figure 58. *ROC Curve (AUC = 0.913)*

The contrast between the AUC computed using predicted probabilities (0.913) and the AUC derived from hard class labels (0.773) is particularly informative. Hard labels discard probability information by enforcing a single threshold, whereas ROC/AUC preserves the full ranking structure of predictions.

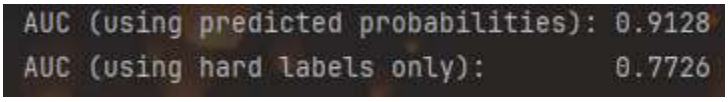


Figure 59. *Contrast between the AUC computed using probabilities and hard-class labels*

Conclusion

This laboratory session demonstrated the practical application of logistic regression as a probabilistic model for binary classification. Using the Social Network Ads dataset, logistic regression was employed to predict purchase behavior based on age and estimated salary, providing a clear and interpretable framework for understanding classification decisions in a two-dimensional feature space.

The experiment showed that logistic regression effectively learns a linear decision boundary that separates buyers from non-buyers while producing meaningful probability estimates rather than only hard class labels. The use of feature standardization was essential for stable optimization, ensuring that both predictors contributed appropriately during model training.

Model evaluation highlighted the importance of analyzing classification performance beyond overall accuracy. Although the classifier achieved an accuracy of 81%, deeper inspection through the confusion matrix and classification report revealed an asymmetric error pattern. The model performed very well at identifying non-buyers, while exhibiting lower recall for buyers, indicating a conservative decision behavior that favors reducing false positives at the cost of missing some true purchasers. This trade-off reflects the inherent overlap between classes in the Age–Salary space rather than a modeling error.

Decision boundary visualizations confirmed that the learned separation is stable across training and test datasets, suggesting good generalization and no evidence of overfitting. The ROC curve and the high AUC value (≈ 0.91) further demonstrated that the model possesses strong discriminative capability across a wide range of decision thresholds, even when the default threshold leads to missed positive cases.

Lab 6 – K-Nearest Neighbors (KNN)

Objective

This experiment aims to explore how the K-Nearest Neighbors (KNN) algorithm performs supervised classification based on proximity to labeled training data. The lab focuses on:

- Understanding the fundamental principles of instance-based (lazy) learning.
- Applying KNN to a synthetic multi-class dataset.
- Examining the importance of feature scaling in distance-based algorithms.
- Evaluating classification performance using accuracy.

Theory

K-Nearest Neighbors (KNN) is a supervised learning algorithm used for classification and regression. Unlike parametric models, KNN does not learn an explicit model during training. Instead, it stores all labeled training instances and defers computation until prediction time, which is why it is often referred to as a lazy learning algorithm.

For classification, KNN operates as follows:

- Given a new data point, compute the distance between this point and all training samples.
- Identify the k nearest neighbors based on a distance metric (commonly Euclidean distance).
- Assign the class label that appears most frequently among the k neighbors (majority voting).

The algorithm relies on the assumption that similar data points tend to belong to the same class. Because distance plays a central role, feature scaling is crucial to prevent attributes with larger numerical ranges from dominating the distance computation.

Code

The KNN classification code used in this experiment is available at the following GitHub repository: KNN_Code

Implementation on Synthetic Data

A synthetic two-dimensional dataset was generated using the make_classification() function from the scikit-learn library. The dataset consisted of 300 samples with two informative features and three distinct class labels.

The generated data was visualized using a scatter plot, where each color represented a different class. The plot revealed three reasonably well-separated groups, making the dataset suitable for multi-class classification using KNN.

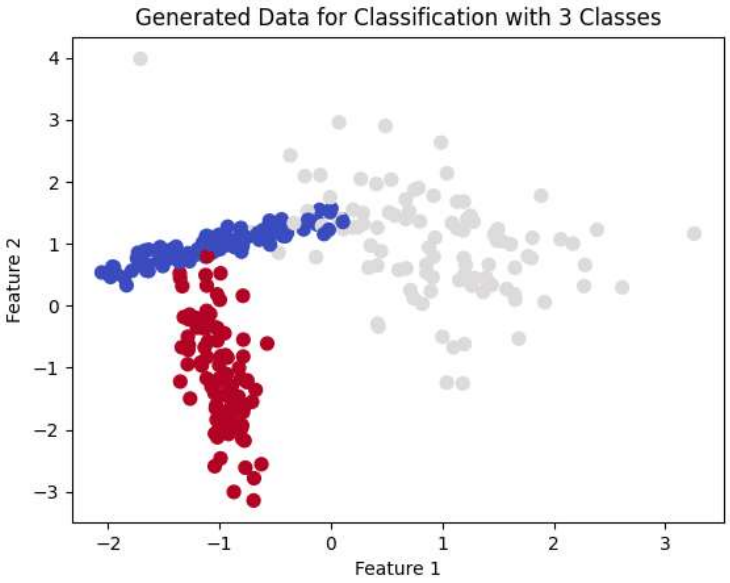
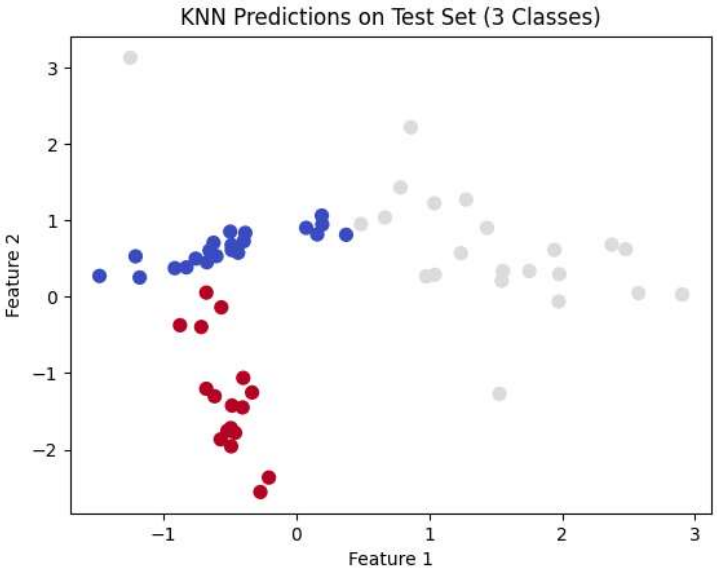


Figure 60. *Synthetic Data with 3 classes*

The dataset was then split into training (80%) and test (20%) sets. Feature standardization was applied using StandardScaler to ensure that both features contributed equally to distance calculations.

A KNN classifier with k = 5 was trained on the standardized training data and evaluated on the test set. The resulting classification accuracy was approximately 0.93, indicating that the algorithm was able to correctly classify most test samples.




```
"D:\Python\Orleans - AI Class\.venv\Scripts\python.exe" "D:\Python\Orleans - AI Class\6. knn.py"
Accuracy of KNN model with 3 classes: 0.93
```

Figure 61. *KNN classification results on the synthetic test dataset (k = 5).*

Effect of Feature Scaling

Because KNN relies on distance calculations, feature scaling has a significant impact on its performance. Without normalization, features with larger numeric ranges would dominate the distance metric and bias the classification results. By applying standardization, each feature was transformed to have zero mean and unit variance, resulting in balanced distance contributions and improved classification stability.

To analyze the impact of the parameter **k**, multiple values were tested:

$$k = \{1,3,5,7,9\}$$

For each value of k, the classifier was trained and evaluated on the test set. The accuracy values were recorded and plotted as a function of k.

The results show that:

- k = 1 produced lower accuracy due to sensitivity to noise and outliers.
- Increasing k improved stability by smoothing decision boundaries.
- Accuracy stabilized for k ≥ 3, indicating a good balance between bias and variance.

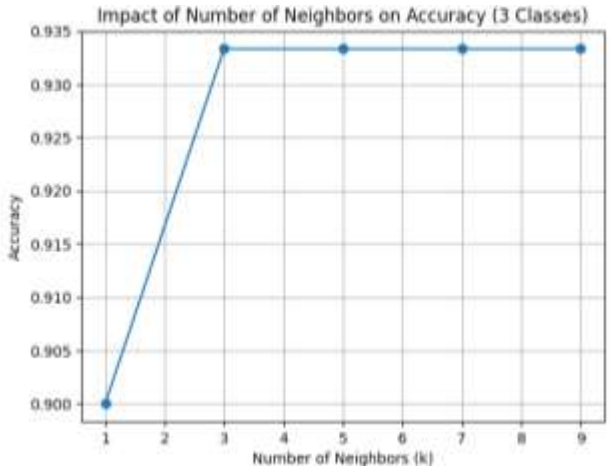


Figure 62. *Classification accuracy as a function of the number of neighbors (k).*

Conclusion

This experiment demonstrated the effectiveness of the K-Nearest Neighbors algorithm for supervised multi-class classification. Using a synthetic dataset, KNN achieved high classification accuracy when appropriate feature scaling and parameter tuning were applied.

The results highlighted the importance of selecting a suitable number of neighbors and standardizing features to ensure meaningful distance calculations. While KNN is simple and powerful for small to medium-sized datasets, its computational cost and sensitivity to dimensionality limit its scalability.

Lab 7 – Tree-Based Clustering (Silhouette Splitting)

Objective

The objective of this laboratory is to study the fundamental principles of decision tree–based learning, with a focus on how datasets are partitioned using feature–threshold splits and how tree structures are constructed recursively.

Based on the tree techniques presented in the course material, this lab aims to:

- Understand the concept of a decision tree as a model that splits a dataset according to a feature and a threshold value in order to make decisions.
- Identify and describe the main components of a decision tree, including the root node, intermediate (decision) nodes, and leaf nodes, and understand their respective roles in the decision process.
- Study the recursive tree construction algorithm, where the dataset is repeatedly divided into smaller subsets until stopping conditions are met, such as node purity, lack of remaining attributes, or insufficient data.
- Examine how the quality of a split is evaluated at each node by selecting the feature that best separates the data, traditionally using node purity measures such as the Gini index.
- Analyze the advantages and limitations of decision trees, particularly their interpretability, simplicity, and sensitivity to overfitting, which motivates the use of ensemble methods.
- Understand the conceptual motivation behind Random Forests as an ensemble of decision trees built using random sampling of data and features to improve robustness and generalization.

This lab applies these theoretical principles in a practical setting by implementing a tree-based approach to data partitioning and analyzing the resulting hierarchical structure.

Theory

Decision trees are supervised learning models based on the idea of recursively partitioning a dataset into smaller and more homogeneous subsets using a sequence of decision rules. Each rule is defined by a feature and a threshold, which determines how the data is split at a given node.

A decision tree is composed of three main elements:

- **Root node:** The starting point of the tree, where the feature that best splits the dataset is evaluated.
- **Intermediate (decision) nodes:** Nodes where additional feature–threshold tests are applied to further divide the data.
- **Leaf nodes:** Terminal nodes of the tree, where a final decision or prediction is produced, such as a class label or numerical value.

At each node, a condition of the form *feature* ≤ *threshold* is evaluated, and the data is routed to one of the child nodes depending on whether the condition is satisfied.

Split Selection and Node Purity

The effectiveness of a decision tree depends on how well each split separates the data. To evaluate split quality, decision trees rely on node purity measures, which quantify how homogeneous the resulting subsets are.

One commonly used measure is the Gini index, which expresses the probability of incorrectly classifying a randomly chosen sample from a node. A lower Gini value indicates higher purity. The goal of the splitting process is to select, at each node, the feature and threshold that minimize impurity and produce the most informative partition of the data.

The Gini impurity for a node is defined as:

$$Gini(D) = 1 - \sum_{i=1}^{m} p_i^2$$

where $p_i$represents the probability that a sample in node Dbelongs to class i.

Recursive Tree Construction Algorithm

The construction of a decision tree follows a recursive algorithm:

1. Select the best splitting attribute using an attribute selection measure (e.g., Gini index).
2. Partition the dataset into subsets according to the selected feature and threshold.
3. Repeat the process for each subset to form child nodes.

This recursive splitting continues until one of the following stopping conditions is met:

- All samples in a node belong to the same class (node purity).
- No remaining attributes are available for further splitting.
- The subset becomes too small to split meaningfully.

Advantages of Decision Trees:

- They are easy to interpret and visualize.
- They require relatively little data preparation.
- They can be applied to both classification and regression problems.

Limitations of Decision Trees:

- They are prone to overfitting, especially when grown to large depths.
- They are sensitive to small variations in the training data.
- A single decision tree often behaves as a weak learner, producing suboptimal predictions.

Ensemble Learning and Random Forests

To overcome the limitations of individual decision trees, ensemble methods such as Random Forests are introduced. A random forest combines multiple decision trees trained on different bootstrap samples of the dataset. Additionally, at each split, a random subset of features is considered, which increases diversity among trees.

Predictions in a random forest are made by aggregating the outputs of all trees, typically using majority voting for classification or averaging for regression. This ensemble approach improves robustness, reduces overfitting, and leads to better generalization performance compared to a single decision tree.

Code

The Tree Based Clustering Code used in this experiment is available at the following GitHub repository: Tree-Based_Clustering_Code

Data generation and initial visualization

The algorithm starts from an unlabeled two-dimensional feature space that is visualized before any splitting is performed. This initial visualization establishes a reference for interpreting how decision-tree splits operate geometrically. Since tree-based methods partition the feature space using axis-aligned thresholds, observing the raw distribution of samples makes it possible to clearly track how successive splits progressively divide the space into distinct regions.

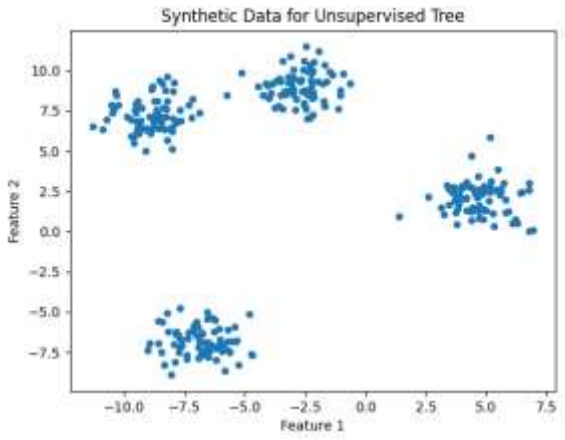


Figure 63. *Synthetic Data for Unsupervised Tree.*

Best split selection using Silhouette score

At each node, the algorithm evaluates candidate splits defined by a feature and a threshold. For a given candidate, the data is divided into two subsets according to the threshold condition. These subsets are treated as provisional clusters, and their quality is evaluated using the Silhouette score.

The Silhouette score quantifies split quality by jointly measuring intra-group compactness and inter-group separation. By maximizing this score, the algorithm selects splits that most clearly separate the data into two coherent groups without relying on class labels.

A minimum leaf size constraint is applied to ensure that both resulting subsets contain enough samples to yield stable and meaningful separation estimates. The split with the highest Silhouette score is selected as the decision rule at the node.

The selected split is visualized by coloring samples according to their assignment on either side of the threshold and drawing the corresponding decision boundary. This visualization illustrates the geometric effect of a single decision-tree node: an axis-aligned partition that divides the feature space into two regions.

Because the split maximizes the Silhouette score, the resulting partition typically separates a compact group from the remainder of the data, highlighting how separation can be optimized even in the absence of labels.

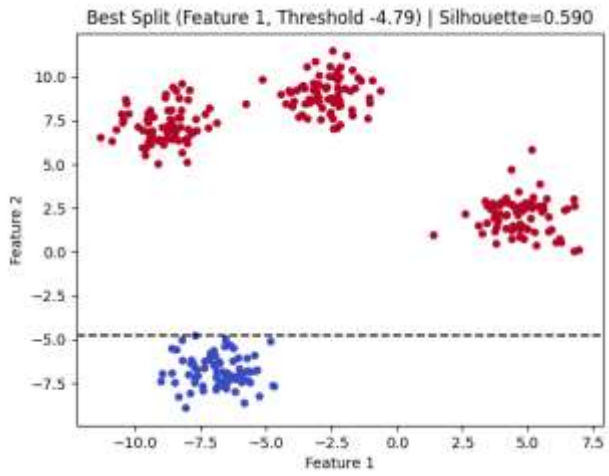


Figure 64. *Best Split (Silhouette-Based).*

Recursive tree construction

To obtain a full clustering, the split-selection procedure is applied recursively to each subset produced by previous splits. At every node, the algorithm selects the feature and threshold that maximize the Silhouette score for the samples reaching that node and partitions the data accordingly.

Tree growth is controlled by limiting the maximum depth, requiring a minimum improvement in Silhouette score, and enforcing a minimum leaf size. These constraints prevent excessive fragmentation and ensure that terminal nodes correspond to meaningful regions of the feature space.

Cluster assignment is performed by traversing the trained tree from the root to a leaf for each sample. Each internal node applies a deterministic feature–threshold decision, and the leaf node reached by a sample defines its final cluster.

The final visualization shows the result of successive axis-aligned partitions applied to the feature space. Each cluster corresponds to a terminal region defined by a sequence of explicit decision rules, making the clustering outcome directly interpretable in terms of feature-based splits.

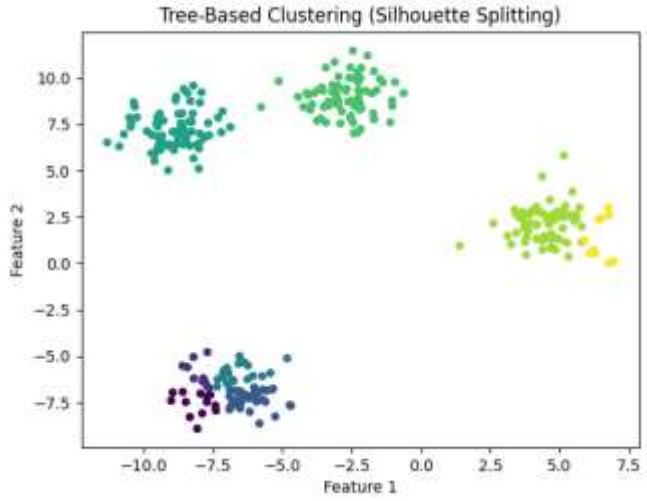


Figure 65. *Tree-Based Clustering (Silhouette Splitting).*

Conclusion

This laboratory demonstrated how the fundamental logic of decision trees can be extended to an unsupervised learning setting by replacing class-based impurity measures with a cluster quality criterion. By using the Silhouette score to guide split selection, the algorithm was able to construct a binary tree that partitions the feature space into compact and well-separated regions without relying on labeled data.

The experiment showed that meaningful clustering can be achieved through recursive feature–threshold splitting, following the same structural principles as supervised decision trees. A single split isolates two coherent groups, while successive splits progressively refine the partitioning, producing a hierarchical clustering structure. The use of stopping criteria such as maximum depth, minimum Silhouette gain, and minimum leaf size proved essential for controlling model complexity and preventing excessive fragmentation.

Visual analysis of the results highlighted the interpretability of the approach. Each final cluster corresponds to a terminal node defined by a sequence of explicit decision rules, making the clustering outcome directly traceable to feature-based decisions.

Lab 8 – Random Forest Classification

Objective

The objective of this laboratory session is to study and apply Random Forest as a supervised learning technique for multi-class classification problems. The experiment focuses on understanding how ensemble methods improve classification performance by combining multiple decision trees trained on randomized subsets of the data and features.

Specifically, this lab aims to:

- Understand the limitations of single decision trees in terms of variance and overfitting.
- Apply the Random Forest algorithm to a multi-class classification problem using structured numerical data.
- Analyze how bootstrap sampling and random feature selection contribute to model robustness and generalization.
- Evaluate classification performance using accuracy, confusion matrices, and class-wise precision and recall.
- Interpret the role of individual features through Gini-based feature importance measures.
- Examine the usefulness of out-of-bag (OOB) estimation as an internal validation metric.

Through experiments on the Wine dataset, this lab provides practical insight into how Random Forests combine multiple weak learners into a strong and reliable classifier, balancing predictive accuracy with interpretability.

Theory

Random Forest is an ensemble learning method designed to improve upon the limitations of individual decision trees. While decision trees are intuitive and easy to interpret, they are highly sensitive to variations in the training data. Small changes in the dataset can lead to significantly different tree structures, resulting in high variance and poor generalization.

Random Forest addresses this issue by constructing a collection of decision trees and aggregating their predictions. Each tree is trained on a slightly different version of the dataset and considers only a subset of features at each split. The final prediction is obtained through majority voting, where each tree contributes one vote for a class label.

Decision Trees and Gini Impurity

At the core of Random Forest lies the decision tree classifier. Each tree recursively partitions the feature space using axis-aligned splits. At every internal node, the algorithm selects a feature and threshold that best separates the classes.

For classification tasks, split quality is commonly measured using the **Gini impurity**, defined as:

$$G = 1 - \sum_{k=1}^{K} p_k^2$$

A lower Gini impurity indicates a purer node, meaning that samples predominantly belong to a single class. Although decision trees can achieve perfect classification on training data, they often overfit because they adapt too closely to the specific structure of the dataset.

Bootstrap Aggregation (Bagging)

Random Forest reduces overfitting through bootstrap aggregation, commonly known as *bagging*. Instead of training each tree on the full dataset, the algorithm creates multiple bootstrap samples by randomly sampling observations with replacement. Each tree is trained on a different bootstrap sample.

This process introduces diversity among the trees. Since each tree sees a slightly different dataset, their individual errors tend to be uncorrelated. When combined through majority voting, these errors cancel out, leading to improved stability and generalization.

Random Feature Selection

In addition to bootstrap sampling, Random Forest introduces randomness at the feature level. At each split, instead of considering all available features, the algorithm selects a random subset of features and chooses the best split only among them.

This mechanism further decorrelates the trees. Without feature randomness, strong predictors could dominate every tree and produce similar structures. By forcing trees to explore different features, Random Forest captures a broader range of decision patterns and reduces dependence on any single variable.

Majority Voting and Generalization

For classification, each decision tree produces a class prediction. The Random Forest classifier assigns the final label based on majority voting across all trees. This ensemble decision is more robust than any individual tree's output.

As the number of trees increases, the variance of the model decreases while the bias remains approximately constant. This explains why Random Forests often achieve high predictive performance without extensive parameter tuning.

Out-of-Bag (OOB) Estimation

An important theoretical advantage of Random Forest is the availability of **out-of-bag estimation**. Because each tree is trained on a bootstrap sample, approximately one-third of the original data is not used in training that tree. These unused samples can serve as a validation set.

By aggregating predictions for each observation using only the trees that did not train on it, the Random Forest can estimate classification accuracy internally, without requiring a separate validation dataset. This provides a reliable measure of generalization performance.

Code

The Random Forest Classification Code used in this experiment is available at the following GitHub repository: Random_Forest_Clustering_Code

Implementation and Analysis using the Wine Dataset

This experiment applies the Random Forest classifier to the **Wine dataset**, a standard multi-class classification benchmark composed of continuous physicochemical measurements. The objective of this implementation is to illustrate how Random Forest performs classification through ensemble learning, how its predictions are evaluated, and how internal validation and feature importance contribute to model interpretation, as outlined in the lab manual

Dataset inspection and structure

The Wine dataset is loaded using load_wine() from scikit-learn. It contains 178 observations, each described by 13 numerical features, and a target variable representing three wine classes corresponding to different cultivars (class_0, class_1, and class_2).

The dataset contains no missing values, and all features are numerical. Since Random Forest relies on decision-tree splits rather than distance-based computations, no normalization or standardization is required, which simplifies preprocessing compared to other supervised learning methods.

The dataset is divided into 70% training data and 30% test data, with stratified sampling to preserve class proportions. The resulting test set contains 54 samples, distributed across the three classes, ensuring a fair and representative evaluation.

Random Forest model construction and training

The Random Forest classifier is configured with 200 decision trees, providing a sufficiently large ensemble to stabilize predictions while keeping computation efficient. Each tree is trained on a bootstrap sample drawn with replacement from the training data, and at each node, a random subset of features is considered for splitting.

Out-of-bag (OOB) estimation is enabled. Because approximately one-third of the training samples are excluded from each tree's bootstrap sample, these unused samples can be used to evaluate model performance internally. This mechanism provides an unbiased estimate of generalization performance without requiring a separate validation set.

After training, class predictions for the test set are obtained through majority voting across all trees.

Classification performance and quantitative evaluation

The command-line output confirms that the Random Forest achieves a test accuracy of 1.00, meaning that all 54 test samples are classified correctly. In addition, the out-of-bag score is 0.976, which is very close to the test accuracy. This close agreement indicates that the model generalizes well and that the perfect test accuracy is not the result of overfitting.

The classification report shows precision, recall, and F1-score equal to 1.00 for all three classes, confirming that the classifier performs equally well across all categories. Each wine class is both perfectly identified and perfectly separated from the others.

These results illustrate one of the strengths of Random Forest emphasized in the lab manual: by aggregating many diverse decision trees, the ensemble produces highly stable and accurate predictions, even in multi-class settings.

The confusion matrix shown in **Figure 66** provides a class-by-class view of the classifier's performance. All samples appear on the main diagonal, with **no off-diagonal entries**, indicating the absence of both false positives and false negatives.

This result confirms that the Random Forest successfully learns decision boundaries that completely separate the three wine classes in the feature space. The confusion matrix visually reinforces the numerical evaluation metrics and demonstrates that classification performance is consistent across all classes rather than being dominated by a subset of them.

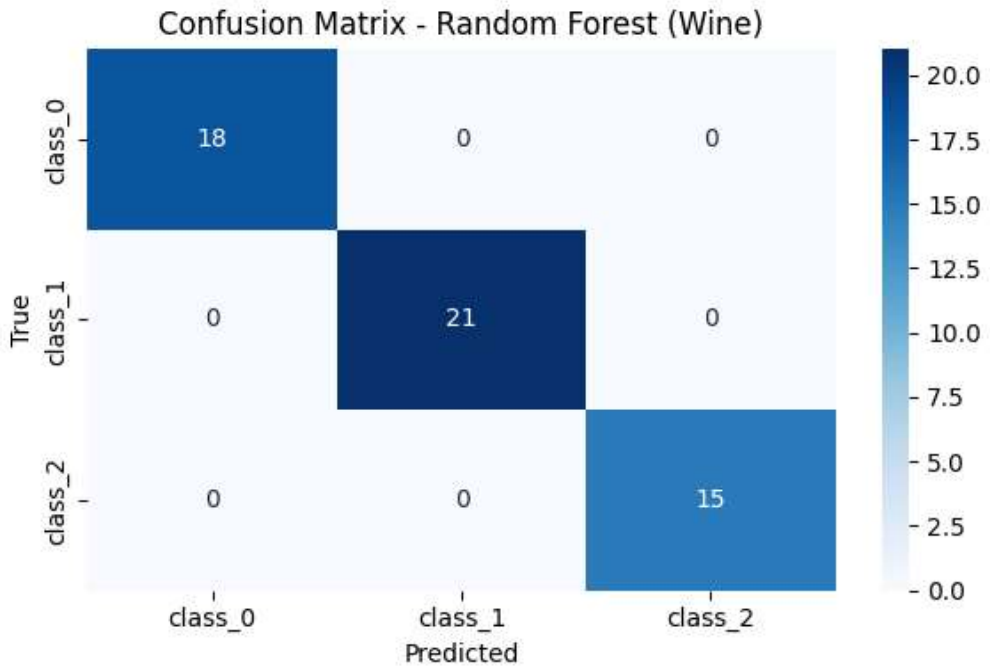


Figure 66. *Confusion Matrix – Random Forest (Wine).*

Feature importance analysis

Random Forest also provides insight into the **relative importance of input features** through Gini-based importance scores. These scores quantify how much each feature contributes, on average, to reducing node impurity across the entire forest.

As shown in **Figure 67**, features such as *color_intensity*, *flavanoids*, *alcohol*, and *proline* have the highest importance values. These variables consistently play a central role in splitting decisions across many trees, making them particularly informative for distinguishing between wine classes.

Other features, while less influential individually, still contribute to the ensemble's overall performance by providing complementary information. This distribution shows that the model does not rely on a single dominant feature, but instead integrates information from multiple predictors, which improves robustness and reduces sensitivity to noise.

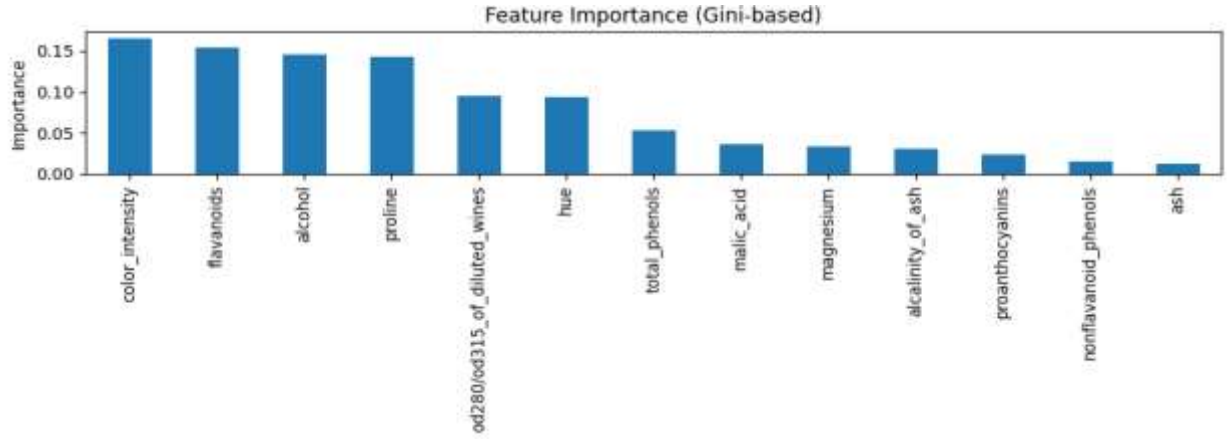


Figure 67. *Feature Importance (Gini-based).*

Conclusion

This laboratory demonstrated the application of Random Forest as a powerful ensemble method for multi-class classification. Using the Wine dataset, the experiment showed how combining multiple decision trees trained on randomized data subsets and feature selections leads to highly accurate and stable predictions.

The results highlighted the effectiveness of Random Forest in reducing the high variance typically associated with individual decision trees. Perfect classification accuracy on the test set, together with a strong out-of-bag score, confirmed that the model generalizes well and does not rely on memorization of the training data.

Analysis of the confusion matrix confirmed consistent performance across all classes, with no evidence of class-specific bias or misclassification. Feature importance analysis provided additional insight into the model's decision-making process, showing that a subset of chemically meaningful variables played a dominant role while still allowing complementary features to contribute to overall robustness.

Lab 9 – Support Vector Machines (SVM)

Objective

The objective of this laboratory session is to study and apply Support Vector Machines (SVM) as a supervised learning method for binary classification. The experiment focuses on understanding how SVM constructs decision boundaries by maximizing the margin between classes and how different kernel functions affect the geometry of the separating surface.

Specifically, this lab aims to:

- Understand the concept of margin maximization and the role of support vectors.
- Apply SVM to a two-dimensional binary classification problem.
- Compare linear and nonlinear decision boundaries using different kernel functions.
- Analyze the geometric interpretation of decision boundaries and margins.
- Observe how kernel choice influences model flexibility and class separation.

Through visualization-based experiments on a synthetic dataset, this lab provides insight into how SVM transitions from linear separators to nonlinear decision surfaces using kernel methods.

Theory

Support Vector Machines are supervised learning models designed to find an optimal separating boundary between classes. For binary classification, SVM constructs a decision boundary that maximizes the margin, defined as the distance between the boundary and the closest data points from each class.

Maximum Margin Principle

Given labeled training samples (x_i, y_i), where $y_i \in \{-1, +1\}$, SVM seeks a hyperplane defined by:

$$w \cdot x + b = 0$$

such that the margin between the two classes is maximized. Only a subset of training points (support vectors) determines the position of the hyperplane.

Soft Margin and Kernel Trick

When data are not linearly separable, SVM allows controlled misclassification using a soft margin. More importantly, SVM can construct nonlinear decision boundaries through the kernel trick, which implicitly maps input data into a higher-dimensional feature space.

Common kernels include:

- **Linear kernel**: produces a straight-line decision boundary.

- **Polynomial kernel**: allows curved boundaries with controlled complexity.
- **Radial Basis Function (RBF) kernel**: produces highly flexible, nonlinear decision regions.

Code

The SVM Code used in this experiment is available at the following GitHub repository: SVM_Classification_Code

Implementation and Analysis

The experiment is conducted on a synthetic two-dimensional dataset composed of 16 samples divided into two classes. The dataset is intentionally small and low-dimensional to allow direct visualization of the decision boundaries, margins, and support vectors produced by Support Vector Machines.

The samples are arranged such that the two classes are not trivially separable by a single geometric rule, making the dataset suitable for illustrating how different kernel functions affect the shape and flexibility of the separating surface.

The dataset remains fixed throughout the experiment. As a result, any observed differences in classification behavior can be attributed exclusively to the choice of kernel function rather than changes in data distribution.

Three SVM classifiers are trained using:

1. Linear kernel,
2. Polynomial kernel, and
3. Radial basis function (RBF) kernel.

For the nonlinear kernels, the parameter $\gamma = 2$is used to control the influence radius of individual samples, in accordance with the lab manual. This choice allows nonlinear effects to be clearly visible without producing excessively irregular decision boundaries.

Linear SVM

The linear SVM constructs a straight-line decision boundary that separates the two classes while maximizing the margin. The solid line represents the decision boundary, while the two dashed lines indicate the margin boundaries. Samples that lie exactly on these margins are identified as support vectors and are highlighted with circular markers.

As shown in **Figure 68**, only a small subset of samples determines the position of the separating hyperplane. These support vectors are the closest points to the boundary and fully define the classifier. All other points have no influence on the learned model.

The resulting separation is clean and stable, demonstrating the effectiveness of the maximum-margin principle when the data is approximately linearly separable. This visualization clearly illustrates the core idea of SVM: prioritizing margin width rather than minimizing classification error on all points.

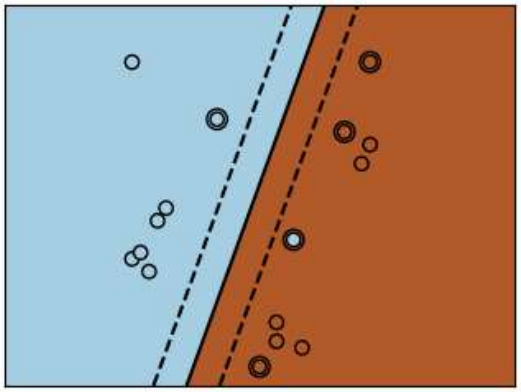


Figure 68. *Linear SVM Decision Boundary and Margins*

Polynomial Kernel SVM

The polynomial kernel introduces controlled nonlinearity into the decision function by implicitly mapping the input data into a higher-dimensional feature space. This allows the classifier to model curved decision boundaries while still maintaining a margin-based separation.

As shown in **Figure 69**, the decision boundary is no longer strictly linear. Instead, it bends to better accommodate the spatial arrangement of the samples. Compared to the linear case, more points act as support vectors, reflecting the increased complexity of the boundary.

Despite this added flexibility, the margin remains well-defined and smooth. The polynomial kernel thus represents a compromise between expressiveness and stability: it improves class separation without producing overly complex decision regions.

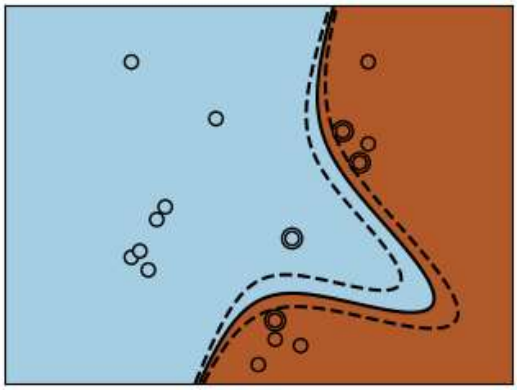


Figure 69. *Polynomial Kernel SVM Decision Boundary*

Radial Basis Function (RBF) Kernel SVM

The RBF kernel produces a highly flexible, nonlinear decision surface by modeling similarity locally around individual samples. Rather than enforcing a global trend, the classifier adapts its boundary to the local density and arrangement of the data.

As illustrated in **Figure 70**, the decision regions form curved, irregular shapes that closely follow the distribution of the samples. The margin contours are nonlinear, and a larger number of support vectors influence different parts of the boundary.

This behavior demonstrates the strength of the RBF kernel: it enables SVM to separate classes that are not linearly or polynomially separable in the original feature space. However, it also highlights the importance of parameter selection, since excessive flexibility can lead to overfitting if the model adapts too closely to individual samples.

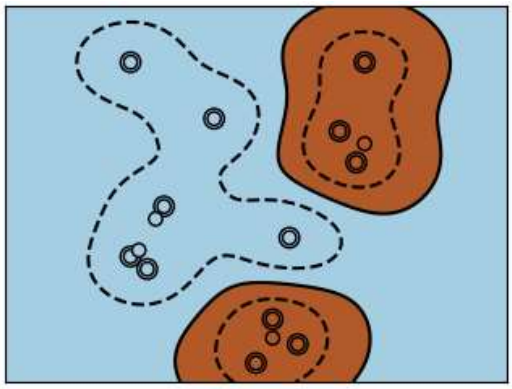


Figure 70. *RBF Kernel SVM Decision Boundary*

Conclusion

This laboratory demonstrated how Support Vector Machines construct decision boundaries by maximizing the margin between classes and how kernel functions extend this concept to nonlinear classification problems.

The linear SVM illustrated the fundamental geometric interpretation of margin-based classification, with a clear separation determined by a small number of support vectors. The polynomial kernel introduced controlled nonlinearity, allowing the classifier to better adapt to data structure while maintaining smooth decision surfaces. The RBF kernel further extended this capability by producing highly flexible boundaries capable of modeling complex class arrangements.

These experiments confirmed that kernel selection plays a critical role in balancing model expressiveness and generalization. While nonlinear kernels can significantly improve class separation, they must be chosen carefully to avoid overfitting.

Lab 10 – Gradient Descent

Objective

The objective of this laboratory session is to introduce gradient descent as a fundamental optimization method used throughout machine learning and artificial intelligence. The experiment focuses on understanding how iterative parameter updates guided by the gradient of a cost function lead to convergence toward a minimum.

Specifically, this lab aims to

- Formally introduce the gradient descent algorithm as a method for minimizing differentiable functions
- Demonstrate how gradients determine the direction and magnitude of parameter updates
- Analyze the influence of the learning rate on convergence speed, stability, and accuracy
- Build intuition for how gradient descent behaves under different step-size configurations
- Establish a conceptual foundation for later algorithms such as logistic regression, neural networks, and deep learning models

Through a simple one-dimensional quadratic function, the lab provides a clear and visual understanding of how optimization proceeds iteratively toward a minimum and why parameter tuning plays a critical role in practical applications.

Theory

Gradient descent is an optimization technique used to minimize a cost function by iteratively adjusting its parameters in the direction that reduces the function's value. The method is based on the observation that the gradient of a function points in the direction of greatest increase. Moving in the opposite direction therefore leads toward a minimum.

For a scalar-valued function $f(x)$, the gradient reduces to the derivative $f'(x)$. At any point x, the derivative indicates how the function changes locally. If the derivative is positive, increasing x increases the function value. If the derivative is negative, increasing x decreases the function value. This property allows gradient descent to move parameters toward a minimum using only local information.

The parameter update rule is defined as

$$x \leftarrow x - \eta \cdot f'(x)$$

where η is the learning rate. The learning rate controls the size of each update step. A small learning rate produces slow but stable convergence. A large learning rate accelerates movement but can lead to oscillations or divergence.

Code

The Gradient_Descent_Code used in this experiment is available at the following GitHub repository: Gradient_Descent_Code

Implementation and Analysis

In this laboratory, gradient descent is applied to the quadratic function: **$f(x) = (x - 3)^2$.**
The derivative of this function is: **$f'(x) = 2(x - 3)$.**

The function has a single global minimum at x = 3. This property makes it ideal for studying gradient descent behavior because convergence can be evaluated unambiguously.

The direction of movement follows a simple rule. If x is greater than 3, the gradient is positive and the update moves x to the left. If x is less than 3, the gradient is negative and the update moves x to the right. As a result, every update moves the parameter toward the minimum. This example captures the essential idea behind gradient descent as used in neural networks, logistic regression, and deep learning architectures.

The function to be minimized and its gradient are defined explicitly. Gradient descent is implemented as an iterative algorithm that records the parameter value at each step to allow visualization of the optimization trajectory.

The experiment evaluates gradient descent under three learning-rate regimes. A moderate learning rate that achieves fast and stable convergence. A small learning rate that converges slowly. A large learning rate that demonstrates instability.

Moderate learning rate

The first experiment uses a learning rate of 0.1, which is recommended in the lab manual as a balanced choice. The algorithm starts from x = 10 and performs 25 iterations.

As shown in **Figure 71**, the sequence of updates moves smoothly toward the minimum at x = 3. The red markers indicate successive parameter values and form a clear descending path along the function curve. The updates decrease in magnitude as the algorithm approaches the minimum, reflecting the reduction of the gradient near the optimum.

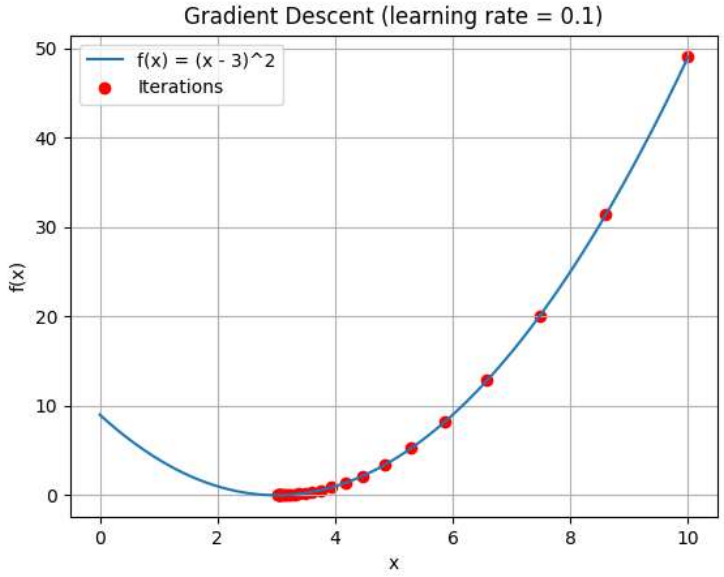


Figure 71. *Gradient descent trajectory on f(x) = (x − 3)² with learning rate 0.1*

Figure 72 shows the corresponding decrease of the objective function value across iterations. The error drops rapidly during the first iterations and then flattens near zero. This behavior indicates fast and stable convergence without oscillations or divergence.

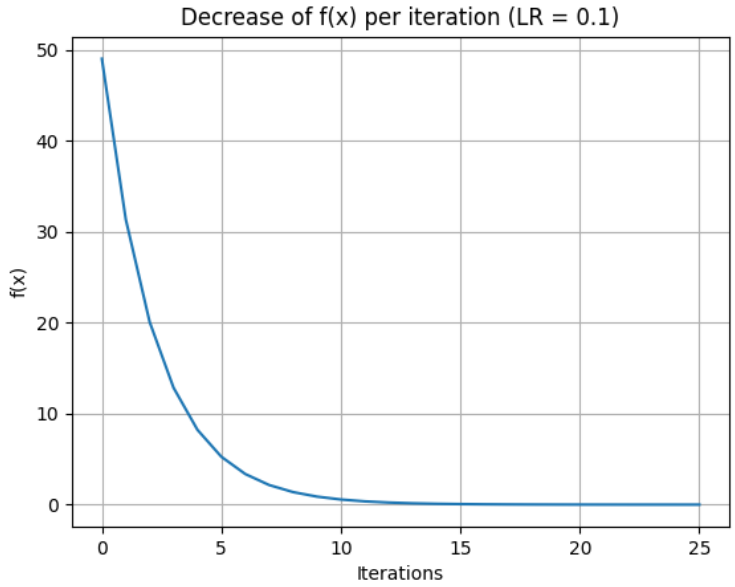


Figure 72. *Decrease of f(x) per iteration for learning rate 0.1*

Small learning rate

The second experiment uses a learning rate of 0.01 to illustrate slow convergence. The same starting point and number of iterations are used to enable direct comparison.

As shown in **Figure 73**, the parameter updates move in the correct direction but advance very slowly toward the minimum. After 25 iterations, the algorithm remains far from x = 3. The update steps are small, and the optimization path covers only a limited portion of the curve.

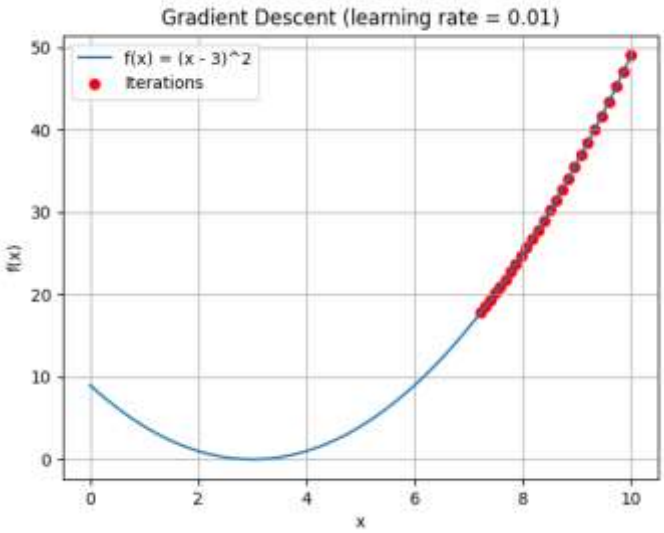


Figure 73. *Gradient descent trajectory on f(x) = (x − 3)² with learning rate 0.01*

Figure 74 confirms this observation. The error decreases gradually and almost linearly, with no sharp drop. This result demonstrates that an excessively small learning rate leads to inefficient optimization, requiring many iterations to reach an acceptable solution.

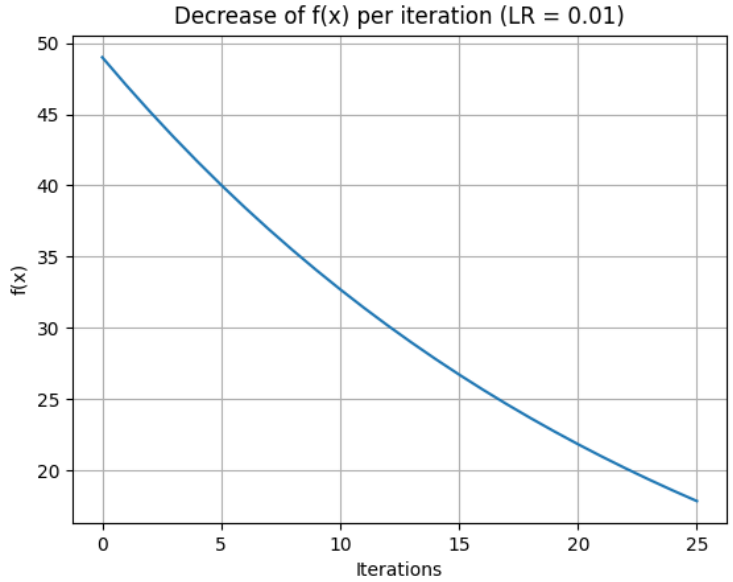


Figure 74. *Decrease of f(x) per iteration for learning rate 0.01*

Large learning rate

The final experiment uses a learning rate of 0.8 to demonstrate instability. This value is intentionally large relative to the curvature of the function.

As shown in **Figure 75**, the parameter updates overshoot the minimum and oscillate across the function. Some steps move far beyond the optimal region, producing erratic jumps in parameter space. Although the function is simple and convex, the algorithm fails to follow a smooth descent path.

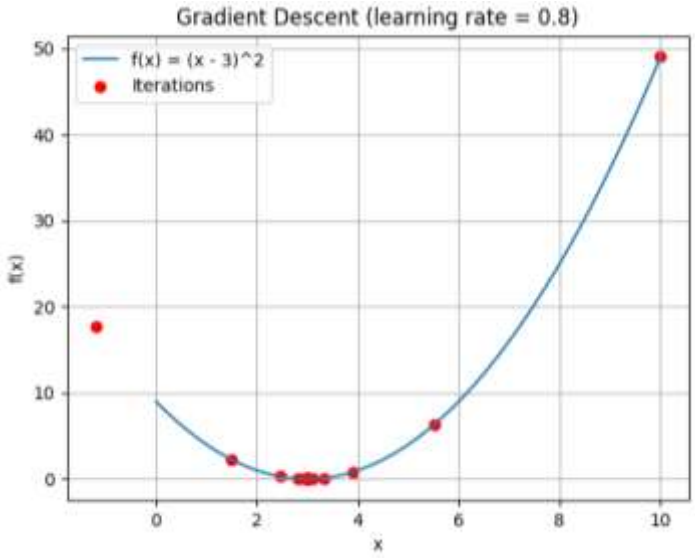


Figure 75. *Gradient descent trajectory on f(x) = (x − 3)² with learning rate 0.8*

Figure 76 illustrates the effect on the objective function value. The error decreases sharply at first but does so in an unstable manner, driven by large corrective steps rather than controlled convergence. This behavior highlights the risk of divergence or numerical instability when the learning rate is too large.

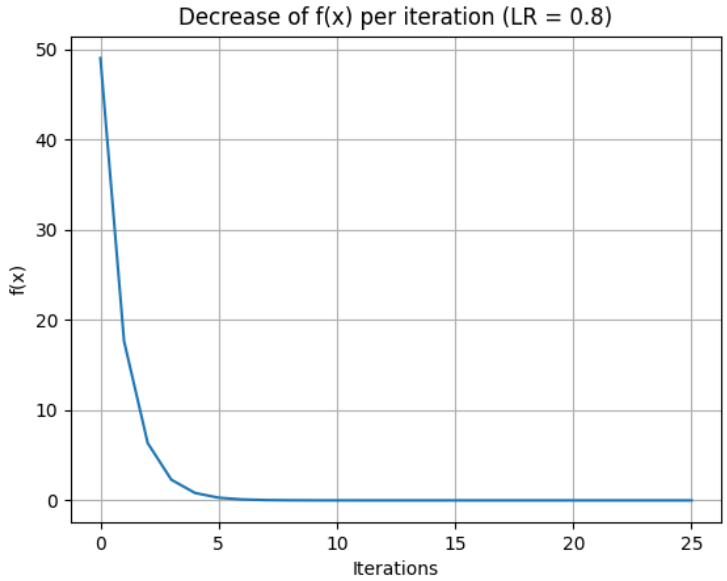


Figure 76. *Decrease of f(x) per iteration for learning rate 0.8*

Conclusion

This laboratory provided a clear and practical introduction to gradient descent as a fundamental optimization algorithm in machine learning. By applying gradient descent to a simple one-dimensional quadratic function, the experiment demonstrated how iterative updates guided by the gradient lead to convergence toward a minimum.

The results highlighted the central role of the learning rate in determining optimization behavior. A moderate learning rate produced fast and stable convergence, efficiently guiding the parameter toward the global minimum. A small learning rate ensured stability but resulted in slow progress, requiring many iterations to achieve meaningful improvement. A large learning rate caused instability, with parameter updates overshooting the minimum and producing erratic behavior despite the simplicity of the objective function.

Visualizing both the optimization trajectory and the evolution of the objective function value was essential for building intuition. These plots made it possible to directly observe convergence speed, stability, and failure modes under different step-size choices. The experiment showed that correct gradient direction alone is not sufficient for effective optimization. Appropriate step-size selection is equally critical.

Lab 11 – Perceptron

Objective

The objective of this laboratory session is to study the perceptron as the fundamental building block of neural networks and to understand how gradient descent is used to train a single artificial neuron for binary classification. The experiment focuses on implementing a perceptron from first principles using Python and analyzing how its parameters are adjusted through backpropagation to minimize a cost function.

Specifically, this lab aims to:

- Introduce the perceptron as a simplified model of a biological neuron used for supervised binary classification
- Understand how a weighted linear combination of input features, combined with a bias term, defines a decision boundary in feature space
- Apply the sigmoid activation function to transform linear outputs into probabilistic predictions
- Use Mean Squared Error (MSE) as a cost function to quantify prediction error
- Train the perceptron using gradient descent and backpropagation to iteratively update weights and bias
- Visualize the learning process through loss curves and decision boundary plots
- Compare a manual Python implementation with an equivalent implementation using the Keras library

This laboratory serves as a conceptual bridge between gradient descent optimization (studied previously) and more complex neural network architectures that will be explored in later experiments.

Theory

The perceptron is the simplest form of an artificial neural unit and represents the core computational element of neural networks. It is designed to perform binary classification by mapping an input feature vector to a single output value that represents the probability of belonging to a given class.

Perceptron model

Given an input vector

$$\mathbf{x} = (x_1, x_2, \dots, x_n)$$

the perceptron computes a weighted sum of its inputs and adds a bias term:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

where:

- w_i are the weights associated with each input feature,
- b is the bias term,
- z is the linear combination of inputs.

This linear expression defines a decision boundary in the input feature space. For two input features, this boundary corresponds to a straight line that separates the two classes.

Activation function

To convert the linear output into a meaningful prediction, the perceptron applies an activation function. In this lab, the sigmoid function is used:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The sigmoid maps any real-valued input to the interval $[0, 1]$, allowing the output to be interpreted as the probability that the input belongs to class 1. This probabilistic interpretation makes the perceptron suitable for supervised binary classification tasks.

Loss function

To evaluate how well the perceptron performs, a cost function is required. The Mean Squared Error (MSE) is used in this experiment:

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

where y_i is the true label and $\hat{y}_i$ is the predicted output. The loss measures the discrepancy between predictions and ground-truth labels and provides a quantitative objective to minimize during training.

Training with gradient descent and backpropagation

Training the perceptron consists of adjusting the weights and bias to minimize the loss function. This is achieved using gradient descent, an iterative optimization method that updates parameters in the opposite direction of the gradient of the loss.

Backpropagation is used to compute the gradients efficiently by applying the chain rule. The error at the output is propagated backward through the activation function to determine how each weight and the bias contribute to the overall error. The parameters are then updated according to:

$$w \leftarrow w - \eta\frac{\partial L}{\partial w}, b \leftarrow b - \eta\frac{\partial L}{\partial b}$$

where η is the learning rate controlling the step size of each update.

Although the perceptron used in this lab consists of only a single neuron, the training mechanism is identical to that used in larger neural networks. Understanding this simple case provides essential insight into how more complex architectures learn from data.

Code

The Perceptron Code used in this experiment is available at the following GitHub repository: Perceptron_Code

Implementation and Analysis

This laboratory implements and analyzes a single perceptron trained using gradient descent for binary classification. The implementation is divided into three main stages: dataset generation and visualization, manual perceptron training using Python, and an equivalent implementation using the Keras library. The results are evaluated through loss curves and decision boundary visualizations.

Dataset generation and visualization

A synthetic two-dimensional dataset was generated using the make_classification() function from the scikit-learn library. The dataset consists of 100 samples, two numerical features, and two class labels (0 and 1). The configuration ensures that the classes are linearly separable, making the dataset suitable for analysis with a single perceptron.

The dataset was split into training and test sets using an 80/20 ratio. Class labels were reshaped into column vectors to facilitate matrix operations during training.

Figure 77 shows the initial training data distribution. The two classes form distinct clusters in feature space, with partial overlap near the center. This visualization highlights the classification task faced by the perceptron: learning a linear decision boundary that separates the two classes as effectively as possible.

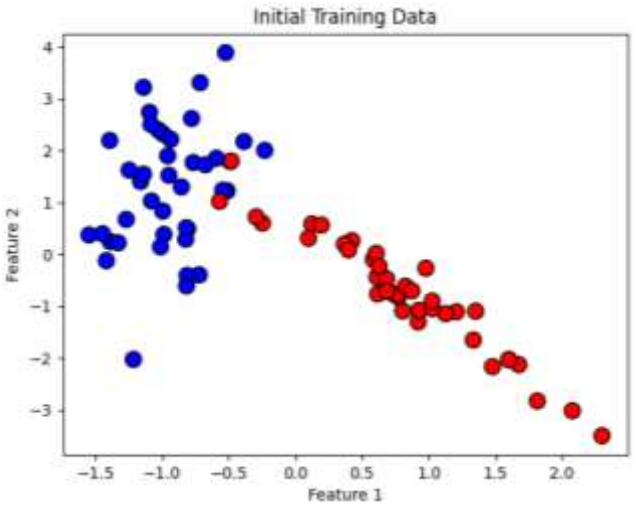


Figure 77. *Initial training data distribution*

Pure Python perceptron implementation

The perceptron was implemented manually using NumPy to explicitly illustrate the learning mechanism of a single artificial neuron. The model consists of two trainable weights (one per input feature) and a bias term. Together, these parameters define a linear decision boundary in the two-dimensional feature space.

The perceptron computes a weighted sum of the input features, adds a bias, and applies the sigmoid activation function to produce a probabilistic output. Mean Squared Error (MSE) was selected as the loss function to quantify prediction error.

Training was performed using gradient descent over 1000 epochs. At each epoch, the perceptron performs a forward pass to generate predictions, computes the loss, and applies backpropagation to update the weights and bias. The learning rate was fixed at 0.1, providing a balance between convergence speed and stability.

Figure 78 illustrates the evolution of the training loss over epochs. The loss decreases rapidly during the initial iterations, indicating effective learning, and then gradually flattens as the model approaches convergence. This behavior is characteristic of gradient-based optimization, where parameter updates become smaller near a minimum.

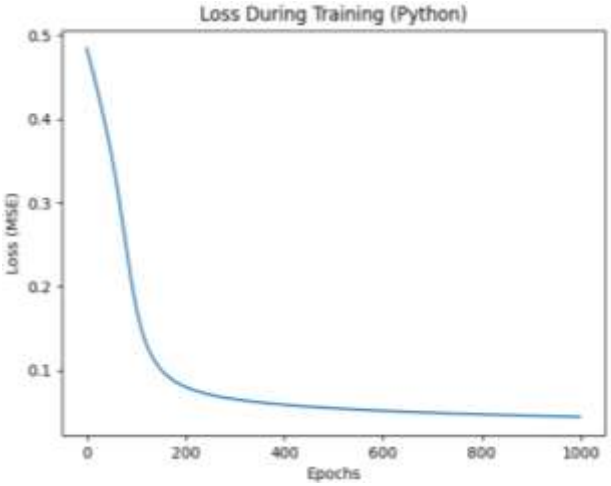


Figure 78. *Loss during training (Python perceptron)*

After training, the learned weights and bias were used to visualize the perceptron's decision boundary. A grid of points spanning the feature space was evaluated, and predicted probabilities were used to color regions according to the model's output.

Figure 79 shows the resulting decision boundary. The perceptron learns a straight-line separation that aligns well with the structure of the data. Most samples from class 0 and class 1 fall on opposite sides of the boundary, demonstrating that a single neuron is sufficient for this linearly separable problem.

Misclassifications occur primarily near the boundary, where class overlap exists. This outcome reflects limitations of linear models rather than training failure.

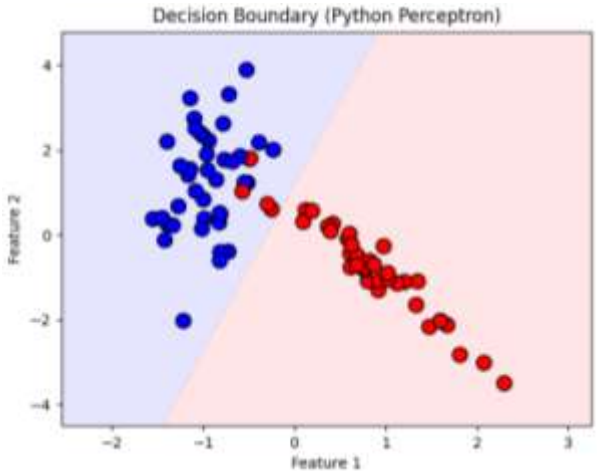


Figure 79. *Decision boundary learned by the Python perceptron*

Keras perceptron implementation

To demonstrate how modern machine learning frameworks abstract low-level operations, the perceptron was reimplemented using the Keras API. The model architecture consists of a single dense neuron with a sigmoid activation function and an explicitly defined input layer.

The model was trained on the same dataset using stochastic gradient descent with the same learning rate and loss function as the Python implementation. This ensures a fair comparison between the two approaches.

Figure 80 shows the loss curve during Keras training. The loss decreases smoothly and converges to a slightly lower value than the manual implementation. Minor differences are expected due to different parameter initialization and internal numerical optimizations performed by TensorFlow.

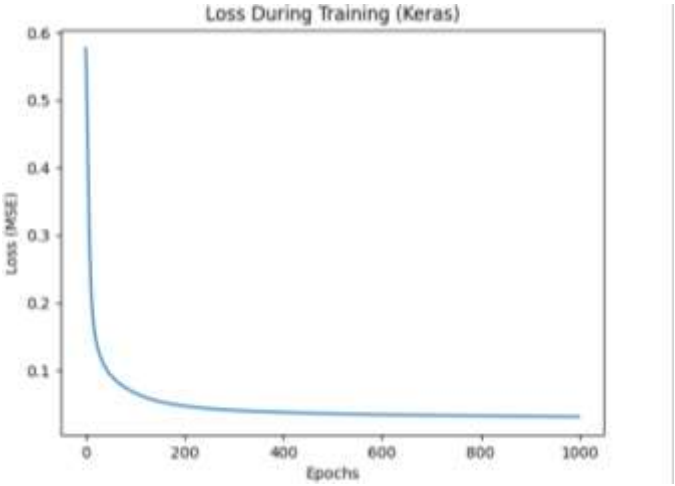


Figure 80. *Loss during training (Keras perceptron)*

Figure 81 presents the decision boundary learned by the Keras perceptron. The boundary closely matches that of the Python implementation, confirming that both models represent the same underlying mathematical formulation.

Small variations in boundary position are due to stochastic initialization and optimization differences but do not affect overall classification behavior. This visual agreement validates the correctness of the manual implementation and highlights the consistency between low-level and high-level approaches.

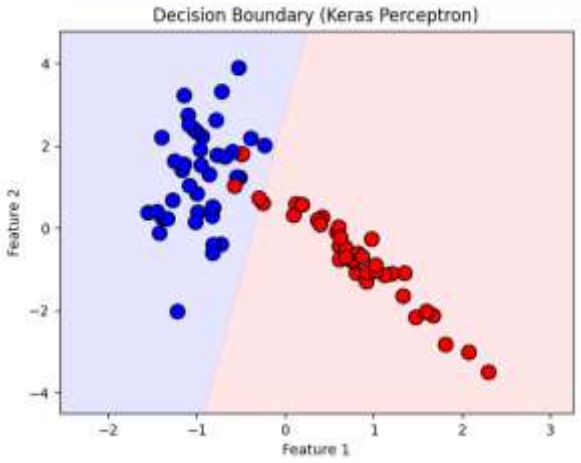


Figure 81. *Decision boundary learned by the Keras perceptron*

Conclusion

This laboratory explored the perceptron as the fundamental building block of neural networks and demonstrated how a single artificial neuron can be trained using gradient descent to perform binary classification. By focusing on a minimal architecture, the experiment provided clear insight into the mechanics of supervised learning at the neuron level.

The results showed that a perceptron with only two weights and a bias is capable of learning a linear decision boundary that effectively separates two classes in a two-dimensional feature space. The decreasing loss curves observed during training confirmed that gradient descent successfully minimized the error by iteratively adjusting the model parameters.

Comparing the pure Python implementation with the Keras-based implementation reinforced the conceptual equivalence between low-level and high-level approaches. Python implementation made the learning process explicit through manual forward and backward passes, while the Keras model achieved the same outcome using a compact and abstracted interface.

The experiment also highlighted an important limitation of the perceptron. Because the model can learn only linear decision boundaries, it can correctly separate data only when the classes are linearly separable. The misclassifications observed near the decision boundary emphasize the need for more complex architectures when modeling non-linear patterns.

Lab 12 – Neuron Network

Objective

The objective of this laboratory session is to examine artificial neural networks as a general-purpose framework for supervised learning. The focus is placed on understanding how neural network models are structured, trained, and evaluated for both binary and multiclass classification problems.

This laboratory aims to:

- Present the fundamental components of feedforward neural networks
- Explain how hidden layers enable the modeling of non-linear relationships
- Apply neural networks to a real-world binary classification task involving customer churn
- Analyze how architectural choices influence predictive performance
- Introduce dropout as a regularization technique for mitigating overfitting
- Extend the approach to multiclass classification using the Iris dataset
- Assess model performance using loss functions, accuracy metrics, and confusion matrices

These experiments illustrate how neural networks extend linear models by learning complex feature interactions in high-dimensional settings.

Theory

Artificial neural networks are function-approximation models inspired by biological neural systems. In machine learning, they are employed to learn mappings between input features and output targets through layered transformations. Each layer applies a weighted combination of inputs followed by a non-linear activation, enabling the model to represent complex relationships.

Learning occurs through iterative adjustment of internal parameters so that predicted outputs align more closely with observed labels. This adaptive process allows neural networks to capture patterns that are not accessible to models based solely on linear decision boundaries.

Although neural networks offer strong representational power and flexibility, this capability comes with increased computational cost and reduced interpretability compared to simpler algorithms.

Network Architecture

A feedforward neural network consists of three principal components:

- **Input layer -** The input layer receives feature values directly from the dataset, with each neuron corresponding to a single input variable.

- **Hidden layers -** Hidden layers perform intermediate transformations. Each neuron computes a weighted sum of its inputs, incorporates a bias term, and applies an activation function. Stacking multiple hidden layers increases the model's capacity to represent non-linear transformations.
- **Output layer -** The output layer generates the final prediction.

The number of layers and neurons determines model capacity and directly influences both learning behavior and generalization performance.

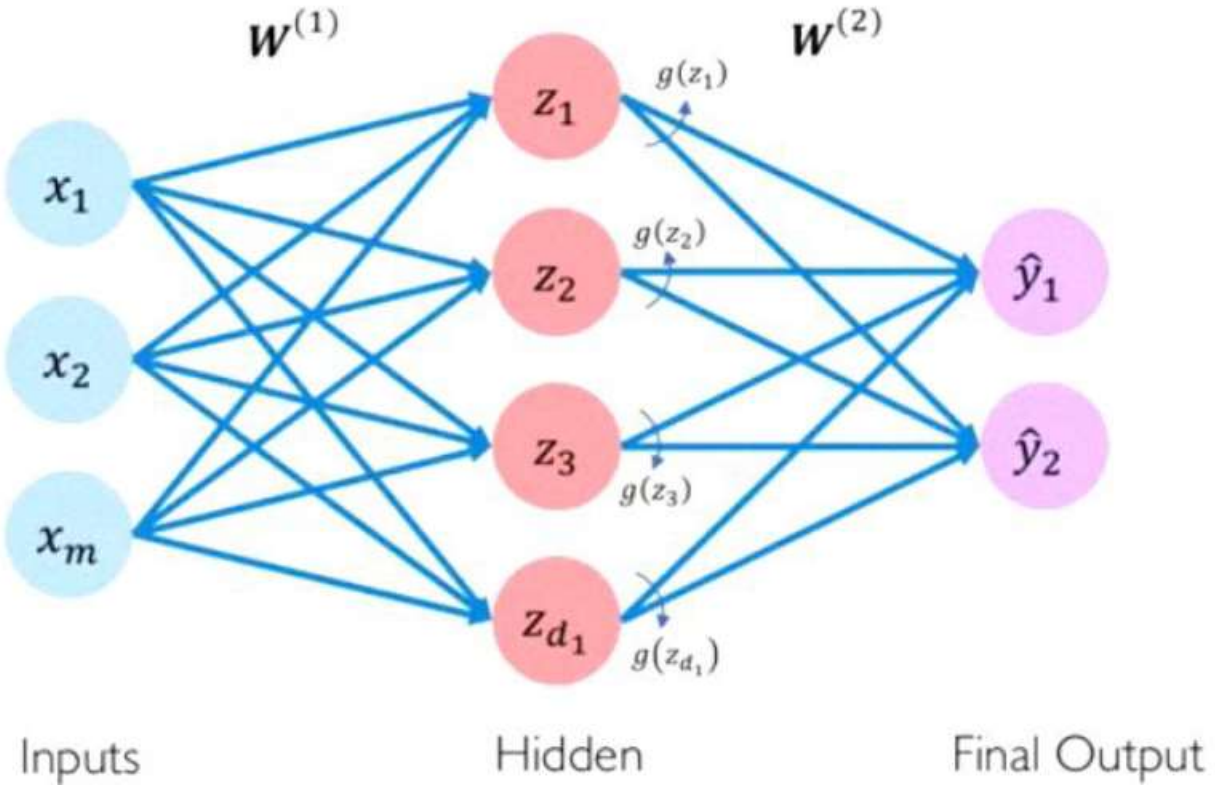


Figure 82. *Neural Network Architecture*

Forward Propagation

During forward propagation, data passes sequentially through the network from input to output. Each layer transforms its input into a higher-level representation using weighted sums and activation functions. The final layer produces either a probability estimate for a single class or a probability distribution across multiple classes.

This mechanism defines how predictions are generated for a fixed set of model parameters.

Activation Functions

Activation functions introduce non-linearity, enabling neural networks to learn complex decision boundaries.

The following activation functions are used in this laboratory:

- **ReLU -** Outputs zero for negative inputs and returns the input value for positive inputs. Its efficiency and gradient behavior make it suitable for hidden layers.
- **Sigmoid -** Maps real-valued inputs to the interval [0, 1], allowing outputs to be interpreted as probabilities in binary classification.
- **Softmax -** Converts raw scores into a normalized probability distribution across multiple classes, making it appropriate for multiclass tasks.

Loss Functions and Learning Objective

Training is guided by a loss function that measures the discrepancy between predictions and true labels.

- **Binary cross-entropy** is employed for binary classification tasks.
- **Categorical or sparse categorical cross-entropy** is used for multiclass classification with softmax outputs.

Minimizing the loss function defines the learning objective and directs parameter updates.

Backpropagation and Optimization

Parameter updates are computed using backpropagation, which applies the chain rule to calculate gradients of the loss with respect to each weight and bias. These gradients are used to iteratively refine model parameters through an optimization algorithm.

The Adam optimizer is used in this lab due to its adaptive learning-rate mechanism and reliable convergence properties.

Overfitting and Regularization

High-capacity models risk overfitting by memorizing training data rather than learning generalizable patterns. This leads to degraded performance on unseen data.

Dropout regularization addresses this issue by randomly deactivating a subset of neurons during training. This constraint encourages distributed learning and reduces dependency on individual neurons, improving generalization performance.

Code

The Neuron Network Code used in this experiment is available at the following GitHub repository: Neuron_Network_Code

Implementation and Analysis

This section describes the implementation of neural networks following the workflow specified in the lab manual and analyzes the resulting performance. The experiments progress through data

preprocessing, baseline model construction, architectural variation, regularization, validation, and extension to multiclass classification.

Binary Classification: Customer Churn Prediction

The churn dataset contains customer attributes represented by a mix of numerical and categorical variables, along with a binary target indicating whether a customer exited the bank. Input features were selected according to the column ranges specified in the lab manual, and the target variable was extracted as a binary label.

Categorical variables (Geography and Gender) were encoded using one-hot encoding, while numerical variables were standardized using z-score normalization. Standardization supports stable gradient-based optimization by ensuring that all features contribute at comparable scales. Redundant dummy variables were removed after encoding to avoid collinearity and unnecessary dimensionality, consistent with the manual's procedure.

The processed dataset was split into training and test subsets using an 80–20 ratio. This separation allows model performance to be evaluated on unseen data, providing a reliable measure of generalization.

A baseline feedforward neural network was implemented with two hidden layers and one output layer. Each hidden layer contains six neurons with ReLU activation, followed by a single sigmoid-activated output neuron. This architecture matches the reference configuration proposed in the lab manual and provides sufficient capacity to model non-linear relationships in the churn data.

The model was compiled using the Adam optimizer and binary cross-entropy loss. Training was performed for 100 epochs with a batch size of 10, and training loss was recorded to monitor convergence behavior.

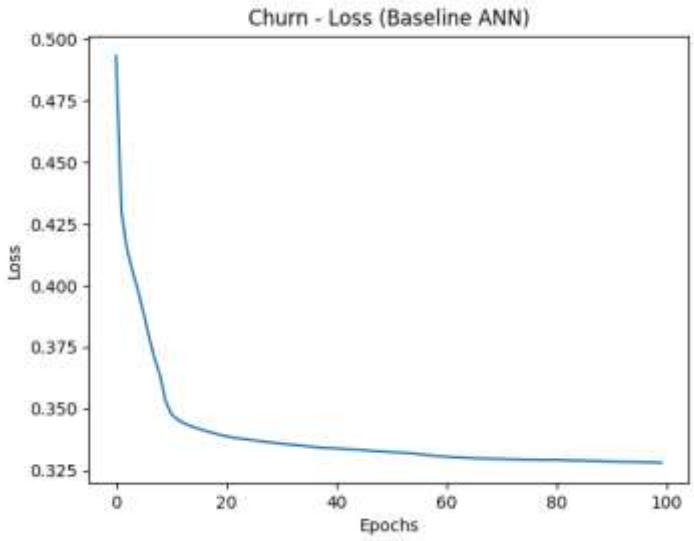


Figure 83. *Training loss evolution for the baseline neural network applied to the churn dataset.*

Figure 83 shows a rapid decrease of the loss curve during the initial epochs, followed by a gradual flattening as training progresses. This pattern indicates effective learning and stable convergence. The smooth trajectory without oscillations suggests that the chosen optimizer, learning rate adaptation, and network architecture are well suited to the problem.

Model Evaluation

After training, predictions were generated on the test set and converted into class labels using a probability threshold of 0.5. The baseline network achieved a test accuracy of **0.8635**, demonstrating strong predictive performance for a real-world classification task.

The classification report reveals high recall for customers who remain with the bank (class 0 recall = 0.94) and lower recall for customers who exit (class 1 recall = 0.55). Precision for the churn class remains reasonable (0.71), indicating that when the model predicts churn, the prediction is often correct. This imbalance reflects the underlying data distribution and the difficulty of separating churn behavior from overlapping customer profiles, rather than a failure of the modeling approach.

Single-Customer Prediction

A single customer record was constructed and passed through the same preprocessing pipeline before inference. The model predicted that the customer would remain with the bank. This step demonstrates correct operational usage and highlights the importance of applying identical preprocessing steps during deployment as during training.

To analyze the impact of model capacity, two alternative architectures were evaluated:

- A smaller network with one hidden layer containing four neurons
- A larger network with two hidden layers containing twelve and six neurons

The smaller network achieved an accuracy of **0.8615**, slightly below the baseline, indicating limited representational capacity. The larger network achieved an accuracy of **0.8595**, showing no meaningful improvement over the baseline.

These results demonstrate that increasing architectural complexity beyond a moderate level does not necessarily improve generalization. The baseline architecture captures most of the learnable structure in the data, reinforcing the principle that appropriate capacity is more effective than maximal capacity.

Dropout Regularization

A dropout-based architecture was implemented to address potential overfitting. The network includes a large hidden layer with 128 neurons, followed by a dropout layer with a rate of 0.2 and a sigmoid output neuron.

Despite the increased number of parameters, the dropout network achieved a test accuracy of **0.862**, comparable to the baseline model. This result confirms that dropout successfully constrains

learning by discouraging reliance on individual neurons, maintaining generalization even in higher-capacity architectures. The benefit of dropout in this experiment lies in robustness rather than raw accuracy improvement.

Cross-Validation

Ten-fold cross-validation was applied to the baseline architecture to assess performance stability across different training subsets. The mean cross-validation accuracy was **0.8584**, with a standard deviation of **0.0089**.

The close agreement between cross-validation performance and test-set accuracy indicates consistent behavior across folds. The low variance confirms that the observed results are not dependent on a particular train–test split.

Grid search was included in the implementation in accordance with the lab manual to illustrate hyperparameter optimization, although full execution can be computationally expensive on CPU-based systems.

Multiclass Classification: Iris Dataset

The final experiment extends neural network modeling to a multiclass classification task using the Iris dataset. Input features were standardized, and a feedforward network with two hidden layers and a softmax output layer was trained using sparse categorical cross-entropy loss.

The trained model achieved high accuracy on the test set and produced a near-diagonal confusion matrix, indicating effective separation among the three flower classes. This result demonstrates that neural networks generalize naturally from binary to multiclass classification by modifying the output layer and loss function, without altering the underlying learning process.

Conclusion

This laboratory explored artificial neural networks as a flexible and effective framework for supervised classification. Through a sequence of controlled experiments, neural networks were applied to both binary and multiclass problems, demonstrating their ability to model non-linear relationships that extend beyond the capabilities of simpler linear classifiers.

On the customer churn dataset, the baseline feedforward neural network achieved strong predictive performance, with stable training behavior and consistent generalization across test data and cross-validation folds. Architectural variations showed that moderate network capacity was sufficient to capture the underlying structure of the data, while increased complexity did not yield meaningful improvements. The introduction of dropout regularization preserved performance while reinforcing robustness against overfitting, particularly in higher-capacity configurations.

The extension to multiclass classification using the Iris dataset confirmed the adaptability of neural networks. By adjusting the output layer and loss function, the same learning principles were applied to a multi-class setting, resulting in accurate predictions and clear class separation.

Lab 13 – MINST Classification with Autoencoders and Keras

Objective

The objective of this laboratory session is to study the use of **autoencoders** for learning compact feature representations from high-dimensional image data and to evaluate how these representations can be exploited for **digit classification**.

The specific goals of this lab are to:

- Understand the fundamental principles of autoencoders and latent space representation.
- Apply unsupervised learning to compress handwritten digit images from the MNIST dataset.
- Analyze how dimensionality reduction can preserve essential visual information.
- Prepare latent representations suitable for supervised classification.
- Develop intuition about representation learning as a preprocessing step for deep learning models.

Theory

Autoencoders are neural network models designed to learn compact and informative representations of data through an unsupervised learning process. The central idea is to train a network to reproduce its input at the output layer, forcing the internal structure of the model to encode the most salient features of the data. This reconstruction task encourages the network to discard redundant information and retain only the patterns that are essential for faithfully representing the input.

The architecture of an autoencoder is organized around a compression and reconstruction process. High dimensional input data are progressively transformed through a series of fully connected layers into a much lower dimensional internal representation, commonly referred to as the latent features. This transformation is learned automatically by minimizing the discrepancy between the original input and the reconstructed output. The reconstruction objective serves as the sole training signal and does not rely on labeled data.

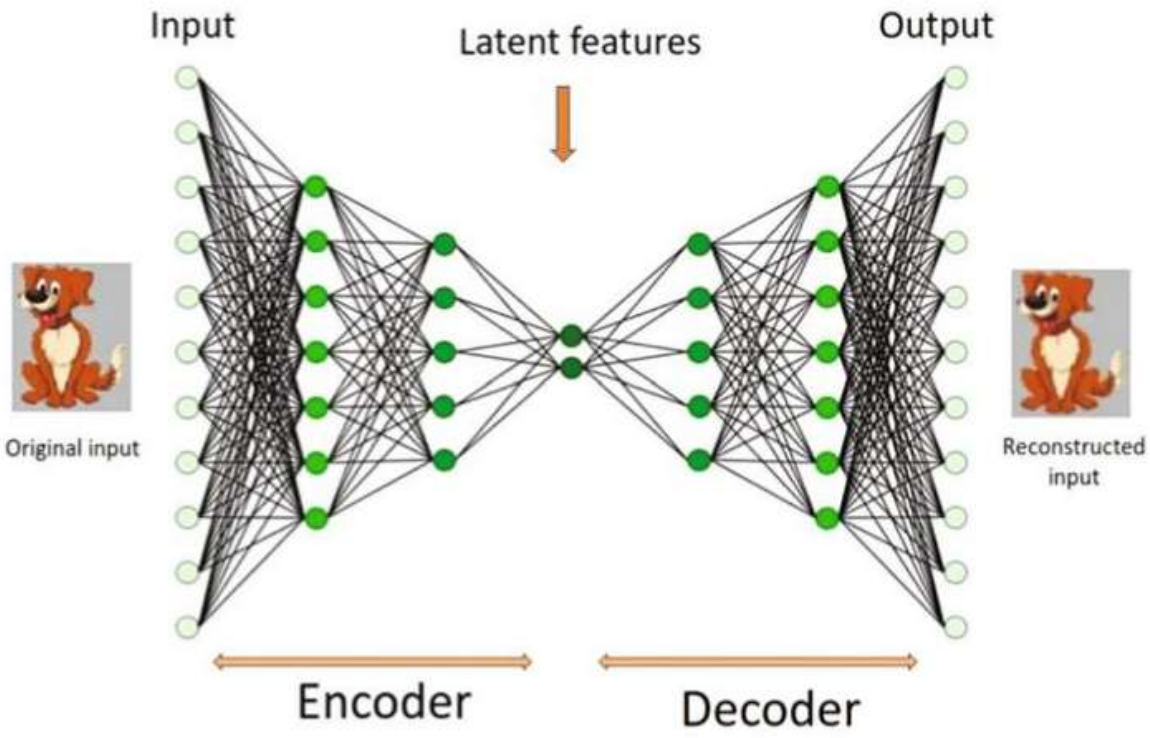


Figure 84. A*rchitecture of an autoencoder.*

The latent representation produced by the network constitutes a dense and abstract encoding of the input data. Rather than preserving raw pixel values, this representation captures structural and semantic characteristics that define the data distribution. In the context of image data, such representations tend to encode information related to shape, stroke patterns, and spatial organization. The dimensionality reduction imposed by the network acts as a form of regularization, promoting generalization and robustness by limiting the capacity to memorize noise.

Reconstruction plays a critical role in validating the quality of the learned representation. Accurate reconstruction indicates that the latent features preserve sufficient information to recover the original input with minimal loss. Poor reconstruction, on the other hand, signals that the compression is overly aggressive or that important information has been discarded. The balance between compression and reconstruction fidelity defines the effectiveness of the learned representation.

Autoencoders provide a nonlinear alternative to classical dimensionality reduction techniques such as Principal Component Analysis. Linear methods constrain representations to lie in linear subspaces, whereas neural network-based encodings can model complex, nonlinear relationships inherent in real world data. This capability makes autoencoders particularly suitable for high dimensional inputs such as images, where pixel correlations are rarely linear.

The representations learned by autoencoders are frequently reused as inputs to subsequent learning stages. Compact latent features reduce computational complexity and often improve learning efficiency for downstream models. This separation between representation learning and task specific modeling forms a foundation for many modern deep learning pipelines, where unsupervised feature extraction precedes supervised decision making.

Finally, the structure of the learned latent space provides insight into how the model organizes data internally. Samples with similar characteristics tend to occupy nearby regions of the latent space, reflecting underlying similarities in the input. Visual analysis of this space can reveal meaningful structure and class separation, offering qualitative evidence of successful representation learning.

Code

The MINST Code used in this experiment is available at the following GitHub repository: MINST_Autoencoder_Code

Implementation and Analysis

Data preprocessing

The MNIST dataset was loaded using keras.datasets.mnist.load_data(), providing a training set and an independent test set. Pixel values were converted to float32 and normalized to the interval [0, 1] in order to stabilize optimization during neural network training. Each 28×28 grayscale image was reshaped into a 784-dimensional vector to match the fully connected architecture used throughout the experiment.

To reduce computational cost while preserving representative digit variability, a subset of 10,000 training samples was selected. The full test set was retained for evaluation, ensuring an unbiased assessment of model performance. This setup follows the procedure outlined in the lab manual.

Autoencoder construction and training

A symmetric dense autoencoder was implemented using the Keras Functional API. The encoder progressively compresses the 784-dimensional input through two hidden layers of 128 and 64 neurons with ReLU activation into a 32-dimensional latent representation. The decoder mirrors this structure, expanding the latent vector back to the original dimensionality through layers of 64 and 128 neurons, followed by a sigmoid output layer consistent with normalized pixel intensities.

The model was trained using the Adam optimizer and Mean Squared Error loss. Training was performed for 10 epochs with a batch size of 256, using the test set for validation. The reconstruction loss decreased steadily across epochs, indicating that the autoencoder learned a stable and compact representation of digit structure. Final validation loss converged to approximately 0.029, reflecting effective compression without excessive information loss.

Latent feature extraction

After training, the encoder submodel was used to transform both the training subset and the test set into 32-dimensional latent vectors. These vectors constitute compact feature representations of the original digit images and replace raw pixel inputs for the classification stage. Dimensionality was reduced from 784 to 32 while preserving salient visual characteristics such as stroke orientation, curvature, and spatial layout.

Digit labels were converted to one-hot vectors using to_categorical, producing ten-class targets. A dense classifier was trained on the latent vectors using a single hidden layer of 64 neurons with ReLU activation and a softmax output layer for class prediction. The model was optimized using categorical cross-entropy loss and evaluated using accuracy.

Training accuracy increased consistently across epochs, with validation accuracy following a similar trend. Final evaluation on the test set produced a test accuracy of 0.8484 and test loss of 0.5111. These results confirm that the latent representations learned by the autoencoder preserve sufficient discriminative information for digit classification despite strong dimensionality reduction. Remaining errors are consistent with intrinsic overlap between visually similar handwritten digits and the limited expressiveness of a fully connected classifier operating in latent space.

The 32-dimensional latent vectors of the test set were projected into two dimensions using t-SNE to support qualitative analysis of representation structure. The resulting visualization reveals clear clustering patterns associated with digit identity. Digits sharing similar stroke patterns exhibit partial overlap, providing an intuitive explanation for observed classification ambiguity.

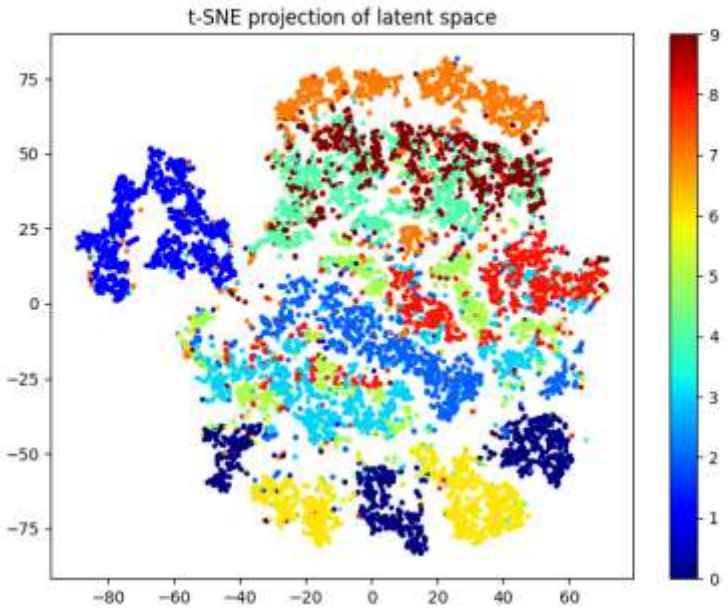


Figure 85. T-*SNE projection of the 32-dimensional latent space for the MNIST test set.*

Qualitative inspection using random predictions

A set of ten randomly selected test images was used to inspect classifier behavior at the individual sample level. Each image is displayed alongside the predicted probability distribution over the ten digit classes. Confident predictions appear as a dominant probability peak, while ambiguous digits produce more distributed probability mass across competing classes.

Misclassifications correspond to samples exhibiting irregular handwriting, incomplete strokes, or atypical digit shapes. This qualitative analysis complements numerical accuracy by illustrating how uncertainty is distributed in challenging cases.

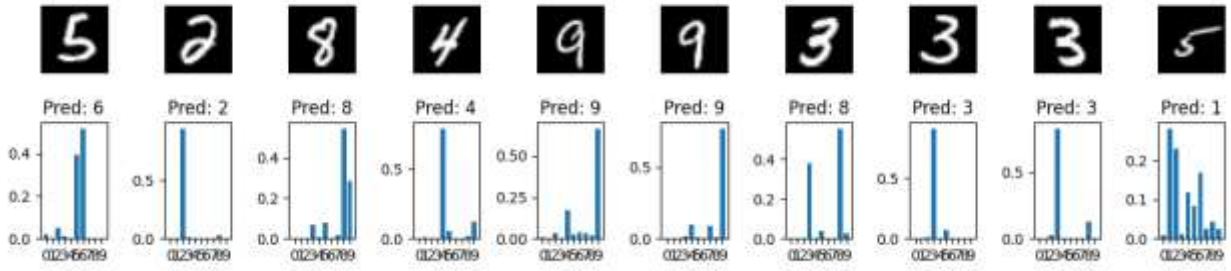


Figure 86. *Random MNIST test samples with predicted probability distributions over the ten-digit classes*

Conclusion

This laboratory investigated the use of autoencoders for unsupervised representation learning and their application to handwritten digit classification using the MNIST dataset. A dense autoencoder was employed to compress high-dimensional image data into a compact latent space while preserving essential visual structure. The learned latent representations were subsequently used as input features for a supervised classifier.

The autoencoder successfully reduced the input dimensionality from 784 to 32 while maintaining reconstruction fidelity, indicating effective feature extraction. Training results demonstrated stable convergence, confirming that the compressed representations retained meaningful information about digit shape and structure. When used for classification, these latent features enabled the classifier to achieve a test accuracy of 0.8484, showing that strong dimensionality reduction can be achieved without severe degradation in classification performance.

Visualization of the latent space using t-SNE revealed clear clustering patterns corresponding to digit classes, with limited overlap among visually similar digits. Qualitative inspection of random predictions further illustrated the model's confidence behavior and its handling of ambiguous handwritten samples.

Lab 14 – MINST Image Classification with a Convolutional Neural Network (CNN)

Objective

The objective of this laboratory session is to design, train, and evaluate a Convolutional Neural Network (CNN) for handwritten digit recognition using the MNIST dataset. The experiment focuses on end-to-end learning, where feature extraction and classification are learned jointly from raw pixel input.

The specific goals of this lab are to:

- Prepare MNIST grayscale images for convolutional processing by normalization and reshaping.
- Build a CNN architecture using convolution, pooling, flattening, and dense layers in Keras.
- Train the model using categorical cross-entropy and monitor learning using a validation split.
- Evaluate model generalization performance on an independent test set.
- Interpret the role of each architectural component in hierarchical feature learning.

Code

The Convolutional Neuron Network Code used in this experiment is available at the following GitHub repository: Convolutional Neuron Network Code

Implementation and Analysis

Data loading and preprocessing

The MNIST dataset was loaded using *keras.datasets.mnist.load_data()*, providing 60,000 training images and 10,000 test images of handwritten digits. Pixel intensities were converted to floating point values and normalized to the range [0, 1] to improve numerical stability during optimization. Each image was reshaped to (28, 28, 1) by adding a channel dimension, allowing the convolutional layers to process grayscale images correctly. Digit labels were converted to one-hot encoded vectors to match the categorical cross-entropy loss function used during training.

CNN architecture design

A Convolutional Neural Network was constructed using a Sequential architecture composed of two convolution–pooling blocks followed by a classification stage. The first convolution layer applies 32 filters of size 3×3 to capture low-level features such as edges and stroke transitions. Max pooling reduces spatial resolution and introduces limited translation invariance. The second

convolution layer increases the number of filters to 64, enabling the network to learn more complex digit structures and spatial relationships.

After spatial feature extraction, the feature maps were flattened into a one-dimensional vector and passed through a dropout layer with a rate of 0.5 to reduce overfitting. The final dense layer uses softmax activation to output class probabilities for the ten-digit classes. The resulting architecture contains 34,826 trainable parameters, providing sufficient capacity for MNIST classification while remaining computationally efficient.

Training configuration and learning behavior

The model was trained using categorical cross-entropy loss and the Adam optimizer, which adapts learning rates during training to improve convergence. Training was performed for 15 epochs with a batch size of 128. A validation split of 10% was used to monitor generalization performance during training.

Training accuracy increased rapidly during the initial epochs and stabilized near 98.9%, while validation accuracy reached approximately 99.2%. Validation loss remained low and stable, indicating that the dropout layer effectively limited overfitting and that the learned representations generalized well beyond the training data.

After training, the model was evaluated on the independent test set to assess final performance. Test accuracy of 0.9904 and Test loss of 0.0271 was obtained. These results demonstrate that the CNN successfully learns hierarchical spatial features directly from raw pixel input and achieves high recognition accuracy on handwritten digits. The close alignment between validation and test accuracy confirms stable generalization and validates the effectiveness of convolutional architectures for image classification tasks

Conclusion

This laboratory demonstrated the effectiveness of Convolutional Neural Networks for handwritten digit classification using the MNIST dataset. By learning features directly from raw pixel input, the CNN eliminated the need for explicit feature engineering and achieved strong classification performance through hierarchical representation learning.

The implemented architecture, consisting of two convolution–pooling blocks followed by a dense classification layer, successfully captured both low-level and high-level spatial patterns present in handwritten digits. Training converged rapidly and remained stable due to appropriate normalization, the use of ReLU activation, and dropout-based regularization.

Compared to representation-based approaches relying on dimensionality reduction, the CNN benefits from end-to-end optimization and explicit modeling of spatial structure, resulting in superior classification accuracy. These results highlight the suitability of convolutional architectures for image recognition tasks and illustrate fundamental deep learning principles used in modern computer vision systems.

References

1. University of Orleans. (2025). Machine Learning Lab Manual Series (TPs): K-Means, DBSCAN, Hierarchical Clustering, Regression, KNN, Logistic Regression, Gradient Descent, and Perceptron. Department of Computer Science. Prepared by Prof. Prof. Frédéric Ros
2. McKinney, W. (2022). Python for Data Analysis. O'Reilly Media.
3. Mworia, V. Mwenda (2025). *Artificial Intelligence and Machine Learning Lab Codes (University of Orléans)*. GitHub repository. https://github.com/vincentmworia/uniorleans-class-artificial-intelligence-and-machine-learning